



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universidad Politécnica de Valencia

SeeQL: Plataforma Web Interactiva para el Aprendizaje de Bases de Datos y SQL

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Autor: Jesús Caravaca

Tutor: Beatriz Marín, Tanja Vos

Curso 2025-2026

Agradecimientos

A mi tutora, Beatriz Marín, por su inestimable guía y por contagiarme su entusiasmo por la docencia y la gamificación durante este proyecto.

A mis amigos de la UPV: Hugo Juan, Javier Moreno, Javier Tovar, Daniela Melean y África Muñoz. Todos los proyectos que hemos hecho juntos han sido una experiencia inolvidable. Esta etapa universitaria no habría sido lo mismo sin vosotros.

Y muy especialmente a mis padres, por su apoyo incondicional. Gracias por estar siempre ahí, os quiero. Este logro es tan vuestro como mío.

Resum

En l'era actual de la informació, la gestió i l'anàlisi de dades constitueixen un dels pilars fonamentals de la indústria del programari. El llenguatge SQL (*Structured Query Language*) s'ha consolidat com l'estàndard universal per a la interacció amb bases de dades relacionals, convertint-se en una competència tècnica imprescindible en molts perfils del sector tecnològic. Per aquest motiu, el seu ensenyament ocupa un lloc estratègic en els plans d'estudi acadèmics, impartint-se habitualment en les etapes inicials de les titulacions, com és el cas del segon curs del Grau en Enginyeria Informàtica.

No obstant això, a pesar de la seua rellevància, l'aprenentatge d'aquest llenguatge suposa sovint un desafiament per als estudiants novells, a causa del nivell d'abstracció necessari per a comprendre com les operacions lògiques transformen les dades. Les eines educatives tradicionals solen limitar-se a la validació sintàctica estàtica o manquen d'entorns segurs on experimentar lliurement sense risc d'alterar sistemes en producció.

Aquest treball de fi de grau presenta el disseny i desenvolupament de SeeQL, una plataforma web interactiva orientada a l'ensenyament visual de bases de dades. L'aplicació, construïda com una aplicació de pàgina única (SPA) amb React i TypeScript, implementa una arquitectura d'execució en el client que utilitza WebAssembly (a través de SQL.js) per a executar consultes de forma segura i ràpida i integra serveis al núvol (Supabase) de forma asíncrona per a garantir la persistència del progrés acadèmic de l'estudiant.

La plataforma s'estructura en dos mòduls principals: un itinerari docent gamificat dividit en nivells de dificultat progressiva, on l'usuari rep retroalimentació visual immediata; i un mode de lliure experimentació, concebut com un laboratori professional que permet la manipulació de registres i l'execució de consultes sense restriccions funcionals.

Per a validar l'eficàcia del sistema, s'ha dut a terme un procés d'avaluació en el qual han participat tant estudiants com docents. L'anàlisi de les dades recollides confirma que la visualització en tres panells i les pistes d'ajuda integrades redueixen significativament la càrrega cognitiva de l'alumnat. Com a conclusió, SeeQL es consolida com una eina versàtil que combina la guia teòrica amb la llibertat pràctica, demostrant la seua viabilitat com a recurs didàctic eficaç per a facilitar una comprensió profunda i tangible del model relacional.

Paraules clau: SQL, Bases de dades, React, WebAssembly, Gamificació, E-learning

Resumen

En la era actual de la información, la gestión y el análisis de datos constituyen uno de los pilares fundamentales de la industria del software. El lenguaje SQL (*Structured Query Language*) se ha consolidado como el estándar universal para la interacción con bases de datos relacionales, convirtiéndose en una competencia técnica imprescindible en muchos perfiles del sector tecnológico. Por este motivo, su enseñanza ocupa un lugar estratégico en los planes de estudio académicos, impartándose habitualmente en las etapas iniciales de las titulaciones, como es el caso del segundo curso del Grado en Ingeniería Informática.

Sin embargo, a pesar de su relevancia, el aprendizaje de este lenguaje supone a menudo un desafío para los estudiantes novatos, debido al nivel de abstracción necesario para comprender cómo las operaciones lógicas transforman los datos. Las herramientas educativas tradicionales suelen limitarse a la validación sintáctica estática o carecen de entornos seguros donde experimentar libremente sin riesgo de alterar sistemas en producción.

Este trabajo de fin de grado presenta el diseño y desarrollo de SeeQL, una plataforma web interactiva orientada a la enseñanza visual de bases de datos. La aplicación, construida como una aplicación de página única (SPA) con React y TypeScript, implementa una arquitectura de ejecución en el cliente que utiliza WebAssembly (a través de SQL.js) para ejecutar consultas de forma segura y rápida, e integra servicios en la nube (Supabase) de forma asíncrona para garantizar la persistencia del progreso académico del estudiante.

La plataforma se estructura en dos módulos principales: un itinerario docente gamificado dividido en niveles de dificultad progresiva, donde el usuario recibe retroalimentación visual inmediata; y un modo de libre experimentación, concebido como un laboratorio profesional que permite la manipulación de registros y la ejecución de consultas sin restricciones funcionales.

Para validar la eficacia del sistema, se ha llevado a cabo un proceso de evaluación en el que han participado tanto estudiantes como docentes. El análisis de los datos recabados confirma que la visualización en tres paneles y las pistas de ayuda integradas reducen significativamente la carga cognitiva del alumnado. Como conclusión, SeeQL se consolida como una herramienta versátil que combina la guía teórica con la libertad práctica, demostrando su viabilidad como recurso didáctico eficaz para facilitar una comprensión profunda y tangible del modelo relacional.

Palabras clave: SQL, Bases de datos, React, WebAssembly, Gamificación, E-learning

Abstract

In the current information age, data management and analysis constitute one of the fundamental pillars of the software industry. SQL (*Structured Query Language*) has established itself as the universal standard for interacting with relational databases, becoming an essential technical skill across many profiles in the tech sector. For this reason, its teaching occupies a strategic place in academic curricula, usually being taught in the early stages of degree programs, as is the case in the second year of the Bachelor's Degree in Computer Engineering.

However, despite its relevance, learning this language often poses a challenge for novice students due to the level of abstraction required to understand how logical operations transform data. Traditional educational tools are often limited to static syntactic validation or lack safe environments to experiment freely without the risk of altering production systems.

This bachelor's thesis presents the design and development of SeeQL, an interactive web platform oriented towards the visual teaching of databases. The application, built as a single-page application (SPA) with React and TypeScript, implements a client-side execution architecture that uses WebAssembly (through SQL.js) to execute queries safely and quickly, and integrates cloud services (Supabase) asynchronously to ensure the persistence of the student's academic progress.

The platform is structured into two main modules: a gamified learning path divided into levels of progressive difficulty, where the user receives immediate visual feedback; and a free experimentation mode, conceived as a professional laboratory that allows the manipulation of records and the execution of queries without functional restrictions.

To validate the system's effectiveness, an evaluation process has been carried out with the participation of both students and teachers. The analysis of the gathered data confirms that the three-panel visualization and the integrated help hints significantly reduce the students' cognitive load. In conclusion, SeeQL consolidates itself as a versatile tool that combines theoretical guidance with practical freedom, demonstrating its viability as

an effective educational resource to facilitate a deep and tangible understanding of the relational model.

Key words: SQL, Databases, React, WebAssembly, Gamification, E-learning

Índice general

Agradecimientos	III
Índice general	VII
Índice de figuras	XI
Índice de tablas	XII
1 Introducción	1
1.1 Planteamiento del problema y motivación	1
1.2 Objetivos	2
1.2.1 Objetivo general	2
1.2.2 Objetivos específicos	3
1.3 Metodología	3
1.4 Arquitectura de la solución propuesta	4
1.4.1 Diagrama de componentes	4
1.4.2 Descripción de módulos	5
1.5 Relación del trabajo desarrollado con los estudios cursados	5
1.6 Estructura de la memoria	6
2 Estado del arte	7
2.1 Gamificación y juegos serios	7
2.2 Lenguajes de consulta visuales	7
2.3 Enfoques basados en datos y tareas	8
2.4 Nuevas tendencias y el impacto de la IA generativa	9
2.5 Conclusión y posicionamiento de SeeQL	9
2.6 Comparativa de soluciones	9
3 Análisis y especificación de requisitos	11
3.1 Requisitos del sistema	11
3.1.1 Requisitos funcionales	11
3.1.2 Requisitos no funcionales	13
3.2 Actores y casos de uso	13
3.2.1 Actores del sistema	13
3.2.2 Casos de uso principales	13
4 Diseño del sistema	17
4.1 Arquitectura del sistema	17
4.1.1 Vista estructural: Arquitectura por capas	17
4.1.2 Vista dinámica: Flujo de ejecución y validación temprana	19
4.1.3 Arquitectura de seguridad por aislamiento	20
4.2 Stack tecnológico	20
4.3 Modelado de datos	20
4.3.1 Esquema de la aplicación	21
4.3.2 Esquemas educativos (semillas de datos)	21
4.4 Diseño de la interfaz de usuario	23
4.4.1 Componentes visuales: Estructura y datos	24
4.4.2 Especialización por modo de uso	24

4.4.3	Prototipado de alta fidelidad	24
4.5	Diseño del algoritmo de validación	27
4.5.1	Estrategia de mitigación de falsos positivos	28
4.5.2	Sistema de <i>feedback</i> y orientación pedagógica	28
4.6	Diseño del itinerario pedagógico	28
4.6.1	Lección 1: SELECT y FROM	29
4.6.2	Lección 2: WHERE	30
4.6.3	Lección 3: AND, OR, IN	31
4.6.4	Lección 4: ORDER BY y LIMIT	32
4.6.5	Lección 5: LIKE	33
4.6.6	Lección 6: Agregaciones (SUM, COUNT, AVG)	34
4.6.7	Lección 7: GROUP BY	34
4.6.8	Lección 8: INNER JOIN	35
4.6.9	Lección 9: HAVING	36
4.6.10	Lección 10: LEFT JOIN	37
5	Implementación	39
5.1	Configuración del entorno y arquitectura <i>frontend</i>	39
5.2	Integración del motor SQL en el cliente	39
5.3	Persistencia y modelado de datos	40
5.4	Procesamiento y análisis de consultas (SQL parser)	40
5.4.1	Módulo de intercepción y traducción pedagógica de errores	41
5.4.2	Orquestación del estado gráfico: El <i>hook</i> useSqlVisualizer	42
5.5	Motor de visualización gráfica	44
5.5.1	Evaluación lógica y resaltado dinámico	44
5.5.2	Representación de operaciones avanzadas	45
5.5.3	Interactividad espacial y navegación en el lienzo	46
5.5.4	Casos de uso visuales: Representación de operaciones complejas	46
5.6	Lógica de gamificación y motor de lecciones	48
5.7	Desarrollo de la interfaz de usuario	49
5.7.1	El espacio de trabajo trilateral	50
5.7.2	Psicología del color y semántica visual	51
5.7.3	Layout relacional inteligente	52
5.7.4	Mejoras de experiencia de usuario e interacciones avanzadas	52
5.8	Sistema de autenticación y gestión de sesión	55
5.8.1	Autenticación multicanal	55
5.8.2	Modo invitado	55
5.8.3	<i>Feedback</i> en tiempo real y prevención de errores	55
5.9	Evolución arquitectónica y optimización del rendimiento	56
5.10	Despliegue y entorno de producción	56
6	Evaluación y pruebas	59
6.1	Validación técnica del software	59
6.1.1	<i>Testing</i> unitario	59
6.1.2	Pruebas de integración	60
6.1.3	Pruebas de aceptación y <i>smoke testing</i>	60
6.2	Metodología de evaluación con usuarios	61
6.2.1	Diseño del instrumento: Perfil estudiante	61
6.2.2	Diseño del instrumento: Perfil docente (validación experta)	62
6.3	Análisis de resultados: Evaluación de estudiantes	62
6.3.1	Perfil demográfico y conocimientos previos	63
6.3.2	Evaluación de dimensiones de usabilidad y diseño	63
6.3.3	Interacción pedagógica: El uso de ayudas	64
6.3.4	Fidelización y viabilidad futura	65

6.4	Análisis de resultados: Perfil docente	66
6.4.1	Valoración cuantitativa de expertos	66
6.4.2	Análisis cualitativo del <i>feedback</i>	67
7	Conclusiones y trabajo futuro	69
7.1	Cumplimiento de objetivos	69
7.2	Trabajo futuro	69
7.3	Reflexión personal	70
	Bibliografía	73
A	Esquema de datos y scripts SQL	77
A.1	Esquema educativo 1: Biblioteca (dificultad básica)	77
A.2	Esquema educativo 2: Gimnasio (dificultad media)	79
A.3	Esquema educativo 3: Aeropuerto (dificultad avanzada)	80
B	Instrumentos de evaluación	83
B.1	Cuestionario para estudiantes	83
B.2	Cuestionario para docentes (validación experta)	84
C	Manual de instalación	87
C.1	Requisitos previos	87
C.2	Clonación del repositorio	87
C.3	Instalación de dependencias	87
C.4	Configuración de variables de entorno	88
C.5	Ejecución en entorno de desarrollo	88
C.6	Compilación para producción	88
C.7	Consideraciones sobre WebAssembly	88
D	Alineación con los Objetivos de Desarrollo Sostenible	89
E	Licencia de software	91

Índice de figuras

1.1	Arquitectura de alto nivel: Separación de responsabilidades entre cliente y nube.	4
3.1	Diagrama de casos de uso de SeeQL.	15
4.1	Arquitectura de SeeQL: Estructura de capas y modelo de ejecución en el cliente.	18
4.2	Diagrama de secuencia del flujo de procesamiento local de una consulta SQL.	19
4.3	Modelo relacional - Esquema educativo: Biblioteca (SeeQL).	21
4.4	Modelo relacional - Esquema educativo: Gimnasio (SeeQL).	22
4.5	Modelo relacional - Esquema educativo: Aeropuerto (SeeQL).	23
4.6	Prototipo de alta fidelidad: Página de aterrizaje y acceso.	25
4.7	Prototipo de alta fidelidad: Dashboard de selección de modo.	26
4.8	Prototipo de alta fidelidad: Menú de progresión de niveles.	26
4.9	Prototipo de alta fidelidad: Interfaz trilateral en modo Lecciones.	27
4.10	Prototipo de alta fidelidad: Interfaz optimizada en modo <i>Sandbox</i>	27
5.1	Efecto de iluminación dinámica sobre los registros que cumplen la condición de filtrado.	47
5.2	Representación visual de un INNER JOIN destacando la relación entre claves.	47
5.3	Generación dinámica de una tabla virtual tras aplicar una función de agregación.	48
5.4	Estado del módulo de lecciones con progresión lineal y desbloqueo secuencial de retos.	49
5.5	<i>Dashboard</i> principal de navegación y selección de esquemas de SeeQL.	49
5.6	Interfaz del modo Lecciones con panel de narrativa integrado.	50
5.7	Interfaz del modo <i>Sandbox</i> , optimizada para experimentación.	51
5.8	Interfaz de la consola de resultados ante un error de sintaxis.	54
5.9	<i>Feedback</i> pedagógico ante un error lógico (datos no coincidentes).	54
5.10	Representación visual de la consola tras una validación exitosa del reto.	54
6.1	Distribución de los participantes según su nivel previo de conocimientos en SQL.	63
6.2	Evaluación media de las dimensiones de usabilidad y diseño del sistema (Escala 1-5).	64
6.3	Relación entre el nivel previo de SQL y el uso de las pistas integradas en la interfaz.	65
6.4	Intención de los usuarios de seguir utilizando la plataforma para aprender SQL.	66
6.5	Evaluación de dimensiones pedagógicas y de usabilidad por parte de docentes.	67

Índice de tablas

2.1	Comparativa funcional de SeeQL frente a las soluciones analizadas en el estado del arte.	10
3.1	Requisitos funcionales de SeeQL.	12
3.2	Requisitos no funcionales de SeeQL.	13
D.1	Matriz de impacto de SeeQL en los Objetivos de Desarrollo Sostenible (Agenda 2030).	89

CAPÍTULO 1

Introducción

El diseño curricular de los grados en Ingeniería Informática enfrenta constantemente el reto de alinear la fuerte abstracción de los fundamentos teóricos con las exigencias prácticas del desarrollo profesional [1, 2]. El lenguaje de consulta estructurado (SQL) ilustra perfectamente este desafío: aunque se mantiene como el estándar universal para la manipulación de bases de datos relacionales y presenta una sintaxis aparentemente cercana al lenguaje natural, su aprendizaje inicial supone una barrera cognitiva significativa para los estudiantes [3]. Esta dificultad radica, en gran medida, en la carencia de herramientas pedagógicas modernas que logren hacer la transición desde la teoría abstracta hacia una visualización clara, interactiva e inmediata de los datos subyacentes.

1.1 Planteamiento del problema y motivación

La motivación principal para el desarrollo de este proyecto nace de la observación directa de las fricciones y deficiencias presentes en las metodologías tradicionales de enseñanza de bases de datos. En el contexto específico del Grado en Ingeniería Informática de la Universitat Politècnica de València, la estructuración de la asignatura de Bases de Datos (BDA) [4] ilustra perfectamente esta problemática.

El plan de estudios de la asignatura BDA, fundamentado en el texto de referencia de Mota, Casamayor y Celma [5], guía al estudiante a través de un itinerario estructurado en tres grandes bloques. Este proceso plantea una curva de aprendizaje exigente, donde el estudiante debe asimilar en la primera unidad didáctica la fuerte abstracción matemática del modelo relacional y el álgebra relacional. En esta fase, los alumnos trabajan teóricamente con conceptos como tuplas, relaciones, operaciones de selección, proyección, productos cartesianos y restricciones de integridad.

Sin embargo, el verdadero reto pedagógico se intensifica al avanzar a la segunda unidad didáctica, centrada íntegramente en el lenguaje SQL, tanto en su faceta de lenguaje de manipulación de datos (DML) como de lenguaje de definición de datos (DDL). En esta etapa, el alumno debe materializar la teoría matemática aprendida previamente en sentencias reales dentro de un entorno de ejecución técnico. Habitualmente, este entorno se apoya en sistemas gestores de bases de datos (SGBD) corporativos a través de clientes de escritorio profesionales como Oracle SQL Developer. Al carecer de una capa pedagógica visual adaptada que conecte la abstracción matemática con la sintaxis del código, el estudiante suele invertir una cantidad desproporcionada de esfuerzo inicial en familiarizarse con el flujo operativo de herramientas empresariales, tales como la gestión manual de transacciones (*commits*), la navegación por menús densos de metadatos o la inserción ordenada de filas respetando la jerarquía de claves ajenas. Esta sobrecarga desvía la atención del objetivo principal: interiorizar el modelo mental de cómo las consultas,

tales como agrupaciones o combinaciones de tablas, transforman y filtran los conjuntos de datos en tiempo real.

Estudios recientes confirman esta tendencia, demostrando que los estudiantes no solo cometen errores de sintaxis, sino que enfrentan profundas complicaciones lógicas al intentar traducir requisitos en consultas estructuradas [6]. Estas limitaciones se agrupan en tres grandes dimensiones:

- **Abstracción teórica y falta de modelos mentales:** A diferencia de la programación imperativa, donde el flujo de ejecución es secuencial, SQL opera sobre la teoría de conjuntos. Resulta extremadamente complejo para un alumno novato construir un modelo mental preciso de cómo las operaciones del álgebra relacional, como las combinaciones de tablas (*JOIN*), las agrupaciones (*GROUP BY*) o las subconsultas, transforman y filtran las tuplas sin el apoyo de una representación gráfica del esquema conceptual en tiempo real.
- **Barreras técnicas y sobrecarga cognitiva inicial:** El aprendizaje efectivo de SQL requiere práctica constante, pero el acceso a esa práctica suele estar bloqueado por una ardua configuración preliminar. Instalar un SGBD local, configurar puertos, gestionar credenciales y poblar las tablas con datos de prueba genera frustración y desvía la atención del objetivo principal: dominar el lenguaje de consulta.
- **Falta de interactividad y retroalimentación demorada:** Gran parte de los ejercicios prácticos y plataformas educativas actuales se basan en validaciones estáticas o entregan resultados en formato de texto plano sin contexto del esquema original. La ausencia de un *feedback* formativo e inmediato impide que el estudiante comprenda el origen de sus violaciones de integridad o errores lógicos, limitando su capacidad para corregirlos de forma autónoma, un factor crítico en los entornos de aprendizaje digital [7].

Para dar respuesta a esta problemática, este trabajo de fin de grado (TFG) propone el diseño y desarrollo de SeeQL, una plataforma web interactiva orientada específicamente a la enseñanza y experimentación segura con bases de datos. Mediante una arquitectura de computación en el cliente (*client-side computing*), SeeQL elimina por completo las barreras de configuración al desplegar un motor relacional directamente en la memoria del navegador a través de un entorno de ejecución ligero y aislado.

La herramienta combina un itinerario pedagógico gamificado con una pizarra de visualización dinámica, permitiendo a los usuarios escribir código estándar y observar instantáneamente, de manera gráfica, el impacto lógico de sus instrucciones. De este modo, se transforma la abstracción matemática del álgebra relacional en una experiencia tangible, interactiva y accesible, alineándose perfectamente con las necesidades docentes del plan de estudios.

1.2 Objetivos

1.2.1. Objetivo general

El objetivo principal de este trabajo de fin de grado es el diseño, desarrollo y despliegue de SeeQL, una plataforma web interactiva y gamificada que facilite la asimilación teórica y práctica del modelo relacional y el lenguaje SQL. La herramienta busca reducir la brecha cognitiva, proporcionando un entorno de ejecución seguro en el navegador que ofrezca retroalimentación visual inmediata y garantice la persistencia del progreso académico en la nube.

1.2.2. Objetivos específicos

Para alcanzar la meta principal y dar cobertura a las necesidades técnicas y pedagógicas, el proyecto se desglosa en los siguientes hitos:

1. **Analizar el estado del arte** de las plataformas de *e-learning* para bases de datos, identificando las carencias actuales en la enseñanza visual de SQL.
2. **Diseñar e implementar una arquitectura web** basada en *client-side computing* eficiente utilizando React [8], TypeScript [9] y Supabase como *backend as a service* (BaaS) [10], separando claramente la lógica de presentación de la persistencia de datos.
3. **Construir un motor de ejecución aislada en el cliente** mediante SQL.js. Esto permitirá la ejecución de sentencias DML o DDL con latencia cero, ofreciendo un entorno seguro que elimina el riesgo de inyecciones SQL en servidores de producción.
4. **Desarrollar un motor gráfico interactivo** que traduzca el esquema lógico de la base de datos en diagramas comprensibles, visualizando dinámicamente las relaciones y las restricciones de integridad.
5. **Diseñar un itinerario pedagógico y un entorno *sandbox*** que guíe al estudiante desde consultas básicas hasta operaciones complejas de álgebra relacional, combinando ejercicios estructurados con la libertad de experimentación.
6. **Evaluar la usabilidad y el impacto pedagógico** de la herramienta, combinando pruebas funcionales técnicas con la realización de cuestionarios a estudiantes y docentes.

1.3 Metodología

Debido a que el marco temporal de un trabajo de fin de grado es limitado y requiere un equilibrio entre la investigación y el desarrollo técnico, se ha adoptado un enfoque orientado a la consecución de un **producto mínimo viable (MVP)**. Esta decisión estratégica permite priorizar las funcionalidades críticas para el aprendizaje del estudiante, como la ejecución local y la pizarra interactiva, postergando para futuras iteraciones los módulos administrativos complejos, como un panel de configuración personalizada para docentes.

Para el desarrollo de este proyecto, se ha seguido la metodología de **investigación-acción** (*action research*) [11]. Este enfoque es especialmente idóneo para el desarrollo de software educativo, ya que combina la construcción técnica con la reflexión crítica y la evaluación continua por parte de los usuarios, permitiendo adaptar la herramienta a las necesidades reales de aprendizaje.

El trabajo se ha estructurado en iteraciones que constan de cuatro fases:

1. **Planificación:** Identificación del problema pedagógico o técnico, como las dificultades del alumnado para interiorizar el concepto de *JOIN*.
2. **Acción:** Implementación de la solución técnica, como el renderizado del esquema visual en React.
3. **Observación:** Pruebas de funcionamiento, pruebas de usabilidad y detección de errores u obstáculos cognitivos.

4. **Reflexión:** Análisis de los resultados junto con los tutores para refinar la interacción y decidir los siguientes pasos del desarrollo.

De forma complementaria, se han mantenido reuniones periódicas de seguimiento para validar el estado del arte, refinar la arquitectura y evaluar el progreso del desarrollo.

Para la gestión técnica del ciclo de vida del software, el desarrollo se ha apoyado en prácticas ágiles y control de versiones distribuido mediante Git [12] y GitHub [13]. Esta infraestructura técnica ha permitido gestionar de forma ordenada la evolución del código fuente, garantizando la trazabilidad de los cambios y facilitando el desarrollo incremental de los componentes de la plataforma.

1.4 Arquitectura de la solución propuesta

El sistema SeeQL implementa una arquitectura de *single page application* (SPA), donde la carga computacional lógica se desplaza **íntegramente** al cliente. Este modelo integra la ejecución local del motor relacional con servicios en la nube dedicados exclusivamente a la persistencia de datos, garantizando una respuesta inmediata al eliminar la latencia de red y asegurando una alta disponibilidad del servicio.

1.4.1. Diagrama de componentes

La arquitectura se estructura en tres capas lógicas fundamentales que separan la interacción, el procesamiento y el almacenamiento. La Figura 1.1 presenta una visión simplificada de estos componentes y su ubicación física (local o remota).

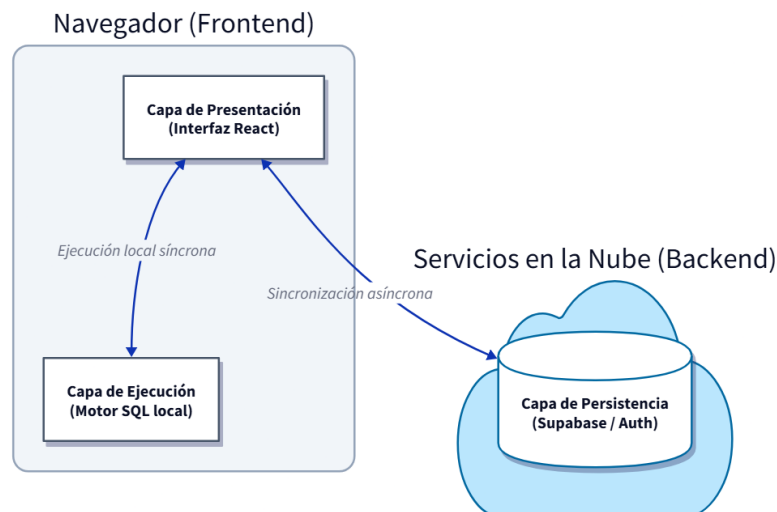


Figura 1.1: Arquitectura de alto nivel: Separación de responsabilidades entre cliente y nube.

1.4.2. Descripción de módulos

- **Capa de presentación (interfaz):** Desarrollada en React, gestiona la interacción con el usuario, el enrutamiento de las vistas y la visualización de los resultados. Actúa como el orquestador principal, capturando los eventos del editor de código y enviándolos al motor de ejecución.
- **Capa de ejecución (core):** Módulo aislado que contiene el motor SQL local compilado en WebAssembly. Recibe las cadenas de texto con las consultas, las procesa en un entorno de memoria virtual y devuelve los resultados en formato JSON. Este aislamiento garantiza que el procesamiento de datos masivos no bloquee el hilo principal del navegador.
- **Capa de persistencia (servicios en la nube):** Componente externo gestionado a través de Supabase. Se comunica con la aplicación mediante una API segura para autenticar a los usuarios, almacenar configuraciones y sincronizar el progreso del itinerario pedagógico.

El análisis y despliegue técnico de cada uno de estos módulos, junto con los algoritmos de validación sintáctica y los mecanismos de aislamiento en el cliente, se abordan en profundidad en el capítulo 5.

1.5 Relación del trabajo desarrollado con los estudios cursados

El desarrollo de SeeQL representa la culminación práctica de los estudios del Grado en Ingeniería Informática, permitiendo consolidar y unificar de manera transversal las competencias técnicas y metodológicas adquiridas a lo largo de la titulación, con especial énfasis en la intensificación de Ingeniería del Software.

El trabajo se fundamenta directamente en tres grandes pilares del plan de estudios:

- **Bases de Datos (BDA):** Esta asignatura ha constituido la motivación principal y la base teórica del proyecto. La comprensión del modelo relacional, el álgebra relacional y la sintaxis DDL/DML asimilados en esta materia han sido imprescindibles para poder diseñar el itinerario pedagógico, estructurar los esquemas de datos educativos y desarrollar el algoritmo lógico de validación de consultas.
- **Interfaces Persona Computador (IPC):** El estudio de esta disciplina supuso un punto de inflexión académico que despertó mi vocación por el desarrollo *frontend* y la experiencia de usuario. Los principios de usabilidad, accesibilidad y diseño centrado en el usuario asimilados en IPC se han aplicado directamente en la concepción del espacio de trabajo de SeeQL, la codificación semántica del color y las técnicas de reducción de la carga cognitiva.
- **Ingeniería de software y gestión de proyectos:** Asignaturas como Diseño de Software (DSW), Análisis y Especificación de Requisitos (AER) y, muy especialmente, Proyecto de Ingeniería de Software (PIN) proveyeron el marco metodológico necesario. La experiencia previa en la gestión de proyectos en equipo adquirida en estas materias ha sido vital para extrapolar esas dinámicas y poder liderar, en solitario, el ciclo de vida completo de un producto de software real: desde la toma de requisitos y el diseño de la arquitectura, hasta el *testing* automatizado y el despliegue de integración continua.

Puesta en práctica de competencias transversales (CT)

Más allá de los conocimientos puramente tecnológicos, el diseño e implementación de este proyecto ha requerido la aplicación intensiva de diversas competencias transversales evaluadas durante el grado:

- **Innovación y creatividad (CT-02):** Aplicada en la búsqueda de una solución novedosa al problema de la abstracción de SQL, transformando de forma creativa el árbol de sintaxis abstracta (AST) en un lienzo de nodos gráficos dinámicos e interactivos.
- **Responsabilidad y toma de decisiones (CT-05):** Ejercida de manera continua a nivel arquitectónico, destacando decisiones críticas fundamentadas empíricamente, como la migración de un modelo de servidor tradicional hacia una ejecución *client-side* mediante WebAssembly para reducir drásticamente el tiempo de respuesta.
- **Comunicación efectiva (CT-04):** Reflejada tanto en la redacción formal y académica de la presente memoria técnica, como en el esfuerzo pedagógico de diseñar un módulo traductor que convierta excepciones técnicas de bases de datos complejas en mensajes formativos amigables para el usuario.

1.6 Estructura de la memoria

El resto de la presente memoria se organiza en los siguientes capítulos:

- **Capítulo 2: Estado del arte.** Se analizan las soluciones educativas existentes en el mercado y se justifican las ventajas de la propuesta frente a ellas.
- **Capítulo 3: Análisis del sistema.** Se identifican los actores principales, se definen los casos de uso y se detallan los requisitos funcionales y no funcionales de la plataforma.
- **Capítulo 4: Diseño del sistema.** Se describe la arquitectura de software seleccionada, el modelado de la base de datos, el diseño de la interfaz de usuario y la estructuración del itinerario pedagógico.
- **Capítulo 5: Implementación.** Se exponen los aspectos técnicos más relevantes del desarrollo, incluyendo la integración del motor SQL en el cliente y el procesamiento dinámico del AST.
- **Capítulo 6: Evaluación y pruebas.** Se detallan las pruebas funcionales aplicadas al motor de ejecución y se analizan los resultados de usabilidad para validar el cumplimiento de los objetivos.
- **Capítulo 7: Conclusiones.** Finalmente, se recapitulan los resultados obtenidos, el grado de cumplimiento de los objetivos iniciales y se plantean las posibles líneas de trabajo futuro.

CAPÍTULO 2

Estado del arte

El aprendizaje del lenguaje SQL ha sido objeto de múltiples investigaciones en la última década, dada su importancia crítica en el currículo de los grados en Ingeniería Informática e Ingeniería de Software. En este capítulo se clasifican y analizan las propuestas existentes en tres grandes categorías: enfoques gamificados, lenguajes de consulta visuales y enfoques basados en datos. El objetivo es identificar las limitaciones actuales que motivan el desarrollo de la plataforma SeeQL.

2.1 Gamificación y juegos serios

Una de las tendencias más destacadas para reducir la fricción en el aprendizaje es la introducción de dinámicas de juego. La gamificación en la ingeniería de software busca mejorar la motivación y la retención de conocimientos complejos [14, 15]. Investigaciones empíricas demuestran que estas técnicas incrementan el compromiso del estudiante al transformar ejercicios repetitivos en retos con recompensas [16].

En esta línea, existen propuestas populares como **SQL Island** o **SQL Murder Mystery**. Estas herramientas son efectivas para despertar el interés inicial mediante narrativas de misterio o aventura [17]. Sin embargo, tras el análisis realizado en este TFG, se observa que presentan una brecha pedagógica importante, ya que no ofrecen una interfaz que se asemeje a un entorno de desarrollo profesional. Esta carencia penaliza la transferibilidad de habilidades, ya que el alumno se acostumbra a interfaces de texto narrativas en lugar de interiorizar la distribución espacial de herramientas empresariales.

Por otra parte, carecen de un sistema de niveles progresivo y estructurado, lo que puede provocar sobrecarga cognitiva, ya que exigen al estudiante combinar múltiples conceptos relacionales complejos de forma simultánea sin un soporte cognitivo previo que afiance las operaciones fundamentales. Además, al ofrecer una validación puramente binaria sin retroalimentación formativa ante los fallos lógicos, permite al estudiante modificar el código basándose en aproximaciones sucesivas para superar el reto, sin llegar a asimilar la semántica real de la instrucción.

2.2 Lenguajes de consulta visuales

Los lenguajes de consulta visuales (*visual query languages*, VQL) están diseñados para facilitar la extracción de datos mediante representaciones gráficas interactivas, como arrastrar y soltar bloques o conectar nodos, eliminando por completo la barrera de la sin-

taxis estricta [18]. Un ejemplo académico reciente es **SQLGo** [19], fundamentado en la conexión gráfica de entidades.

No obstante, la evaluación de herramientas puramente visuales como **VisualSQL** demuestra que este enfoque adolece de dos problemas críticos en el ámbito de la ingeniería superior. En primer lugar, presentan una **baja escalabilidad visual**, ya que al intentar modelar consultas avanzadas que involucren múltiples combinaciones de tablas, agrupaciones o subconsultas, la interfaz de nodos se vuelve extremadamente confusa y difícil de gestionar.

En segundo lugar, generan una **dependencia gráfica que penaliza la competencia técnica**. Al aislar al estudiante del código fuente, este no desarrolla la memoria sintáctica necesaria ni aprende a depurar errores nativos del motor, creando una brecha significativa respecto a las herramientas de línea de comandos o entornos de desarrollo que utilizará en el mundo laboral.

SeeQL soluciona este conflicto adoptando un enfoque híbrido: exige la escritura de código SQL estándar para garantizar la adquisición de competencias profesionales, pero mantiene el soporte visual del esquema en tiempo real como método de validación espacial y desahogo cognitivo.

2.3 Enfoques basados en datos y tareas

La validación automatizada de consultas SQL se ha abordado históricamente desde frentes teóricos muy diversos, evolucionando desde los primeros evaluadores estáticos académicos hacia los sistemas actuales de validación dinámica o coincidencia de resultados (*output matching*). SeeQL adopta este último enfoque, validando que el conjunto de datos devuelto por el alumno coincida exactamente con la consulta solución mediante el uso de bases de datos locales provistas de casos de prueba frontera.

Dentro de las herramientas accesibles en el mercado que implementan la validación dinámica basadas en la consecución de tareas concretas, se identifican dos grandes vertientes con limitaciones pedagógicas claras:

- **Entornos de instrucción guiada:** Plataformas de referencia como **SQLBolt** destacan por un diseño instruccional muy bien estructurado en niveles. Sin embargo, adolecen de una grave falta de contexto visual espacial. El alumno trabaja sobre nombres de columnas proporcionados de forma estática en texto plano por el enunciado, pero carece de un lienzo interactivo que grafique el modelo relacional o las restricciones de integridad entre tablas. Esta **opacidad estructural** dificulta enormemente la creación de modelos mentales sólidos sobre cómo se conectan los datos entre sí.
- **Plataformas de programación competitiva:** Entornos masivos de evaluación técnica como **HackerRank** o **LeetCode** ofrecen interfaces potentes similares a un entorno de desarrollo integrado (IDE) profesional y una batería rígida de pruebas de datos. No obstante, están concebidos exclusivamente para la evaluación laboral y no para el aprendizaje: carecen por completo de andamiaje pedagógico, no ofrecen representaciones gráficas del esquema y su retroalimentación ante errores lógicos o de sintaxis es nula o críptica, limitándose a emitir mensajes del compilador que frustran y bloquean al estudiante.

SeeQL se posiciona en el centro de ambas vertientes, resolviendo de manera directa el aislamiento visual de estas plataformas al integrar una pizarra gráfica interactiva en el propio flujo de trabajo que actúa como soporte cognitivo en tiempo real.

2.4 Nuevas tendencias y el impacto de la IA generativa

La irrupción de los modelos de lenguaje de gran tamaño (LLM) está transformando la educación [20]. Sin embargo, confiar ciegamente en herramientas de inteligencia artificial (IA) puede ser contraproducente, ya que los alumnos tienden a aceptar soluciones con inferencias lógicamente incorrectas sin aplicar un análisis crítico [21]. SeeQL responde a esto permitiendo al alumno auditar visualmente el código generado por la IA para comprender si la estructura resultante coincide con el objetivo pedagógico.

2.5 Conclusión y posicionamiento de SeeQL

Tras el análisis del estado del arte, se han identificado tres grandes limitaciones estructurales en las soluciones actuales:

- **Priorización del enfoque narrativo:** Herramientas como SQL Island que sacrifican el rigor técnico y la interfaz profesional en favor de la narrativa lúdica.
- **Ceguera de esquema y falta de andamiaje:** Plataformas como SQLBolt o HackerRank que carecen de una representación gráfica interactiva del modelo relacional o de un itinerario tutorizado ante los errores lógicos.
- **Limitación de escalabilidad visual:** Interfaces VQL que se vuelven inmanejables en consultas complejas y no fomentan la escritura de código real.

El posicionamiento estratégico de SeeQL consiste en unificar las fortalezas de estos dominios: ofrece el rigor y realismo de un IDE profesional presente en HackerRank; el andamiaje pedagógico de los validadores estructurados por niveles como SQLBolt; y la retroalimentación visual inmediata propia de los VQL, integrados de forma que operan nativamente en una arquitectura de ejecución en el cliente.

Para dotar a esta plataforma de un valor pedagógico diferencial, SeeQL integra un sistema de *andamiaje* (*scaffolding*) fundamentado en la teoría de la zona de desarrollo próximo de Vygotsky, tomando como referencia el marco conceptual de Álvarez y Del Río [22]. Bajo este paradigma, la herramienta actúa como un par más experto que proporciona soporte temporal a través de pistas contextuales, validación asíncrona y retroalimentación adaptativa, permitiendo al estudiante resolver consultas de complejidad creciente que, sin este soporte estructurado, excederían su capacidad actual de resolución autónoma.

2.6 Comparativa de soluciones

La Tabla 2.1 resume el análisis comparativo de la propuesta frente a las tecnologías evaluadas en este capítulo.

Criterio de evaluación	SQL Island	SQLBolt	HackerRank	VQL	SeeQL
Uso de sintaxis SQL estándar	✓	✓	✓	–	✓
Visualización dinámica del esquema	–	–	–	✓	✓
Simulación de entorno IDE	–	–	✓	–	✓
Progresión por niveles estructurados	–	✓	–	–	✓
Escalabilidad ante consultas complejas	Media	Alta	Alta	Baja	Alta
Resistencia a alucinaciones de IA	Baja	Baja	Media	Media	Alta

Tabla 2.1: Comparativa funcional de SeeQL frente a las soluciones analizadas en el estado del arte.

CAPÍTULO 3

Análisis y especificación de requisitos

El análisis que se presenta a continuación se centra en este MVP, garantizando que el diseño del sistema responda con robustez a las interacciones esenciales del alumnado antes de escalar hacia una plataforma de gestión académica integral.

Tras justificar la necesidad de una nueva herramienta pedagógica en el capítulo anterior, el presente capítulo aborda el análisis detallado de la plataforma SeeQL. El objetivo principal de esta fase es delimitar el alcance del proyecto y traducir los objetivos didácticos y técnicos iniciales en especificaciones concretas de desarrollo.

Para lograr este propósito, el capítulo se estructura en dos grandes bloques de especificación analítica. En primer lugar, se define el catálogo completo de requisitos del sistema, abarcando tanto las exigencias funcionales que determinan el comportamiento del software, como las restricciones no funcionales que garantizan su rendimiento y seguridad. Posteriormente, se identifican los actores que interactuarán con el sistema y se modelan los casos de uso principales, proporcionando una visión clara de las interacciones esperadas entre el estudiante, el entorno de ejecución y la base de datos visual.

3.1 Requisitos del sistema

El sistema SeeQL se ha diseñado para estructurar su propuesta pedagógica en torno a dos modalidades operativas principales: el **modo Lecciones**, orientado al aprendizaje estructurado mediante un itinerario de retos progresivos, y el **modo Sandbox**, concebido como un entorno de experimentación libre sobre bases de datos preconfiguradas. Además, se han incorporado funcionalidades de usabilidad avanzadas destinadas a optimizar la experiencia de usuario y agilizar el proceso de asimilación de competencias del estudiante.

3.1.1. Requisitos funcionales

Los requisitos funcionales describen los comportamientos específicos del sistema. Se han priorizado utilizando la escala MoSCoW (*Must, Should, Could, Won't*) [23], simplificada en Alta, Media y Baja.

Tabla 3.1: Requisitos funcionales de SeeQL.

ID	Descripción	Prioridad
Gestión de usuarios y acceso		
RF-01	El sistema permitirá el acceso a usuarios invitados (sin registro) al modo <i>Sandbox</i> con bases de datos predefinidas.	Alta
RF-02	El sistema permitirá el registro e inicio de sesión de usuarios para almacenar y sincronizar el progreso académico en la nube.	Alta
Aprendizaje y feedback (lecciones)		
RF-03	El sistema presentará ejercicios organizados linealmente, bloqueando los niveles superiores hasta completar los anteriores.	Alta
RF-04	El sistema validará la consulta SQL introducida y determinará si el resultado coincide con la solución esperada.	Alta
RF-05	Traductor de errores: El sistema interceptará los mensajes de error técnicos del motor SQL nativo para transformarlos en explicaciones comprensibles y accesibles para el estudiante.	Media
RF-06	El sistema ofrecerá pistas visuales o textuales tras varios intentos fallidos.	Baja
Entorno de trabajo y herramientas		
RF-07	Modo <i>Sandbox</i>: El usuario podrá escribir y ejecutar consultas SQL arbitrarias libremente.	Alta
RF-08	Visualización dinámica: El sistema generará una representación gráfica interactiva del modelo relacional (esquema) y de las tablas de datos resultantes en tiempo real.	Alta
RF-09	Exportación: El usuario podrá descargar los resultados de sus consultas en formato CSV, JSON o la base de datos resultante.	Baja
RF-10	Personalización: La interfaz incluirá un selector de tema visual para reducir la fatiga visual.	Baja
RF-11	Reinicio del ejercicio: El sistema permitirá restaurar el editor de código y el entorno de trabajo a su estado predeterminado para el nivel actual, facilitando la rectificación autónoma del estudiante.	Alta
RF-12	Indicadores de progreso: La interfaz mostrará una barra de porcentaje que refleje el avance del usuario en el itinerario de lecciones.	Media

3.1.2. Requisitos no funcionales

Los requisitos no funcionales establecen las restricciones técnicas y de calidad del sistema.

ID	Categoría	Criterio de calidad
RNF-01	Accesibilidad	La aplicación será accesible vía navegadores web estándar sin instalaciones locales.
RNF-02	Usabilidad	Interfaz <i>desktop-first</i> , optimizada para resoluciones $\geq 1024\text{px}$.
RNF-03	Compatibilidad	El sistema requiere un navegador moderno con soporte nativo para ejecución de WebAssembly y ES6+.
RNF-04	Persistencia	Los datos de progreso se almacenarán permanentemente en una base de datos relacional.
RNF-05	Rendimiento	El tiempo de respuesta para ejecución y visualización debe ser inferior a 3 segundos.
RNF-06	Mantenibilidad	La definición de ejercicios y soluciones estará desacoplada del código fuente para facilitar la ampliación de contenidos.

Tabla 3.2: Requisitos no funcionales de SeeQL.

3.2 Actores y casos de uso

Para definir la interacción con el sistema, se han identificado los roles que intervienen en la aplicación. Dado que SeeQL opera como una SPA con ejecución principal en el lado del cliente, la interacción recae sobre el usuario final, delegando en servicios externos únicamente la persistencia de datos y la autenticación.

3.2.1. Actores del sistema

Se distinguen dos actores principales con una relación de herencia (el usuario registrado hereda las capacidades del invitado):

- **Estudiante invitado:** Usuario que accede a la plataforma de forma anónima sin necesidad de introducir credenciales. Su flujo de interacción se procesa íntegramente en el *frontend*, permitiéndole utilizar el modo *Sandbox* sobre instancias de bases de datos efímeras, visualizar diagramas de tablas y ejecutar consultas libres. Este rol carece de persistencia de datos entre sesiones independientes.
- **Estudiante registrado:** Usuario autenticado en el sistema mediante correo o un proveedor OAuth. Además de las funcionalidades del invitado, se comunica asíncronamente con el *backend* para almacenar de forma permanente su avance dentro del modo Lecciones y mantener la consistencia de su progreso académico.

3.2.2. Casos de uso principales

Los casos de uso describen las metas que los actores pueden alcanzar utilizando el sistema, siguiendo el estándar de modelado del *Unified Modeling Language* (UML) [24]. A continuación, se detallan los más relevantes para el MVP:

- **CU-01 Consultar teoría y enunciado:** (*Exclusivo para estudiante registrado*). El estudiante accede al panel lateral de una lección para leer la explicación teórica, el enunciado del problema y las ayudas de sintaxis, pudiendo colapsar el panel para optimizar su espacio visual.
- **CU-02 Interactuar con el lienzo gráfico:** (*Disponible para todos*). El usuario explora la pizarra de React Flow, realizando zoom, arrastrando nodos de tablas virtuales generadas dinámicamente y utilizando botones de auto-centrado.
- **CU-03 Escribir y ejecutar consulta:** (*Disponible para todos*). El usuario utiliza el editor integrado para formular instrucciones SQL e interactuar con el motor local (aplicable tanto en el modo Lecciones como en el modo *Sandbox*).
- **CU-04 Validar ejercicio:** (*Exclusivo para estudiante registrado*). El sistema evalúa la consulta del alumno frente a la solución esperada. Si es correcta, permite avanzar o seguir explorando libremente.
- **CU-05 Analizar *feedback* de error:** (*Disponible para todos*). Al introducir una consulta sintácticamente incorrecta, el usuario recibe un mensaje de error pedagógico traducido al español, en lugar del código de error bruto del motor de base de datos.
- **CU-06 Experimentar en el modo *Sandbox*:** (*Disponible para todos*). El usuario accede a un entorno de laboratorio no evaluado donde puede alternar libremente entre los diferentes esquemas educativos para probar consultas estructurales y lógicas sin la restricción de un enunciado concreto.
- **CU-07 Gestionar sesión:** (*Disponible para todos*). El usuario invitado se registra o inicia sesión para habilitar las funciones de identidad, y el usuario registrado la cierra para salir del sistema.
- **CU-08 Sincronizar progreso:** (*Exclusivo para estudiante registrado*). Al superar una lección, el sistema envía automáticamente una petición a la base de datos externa para registrar el nivel como completado.

La Figura 3.1 muestra el diagrama general de casos de uso, destacando la interacción de ambos actores con el sistema.

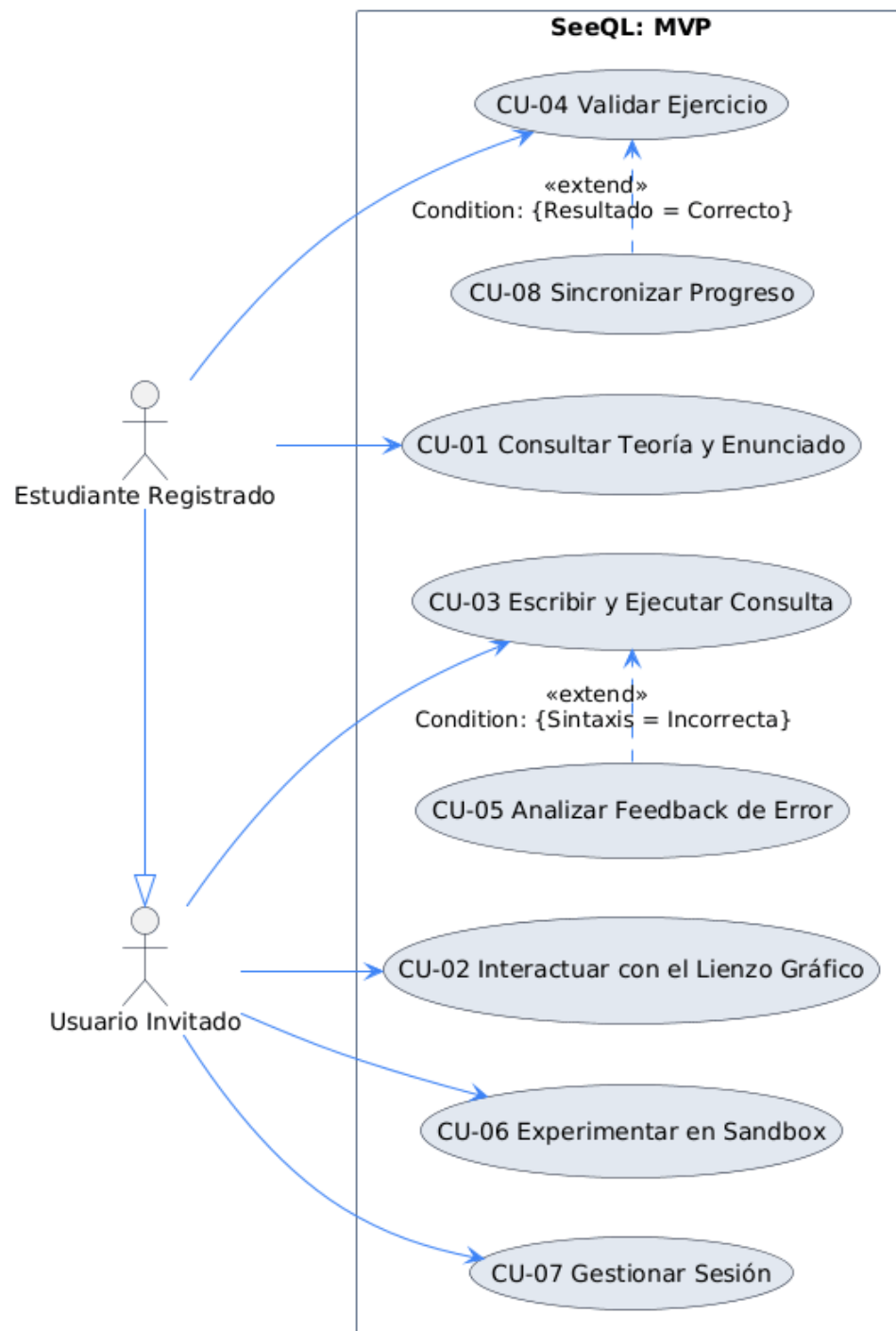


Figura 3.1: Diagrama de casos de uso de SeeQL.

CAPÍTULO 4

Diseño del sistema

En este capítulo se describe la solución técnica diseñada para satisfacer los requisitos analizados previamente. Se detalla la arquitectura de software, la selección del stack tecnológico, el modelado de datos y los principios de diseño de interfaz. Además, se profundiza en el diseño del itinerario pedagógico y el algoritmo de validación de consultas.

4.1 Arquitectura del sistema

4.1.1. Vista estructural: Arquitectura por capas

El sistema implementa una arquitectura de tres capas verticales fundamentada en un modelo de ejecución en el cliente (*client-side computing*). A diferencia de las arquitecturas web tradicionales, SeeQL descentraliza el procesamiento relacional delegando la totalidad de la carga computacional lógica a una capa de ejecución local integrada por el motor SQL.js compilado en WebAssembly. Esta estrategia optimiza los recursos de la infraestructura central y elimina por completo la latencia de red, debido a que el estudiante opera sobre una instancia efímera de la base de datos cargada directamente en la memoria RAM de su dispositivo.

Al trabajar en este entorno aislado, el alumno puede realizar operaciones DML y DDL sobre los datos con la certeza de que el estado original se restaurará íntegramente al reiniciar la vista, sin haber afectado nunca a la infraestructura del servidor.

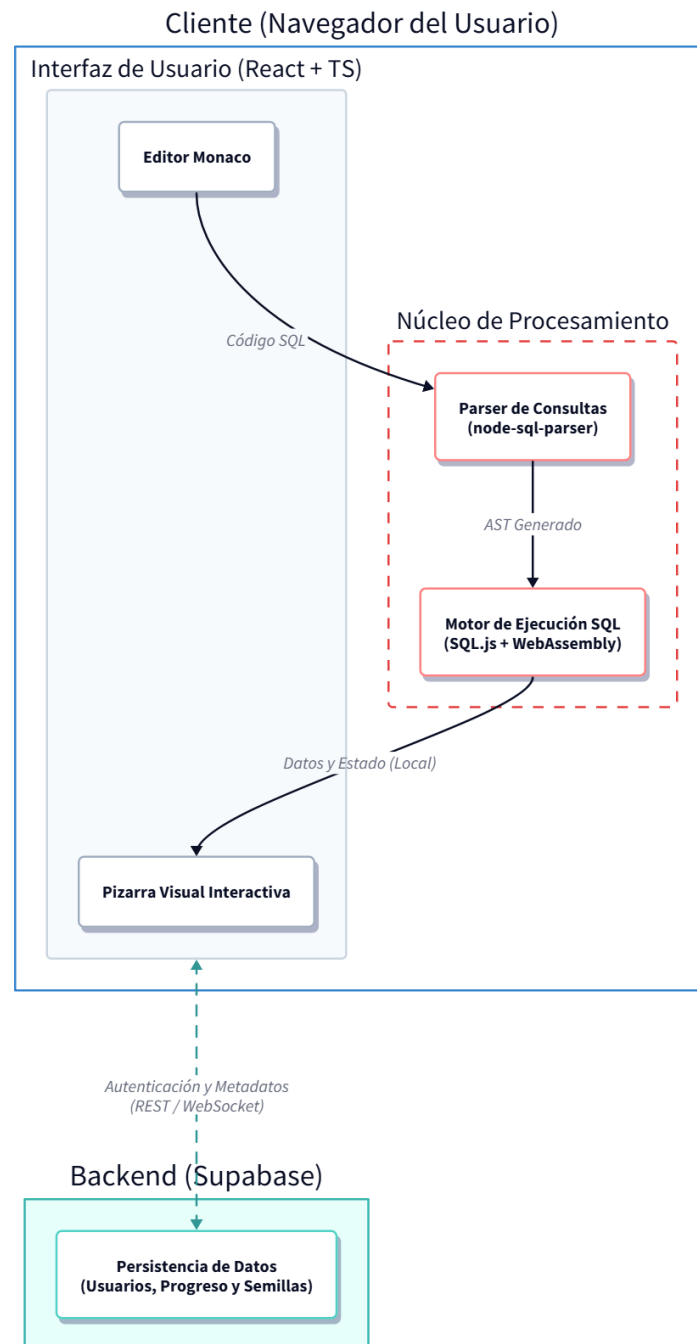


Figura 4.1: Arquitectura de SeeQL: Estructura de capas y modelo de ejecución en el cliente.

Este enfoque arquitectónico presenta tres ventajas críticas para el ámbito educativo:

- **Seguridad por aislamiento:** Al no existir un motor de ejecución en el servidor que reciba sentencias SQL dinámicas, se anula cualquier riesgo de inyección o denegación de servicio (DoS) sobre la infraestructura central.

- **Latencia cero:** El *feedback* pedagógico y visual es instantáneo, ya que no depende de la calidad de la conexión a internet del usuario para procesar las instrucciones.
- **Escalabilidad horizontal:** El coste de computación para el mantenimiento de la plataforma es prácticamente nulo, ya que el procesamiento recae en el hardware del propio cliente.

En este esquema, la **Capa de Persistencia Remota** se reserva exclusivamente para la autenticación y la sincronización asíncrona del progreso académico, actuando como un almacén de metadatos no ejecutables y desligándose por completo de la lógica relacional y del procesamiento de consultas.

4.1.2. Vista dinámica: Flujo de ejecución y validación temprana

Para comprender la interacción entre las capas durante el ciclo de vida de una consulta, el sistema prioriza la inmediatez y la pedagogía mediante una estrategia de validación en cascada. Como se detalla en el diagrama de secuencia (Figura 4.2), el flujo de datos es íntegramente local, orquestando la comunicación entre el editor, el generador AST y el motor WebAssembly.

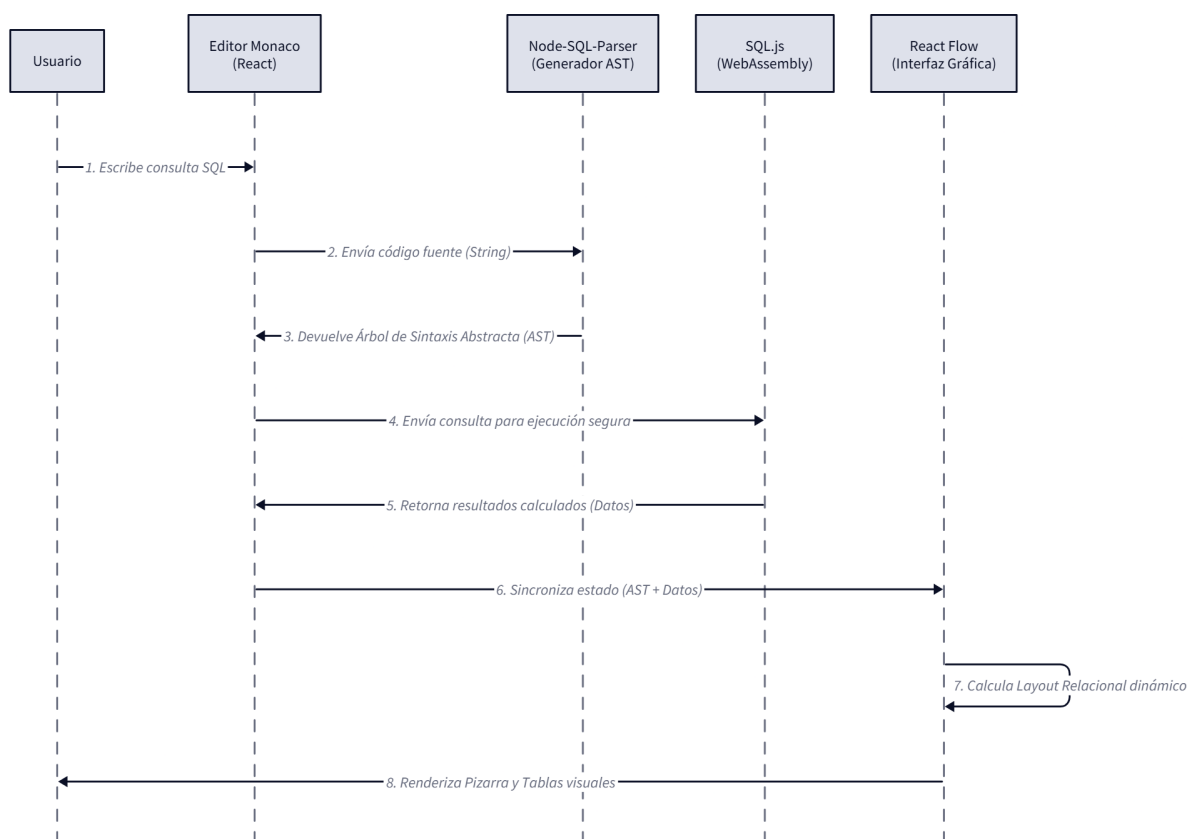


Figura 4.2: Diagrama de secuencia del flujo de procesamiento local de una consulta SQL.

Este diseño permite implementar una técnica de *fast-fail* adaptativa: el sistema analiza la intención de la consulta mediante análisis sintáctico antes de enviarla al motor local. Por ejemplo, si un estudiante introduce una sentencia destructiva dentro de una lección diseñada exclusivamente para evaluar la selección de datos (SELECT), el analizador intercepta la instrucción y aborta la operación de manera inmediata. Esto no solo ahorra ciclos

de cómputo en el dispositivo del alumno, sino que proporciona un *feedback* pedagógico inmediato, eliminando la frustración derivada de esperas innecesarias.

4.1.3. Arquitectura de seguridad por aislamiento

Dado que SeeQL permite escribir código SQL libremente, la seguridad no se fundamenta en el filtrado restrictivo en un servidor central, sino en el aislamiento físico del entorno de ejecución:

- **Modo Lecciones (validación pedagógica):** El sistema valida que la consulta se ajuste al objetivo del ejercicio. Dado que el catálogo actual se enfoca en la lectura (SELECT), si el alumno introduce sentencias DML o DDL, la ejecución se bloquea para evitar que altere accidentalmente los datos necesarios para resolver el reto. La arquitectura, no obstante, está preparada para habilitar estas operaciones en futuras lecciones de manipulación de datos.
- **Modo Sandbox (aislamiento total):** Al ejecutarse la base de datos íntegramente en la memoria volátil del navegador, el concepto tradicional de inyección SQL pierde su peligrosidad. El estudiante opera sobre una instancia efímera e independiente; cualquier operación de borrado o modificación solo afecta a su sesión local, y los datos originales se restauran simplemente refrescando la vista, sin haber comprometido nunca la infraestructura del sistema.

4.2 Stack tecnológico

Para garantizar un entorno de ejecución seguro y de baja latencia, se han seleccionado las siguientes tecnologías:

- **React y TypeScript:** Para la construcción de una interfaz robusta y con tipado estricto, facilitando el manejo de los estados complejos de las consultas SQL.
- **SQL.js (WebAssembly):** Componente crítico que permite instanciar una base de datos relacional completa en la memoria del navegador, eliminando la necesidad de un servidor de base de datos para la ejecución de las lecciones.
- **Supabase (BaaS):** Proporciona la infraestructura de persistencia mediante una base de datos PostgreSQL externa para almacenar el progreso académico y los perfiles de usuario.
- **Monaco Editor:** Integración del motor de edición de Visual Studio Code para ofrecer una experiencia de escritura de código profesional con resaltado de sintaxis nativo.

4.3 Modelado de datos

El diseño de los datos se divide en dos dominios claramente diferenciados para garantizar la integridad del sistema:

4.3.1. Esquema de la aplicación

Gestionado en el servicio de persistencia remota, almacena la información necesaria para la plataforma: perfiles de usuario, registro de progreso (niveles completados) y metadatos de las lecciones.

4.3.2. Esquemas educativos (semillas de datos)

Para garantizar un aprendizaje progresivo y guiado, se han diseñado tres bases de datos temáticas preconfiguradas que se inyectan dinámicamente en el motor en memoria SQL.js, dependiendo de la dificultad del ejercicio. La complejidad estructural de estos esquemas aumenta de forma escalonada. A continuación, se detallan y visualizan mediante el motor interactivo de SeeQL, mostrando sus atributos y restricciones de integridad estructural.

Nivel básico: Biblioteca

Este esquema constituye el punto de partida pedagógico de la plataforma. Se compone de un número reducido de entidades con relaciones directas y semánticamente muy intuitivas. Su diseño busca minimizar la carga cognitiva del estudiante en su primer contacto con el sistema, permitiéndole familiarizarse con la estructura de tablas y registros en un entorno controlado y sencillo antes de abordar modelos más densos.

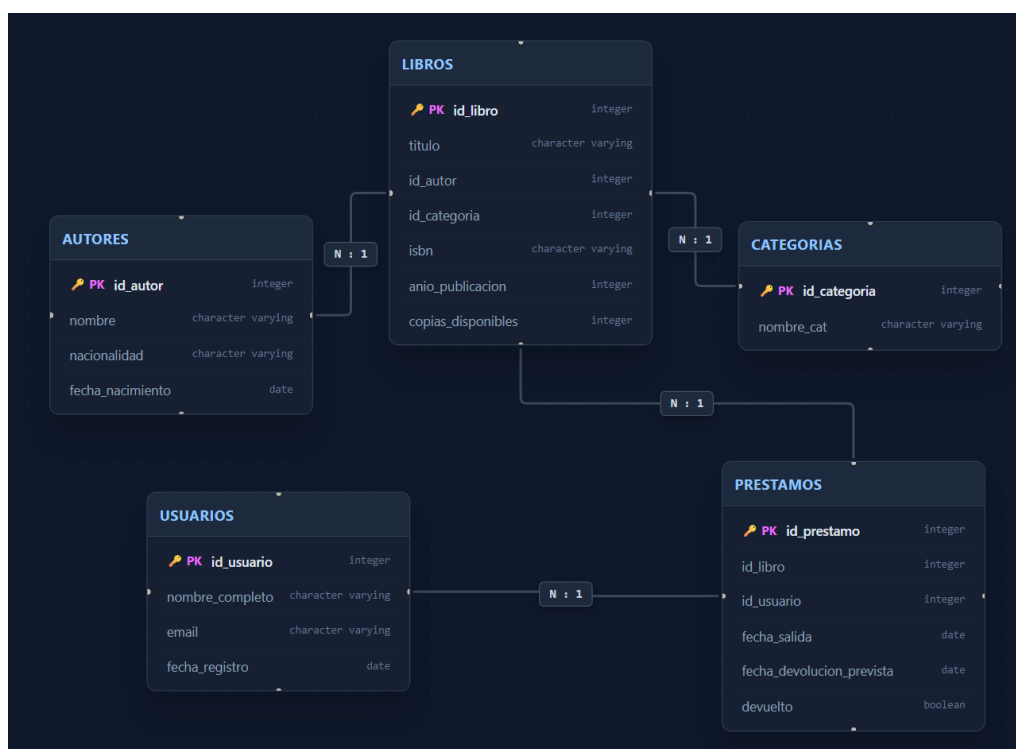


Figura 4.3: Modelo relacional - Esquema educativo: Biblioteca (SeeQL).

Nivel intermedio: Gimnasio

Basado en un modelo de gestión transaccional, este esquema incrementa la complejidad al introducir entidades con mayor volumen de atributos y relaciones de cardinalidad

múltiple. La estructura está diseñada para representar flujos de datos cercanos a la realidad empresarial, permitiendo al alumno observar cómo la información se distribuye y se relaciona en sistemas de gestión de servicios estándar.

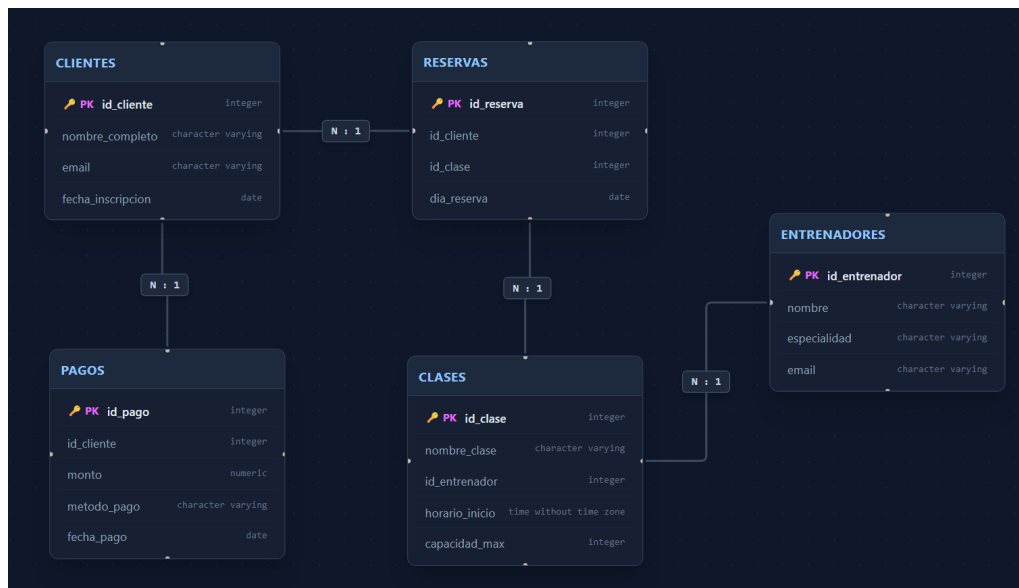


Figura 4.4: Modelo relacional - Esquema educativo: Gimnasio (SeeQL).

Nivel avanzado: Aeropuerto

Es el modelo de mayor densidad técnica y profundidad relacional de la plataforma. Presenta una red compleja de tablas interconectadas que incluyen múltiples claves foráneas y referencias cruzadas, como aeropuertos de origen y destino. Este esquema está concebido para desafiar la capacidad de abstracción del alumno, proporcionando un entorno rico para el análisis de datos multidimensionales y la resolución de problemas que requieren una visión global de la jerarquía de la base de datos.

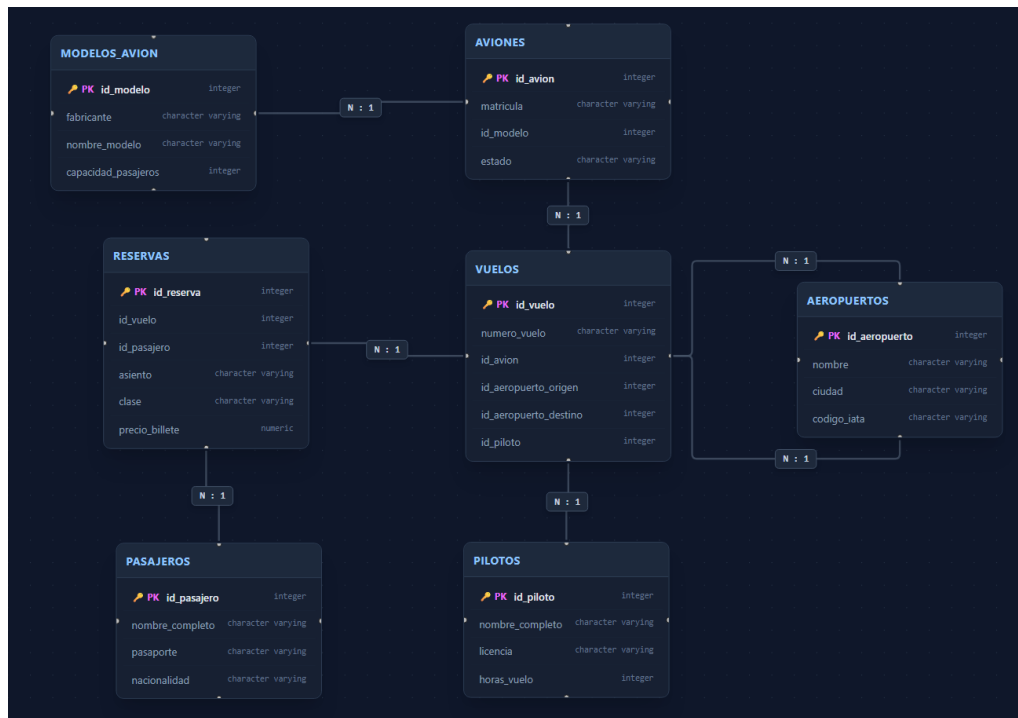


Figura 4.5: Modelo relacional - Esquema educativo: Aeropuerto (SeeQL).

4.4 Diseño de la interfaz de usuario

La interfaz de SeeQL se ha diseñado bajo un enfoque de «pizarra interactiva», priorizando la reducción de la carga cognitiva del estudiante. La disposición se organiza en torno a un **diseño trilateral** de paneles inamovibles que permite al usuario visualizar todo el flujo de trabajo sin necesidad de cambios de contexto. Este esquema se divide en tres áreas funcionales críticas:

1. **Panel de control y edición (izquierda):** Espacio dedicado a la interacción textual donde el usuario redacta las consultas SQL y, en el caso de las lecciones, consulta los objetivos pedagógicos.
2. **Pizarra de visualización (centro):** El núcleo visual de la aplicación, donde se representan de forma dinámica tanto el esquema relacional como los datos resultantes mediante nodos interactivos.
3. **Panel de feedback y resultados (inferior):** Consola técnica que devuelve el estado de la ejecución, mensajes de error depurados y métricas de éxito de la consulta.

Este diseño busca minimizar la denominada carga cognitiva. Al mantener el editor, la representación gráfica y los resultados en una misma vista, el estudiante puede observar el impacto inmediato de sus consultas sobre el modelo de datos. Siguiendo las directrices actuales sobre la optimización de interfaces visuales para reducir la fatiga cognitiva y mejorar la experiencia de usuario [25], la elección de una paleta de colores de alto contraste y tipografías monoespaciadas refuerza la similitud con un IDE profesional, facilitando la futura transición del alumno al ámbito laboral.

4.4.1. Componentes visuales: Estructura y datos

Una de las premisas fundamentales del diseño es la distinción visual entre la **capa estructural** y la **capa de datos**:

- **Nodos de Estructura:** Representan las entidades del esquema (tablas) y sus atributos. Con el objetivo de facilitar la comprensión de la jerarquía de los datos, cada nodo destaca visualmente la **clave primaria** mediante un indicador específico. Este marcador permite al usuario identificar de forma instantánea el identificador único de la entidad, facilitando la lógica necesaria para comprender la estructura fundamental del modelo relacional de un vistazo.
- **Nodos de Datos:** Esta es la vista más crítica del ecosistema de SeeQL, ya que materializa el contenido específico de las tablas y muestra cómo están pobladas en tiempo real. A diferencia de la vista estructural, que es estática, la vista de datos es un componente dinámico sobre el que se proyectan todas las modificaciones visuales derivadas de la interacción del usuario. Su importancia radica en que constituye el **mecanismo de validación empírica** del sistema: es el espacio donde el estudiante corrobora la exactitud de su lógica al observar cómo la sentencia SQL se materializa en una estructura de datos concreta. Al visualizar la transformación de las filas y columnas tras la ejecución, se facilita una comprensión profunda de las operaciones lógicas, convirtiendo una instrucción abstracta en un resultado visual inmediato y comprensible.

4.4.2. Especialización por modo de uso

Aunque la estructura base es común, la interfaz se adapta dinámicamente según el modo seleccionado para maximizar su eficacia pedagógica:

- **Interfaz del modo Lecciones:** Se añade un panel superior de narrativa que contiene el enunciado del reto. La consola de resultados se especializa en ofrecer tres niveles de *feedback*: el *output* de la consulta, la comparación con el resultado esperado y una pista dinámica en caso de error lógico.
- **Interfaz del modo Sandbox:** Elimina la narrativa para maximizar el área de la pizarra visual. En este modo, la consola de resultados se simplifica para actuar como una terminal estándar, permitiendo una experimentación libre y sin restricciones de guiado.

4.4.3. Prototipado de alta fidelidad

Antes de proceder a la implementación técnica, se desarrollaron prototipos de alta fidelidad que permitieron validar la disposición de los elementos, la jerarquía visual y la fluidez de la navegación. Este proceso de diseño permitió iterar sobre la experiencia de usuario antes de la escritura de código, asegurando una interfaz coherente en todos sus módulos.

A continuación, se describen los cinco hitos visuales que componen la arquitectura de SeeQL:

1. **Página de aterrizaje (*landing page*):** Punto de contacto inicial diseñado para establecer la propuesta de valor de la plataforma. Presenta una estética moderna y profesional que facilita el acceso al sistema mediante el flujo de autenticación (Figura 4.6).

2. **Panel de control (*dashboard*):** Actúa como el centro neurálgico del usuario tras el inicio de sesión. Permite la conmutación entre los dos modos principales de la plataforma, utilizando una iconografía semántica que refuerza la intención de cada área (Figura 4.7).
3. **Itinerario de aprendizaje:** Interfaz que materializa la progresión gamificada del curso. Muestra los diez niveles estructurados, permitiendo al estudiante visualizar su avance y los retos pendientes de forma intuitiva (Figura 4.8).
4. **Espacio de trabajo: Modo Lecciones:** Representa la implementación real del diseño trilateral. Integra el panel de narrativa pedagógica con el editor de código, la pizarra interactiva y la consola de resultados, permitiendo un flujo de trabajo sin saltos de contexto (Figura 4.9).
5. **Espacio de trabajo: Modo *Sandbox*:** Versión optimizada de la interfaz trilateral que prescinde de la narrativa para otorgar la máxima prioridad visual a la pizarra. Está diseñado para la experimentación libre con consultas complejas en el modelo relacional (Figura 4.10).

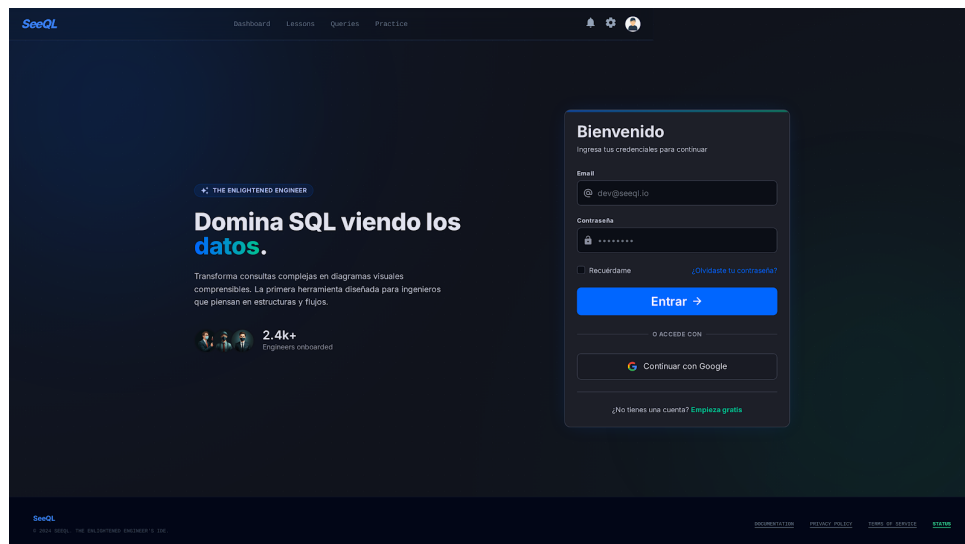


Figura 4.6: Prototipo de alta fidelidad: Página de aterrizaje y acceso.

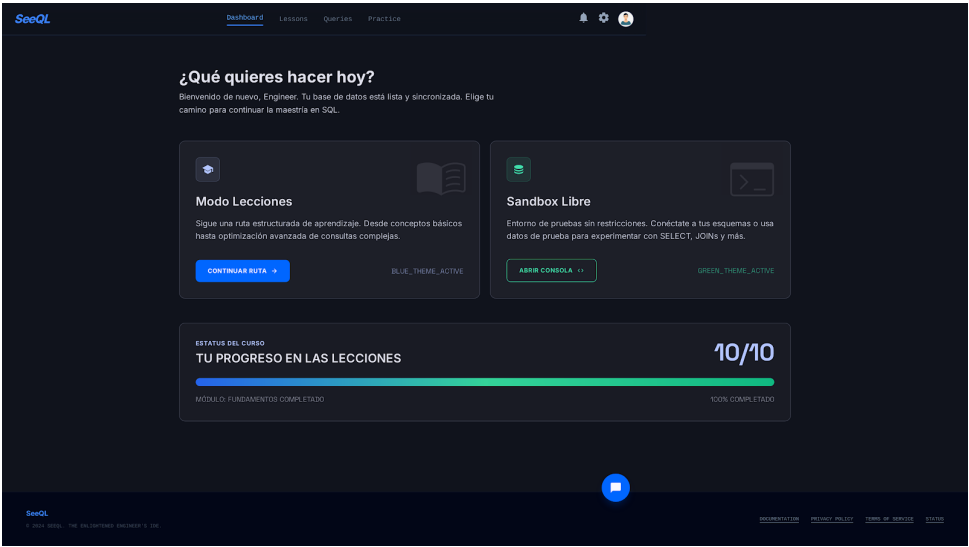


Figura 4.7: Prototipo de alta fidelidad: Dashboard de selección de modo.

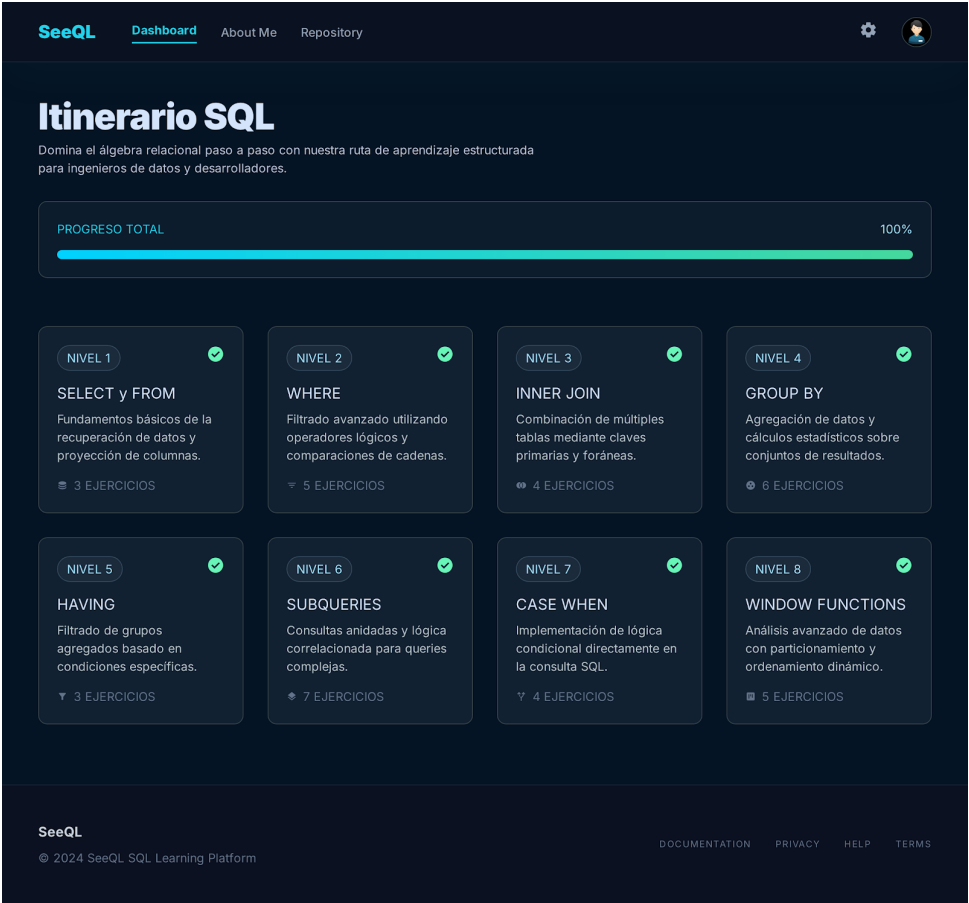


Figura 4.8: Prototipo de alta fidelidad: Menú de progresión de niveles.

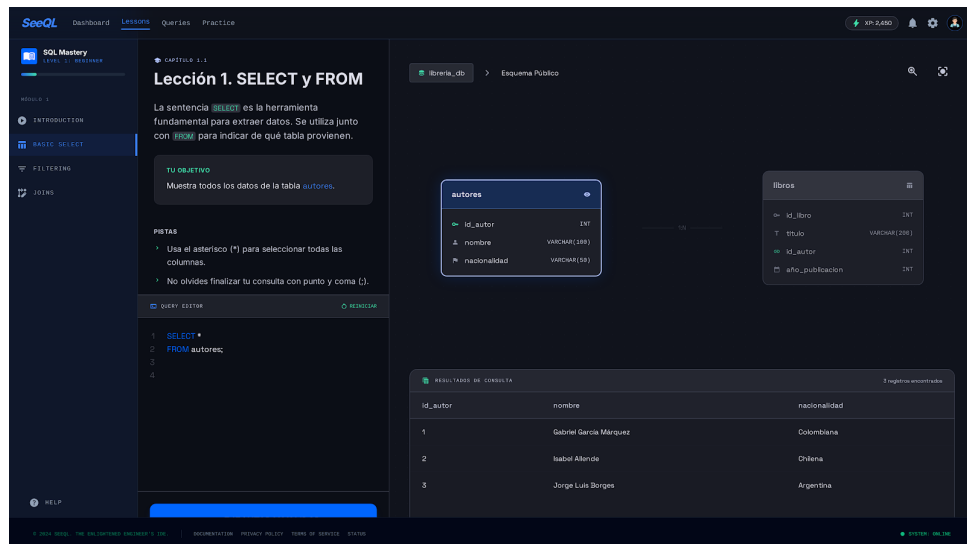


Figura 4.9: Prototipo de alta fidelidad: Interfaz trilateral en modo Lecciones.

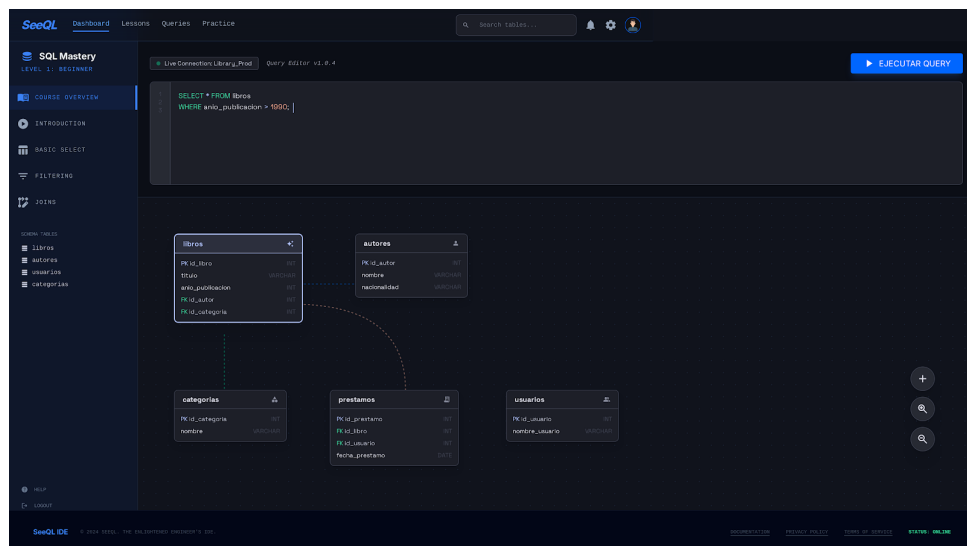


Figura 4.10: Prototipo de alta fidelidad: Interfaz optimizada en modo *Sandbox*.

4.5 Diseño del algoritmo de validación

Uno de los retos técnicos de SeeQL es validar si la consulta del alumno es correcta sin imponer una sintaxis rígida. Para ello, se ha diseñado un algoritmo basado en la **comparación del estado de salida** (*output matching*).

En lugar de comparar el texto de la consulta (lo cual penalizaría el uso de alias o variaciones en los *JOIN*), el sistema ejecuta en paralelo la consulta del alumno y la consulta solución almacenada. El *checker* valida que ambos conjuntos de resultados (filas y columnas) sean idénticos en contenido y estructura.

4.5.1. Estrategia de mitigación de falsos positivos

Este enfoque plantea el riesgo de *falsos positivos lógicos*. Por ejemplo, si un alumno escribe `precio > 10` cuando la respuesta correcta es `precio > 11`, el sistema podría darla por buena si no existen registros con el valor exacto de 11 en la base de datos.

Para resolver esto, el diseño de las **semillas de datos** no es aleatorio; se han aplicado técnicas de diseño de casos de prueba frontera. Cada base de datos educativa contiene registros con valores límite diseñados específicamente para romper la coincidencia de resultados si la lógica de la consulta del alumno no es exacta a la requerida. De este modo, se garantiza que solo la lógica correcta produzca el conjunto de datos esperado.

4.5.2. Sistema de *feedback* y orientación pedagógica

El algoritmo de validación va más allá de evaluar la corrección computacional de la consulta emitiendo un simple veredicto binario (acierto o fallo); actúa como un tutor virtual proporcionando *feedback* inmediato y didáctico. Dependiendo del resultado de la ejecución y la comparación, el sistema emite respuestas diferenciadas en la consola interactiva:

- **Errores de sintaxis:** Si el motor SQL.js falla al interpretar la instrucción, el error técnico subyacente es capturado y procesado por un módulo traductor. Este componente intercepta los mensajes de error nativos del motor (habitualmente en inglés y con una verbosidad técnica compleja) y los transforma en avisos pedagógicos amigables en castellano, guiando al alumno hacia la corrección de su sintaxis.
- **Errores lógicos:** Si la consulta se compila correctamente, pero el conjunto de datos devuelto no coincide con el esperado, el sistema informa al alumno mediante un aviso guiado: «*La consulta se ha ejecutado con éxito, pero los datos devueltos no son exactamente los solicitados*». De este modo, se le confirma que su sintaxis es correcta, pero su lógica de filtrado o proyección debe revisarse.
- **Validación exitosa:** Cuando el *checker* confirma la coincidencia exacta de los conjuntos de datos, la consola emite un refuerzo positivo remarcado visualmente en color verde y se invoca el flujo de gamificación para desbloquear el siguiente reto.

Como se describirá en el apartado de implementación de la interfaz (Sección 5.7), esta comunicación constante reduce la frustración del estudiante y convierte los errores en oportunidades guiadas de mejora.

4.6 Diseño del itinerario pedagógico

El itinerario se basa en la *transferencia gradual de responsabilidad*, dividiendo cada concepto en tres fases: ejemplo guiado (fácil), práctica de transferencia (medio) y práctica independiente (difícil).

Para mitigar la curva de aprendizaje y reducir la frustración del estudiante frente a errores comunes, el modelo de datos de cada ejercicio incorpora mecanismos de asistencia pasiva. Mientras que los niveles iniciales proporcionan código predefinido listo para su ejecución, los niveles de mayor complejidad ofrecen **pistas contextuales** orientadas a la lógica de la base de datos y **ayudas de sintaxis**. Estas últimas actúan como recordatorios técnicos preventivos, tales como el uso obligatorio de comillas simples al filtrar

cadena de texto en cláusulas WHERE o LIKE, guiando al alumno sin resolverle el problema directamente.

A continuación, se detallan los 10 niveles que componen el catálogo de SeeQL:

4.6.1. Lección 1: SELECT y FROM

Concepto a aprender: Proyección de atributos y selección de relaciones.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Muestra todos los datos de la tabla de autores.
- **Teoría:** La sentencia SELECT * se utiliza para pedir todas las columnas de una tabla. El FROM indica el nombre de la tabla que contiene los datos.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT *  
2 FROM autores
```

Solución esperada:

```
1 SELECT *  
2 FROM autores
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** Muestra el nombre completo y el email de todos los clientes.
- **Pista:** Sustituye el asterisco (*) por los nombres de las columnas separados por coma: nombre_completo, email.

Solución esperada:

```
1 SELECT nombre_completo, email  
2 FROM clientes
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** Lista el fabricante, el modelo y la capacidad de pasajeros de todos los modelos de aviones.
- **Pista:** Consulta las columnas fabricante, nombre_modelo y capacidad_pasajeros en la tabla modelos_avion.

Solución esperada:

```
1 SELECT fabricante, nombre_modelo, capacidad_pasajeros  
2 FROM modelos_avion
```

4.6.2. Lección 2: WHERE

Concepto a aprender: Restricción de tuplas mediante operadores de comparación.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Título de los libros cuyo año de publicación sea igual o posterior al año 2000.
- **Teoría:** La cláusula WHERE permite filtrar las filas. Solo aparecerán los registros que cumplan la condición.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT titulo
2 FROM libros
3 WHERE anio_publicacion >= 2000
```

Solución esperada:

```
1 SELECT titulo
2 FROM libros
3 WHERE anio_publicacion >= 2000
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** Obtén todos los datos de los pagos que superen los 50 euros.
- **Pista:** Recuerda que para seleccionar todos los campos se utiliza el asterisco (*).
- **Ayuda de sintaxis:** En SQL, los números se escriben tal cual, sin comillas.

Solución esperada:

```
1 SELECT *
2 FROM pagos
3 WHERE monto > 50
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** Busca el nombre completo de los pasajeros que tengan nacionalidad británica.
- **Pista:** Recuerda poner el texto entre comillas simples: nacionalidad = 'Británica'.
- **Ayuda de sintaxis:** En SQL, los textos se escriben entre comillas simples (ej: 'Española').

Solución esperada:

```
1 SELECT nombre_completo
2 FROM pasajeros
3 WHERE nacionalidad = 'Británica'
```

4.6.3. Lección 3: AND, OR, IN

Concepto a aprender: Combinación de múltiples condiciones lógicas de filtrado.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Títulos de libros publicados desde el año 1900 que tengan más de 1 copia disponible.
- **Teoría:** Podemos combinar filtros. AND exige que se cumplan ambas condiciones. OR exige que se cumpla al menos una. IN busca dentro de una lista de valores.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT titulo
2 FROM libros
3 WHERE anio_publicacion >= 1900
4 AND copias_disponibles > 1
```

Solución esperada:

```
1 SELECT titulo
2 FROM libros
3 WHERE anio_publicacion >= 1900
4 AND copias_disponibles > 1
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** Nombre de las clases que tengan capacidad de exactamente 20 personas o que sean impartidas por el entrenador con ID 1.
- **Pista:** Utiliza el operador lógico OR entre las dos condiciones.

Solución esperada:

```
1 SELECT nombre_clase
2 FROM clases
3 WHERE capacidad_max = 20
4 OR id_entrenador = 1
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** Número de los vuelos que salen desde los aeropuertos con ID 1 o 3.
- **Pista:** Utiliza el operador IN para buscar dentro de la lista de aeropuertos origen.

Solución esperada:

```
1 SELECT numero_vuelo
2 FROM vuelos
3 WHERE id_aeropuerto_origen IN (1, 3)
```

4.6.4. Lección 4: ORDER BY y LIMIT

Concepto a aprender: Ordenación del conjunto de resultados y restricción de salida.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Muestra el nombre del autor más antiguo.
- **Teoría:** ORDER BY ordena los resultados (ASC por defecto). LIMIT restringe el número de filas que se muestran al final.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT nombre
2 FROM autores
3 ORDER BY fecha_nacimiento ASC
4 LIMIT 1
```

Solución esperada:

```
1 SELECT nombre
2 FROM autores
3 ORDER BY fecha_nacimiento ASC
4 LIMIT 1
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** Nombre y fecha de inscripción de los 3 clientes que se han apuntado más recientemente.
- **Pista:** Ordena por fecha_inscripcion de forma descendente (DESC) para ver los últimos primero.

Solución esperada:

```
1 SELECT nombre_completo, fecha_inscripcion
2 FROM clientes
3 ORDER BY fecha_inscripcion DESC
4 LIMIT 3
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** ID de reserva y precio de las 2 reservas más caras.
- **Pista:** Usa DESC en el precio_billete para poner los más caros arriba.

Solución esperada:

```
1 SELECT id_reserva, precio_billete
2 FROM reservas
3 ORDER BY precio_billete DESC
4 LIMIT 2
```

4.6.5. Lección 5: LIKE

Concepto a aprender: Filtrado de cadenas de texto mediante patrones de búsqueda.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Títulos de los libros que empiecen por la palabra 'Harry'.
- **Teoría:** LIKE busca patrones. El símbolo % representa cualquier cantidad de caracteres. Por ejemplo, A% busca palabras que empiezan por A.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT titulo
2 FROM libros
3 WHERE titulo LIKE 'Harry%'
```

Solución esperada:

```
1 SELECT titulo
2 FROM libros
3 WHERE titulo LIKE 'Harry%'
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** Busca el nombre y el email de los clientes cuyo email termine exactamente en '@gmail.com'.
- **Pista:** Usa el porcentaje al principio del patrón: '%@gmail.com'.
- **Ayuda de sintaxis:** En SQL, los textos se escriben entre comillas simples.

Solución esperada:

```
1 SELECT nombre_completo , email
2 FROM clientes
3 WHERE email LIKE '%@gmail.com'
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** Busca el fabricante y el nombre de los modelos de aviones que contengan 'A3' en cualquier parte del nombre del modelo.
- **Pista:** Rodea la palabra con porcentajes: '%A3%'.
- **Ayuda de sintaxis:** En SQL, los textos se escriben entre comillas simples.

Solución esperada:

```
1 SELECT fabricante , nombre_modelo
2 FROM modelos_avion
3 WHERE nombre_modelo LIKE '%A3%'
```

4.6.6. Lección 6: Agregaciones (SUM, COUNT, AVG)

Concepto a aprender: Uso de funciones de agregación para cálculos métricos globales.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Cuenta cuántos libros hay registrados en total en la biblioteca.
- **Teoría:** Las funciones de agregación resumen datos. COUNT cuenta filas, SUM suma valores numéricos y AVG calcula la media.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT COUNT(*)  
2 FROM libros
```

Solución esperada:

```
1 SELECT COUNT(*)  
2 FROM libros
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** Calcula la suma total de dinero recaudado a través de todos los pagos realizados.
- **Pista:** Usa la función SUM sobre la columna monto.

Solución esperada:

```
1 SELECT SUM(monto)  
2 FROM pagos
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** Encuentra cuál es la capacidad media de pasajeros de los aviones del fabricante Airbus.
- **Pista:** Usa la función AVG sobre la columna capacidad_pasajeros.

Solución esperada:

```
1 SELECT AVG(capacidad_pasajeros)  
2 FROM modelos_avion  
3 WHERE fabricante = 'Airbus'
```

4.6.7. Lección 7: GROUP BY

Concepto a aprender: División de registros en subgrupos lógicos.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Muestra el ID de categoría y cuántos libros hay asociados a cada una.
- **Teoría:** GROUP BY agrupa filas que tienen los mismos valores. Se suele usar junto a funciones como COUNT o SUM para obtener totales por grupo.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT id_categoria , COUNT(*)  
2 FROM libros  
3 GROUP BY id_categoria
```

Solución esperada:

```
1 SELECT id_categoria , COUNT(*)  
2 FROM libros  
3 GROUP BY id_categoria
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** Obtén el ID de cada cliente y el dinero total que ha gastado cada uno de ellos.
- **Pista:** Suma el monto y agrupa los resultados por el id_cliente.

Solución esperada:

```
1 SELECT id_cliente , SUM(monto)  
2 FROM pagos  
3 GROUP BY id_cliente
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** ID del aeropuerto de origen y cantidad de vuelos que salen de cada uno de ellos.
- **Pista:** Agrupa por la columna id_aeropuerto_origen.

Solución esperada:

```
1 SELECT id_aeropuerto_origen , COUNT(*)  
2 FROM vuelos  
3 GROUP BY id_aeropuerto_origen
```

4.6.8. Lección 8: INNER JOIN

Concepto a aprender: Composición interna de tablas mediante claves foráneas.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Muestra el título de cada libro junto al nombre real de su autor.
- **Teoría:** JOIN une dos tablas. El INNER JOIN solo muestra los registros que tienen una coincidencia en ambas tablas mediante una clave común.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT libros.titulo , autores.nombre
2 FROM libros
3 INNER JOIN autores
4   ON libros.id_autor = autores.id_autor
```

Solución esperada:

```
1 SELECT libros.titulo , autores.nombre
2 FROM libros
3 INNER JOIN autores
4   ON libros.id_autor = autores.id_autor
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** Lista el nombre de cada clase junto con el nombre del entrenador que la imparte.
- **Pista:** Cruza la tabla clases con entrenadores usando la columna id_entrenador.

Solución esperada:

```
1 SELECT clases.nombre_clase , entrenadores.nombre
2 FROM clases
3 INNER JOIN entrenadores
4   ON clases.id_entrenador = entrenadores.id_entrenador
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** Identificador de la reserva y nombre completo de los pasajeros que tienen dicha reserva.
- **Pista:** Une las tablas reservas y pasajeros por el id_pasajero.

Solución esperada:

```
1 SELECT reservas.id_reserva , pasajeros.nombre_completo
2 FROM reservas
3 INNER JOIN pasajeros
4   ON reservas.id_pasajero = pasajeros.id_pasajero
```

4.6.9. Lección 9: HAVING

Concepto a aprender: Filtrado de conjuntos de resultados posteriores a una agrupación.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Muestra el ID de categoría y cantidad de libros, pero solo para aquellas que tengan más de 1 libro.
- **Teoría:** HAVING es el "WHERE" de los grupos. Se usa después de un GROUP BY para filtrar grupos que cumplen una condición estadística.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT id_categoria , COUNT(*)
2 FROM libros
3 GROUP BY id_categoria
4 HAVING COUNT(*) > 1
```

Solución esperada:

```
1 SELECT id_categoria , COUNT(*)
2 FROM libros
3 GROUP BY id_categoria
4 HAVING COUNT(*) > 1
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** ID del cliente y suma de sus pagos, solo para aquellos clientes que hayan gastado más de 100 euros en total.
- **Pista:** Filtra el grupo con HAVING SUM(monto) >100.

Solución esperada:

```
1 SELECT id_cliente , SUM(monto)
2 FROM pagos
3 GROUP BY id_cliente
4 HAVING SUM(monto) > 100
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** ID del aeropuerto y cantidad de vuelos, solo si operan más de 2 vuelos de salida desde allí.
- **Pista:** Usa HAVING sobre el COUNT(*) del grupo.

Solución esperada:

```
1 SELECT id_aeropuerto_origen , COUNT(*)
2 FROM vuelos
3 GROUP BY id_aeropuerto_origen
4 HAVING COUNT(*) > 2
```

4.6.10. Lección 10: LEFT JOIN

Concepto a aprender: Composición externa conservando los registros de la relación principal.

Ejemplo guiado (Nivel fácil - Biblioteca)

- **Enunciado:** Nombre de todos los usuarios y el ID de sus préstamos (deben aparecer incluso si nunca han pedido un libro).
- **Teoría:** El `LEFT JOIN` devuelve todas las filas de la tabla de la izquierda, y las filas coincidentes de la derecha. Si no hay coincidencia, devuelve `NULL`.
- **Pista:** ¡Simplemente ejecuta, ya está resuelto!

Código inicial proporcionado:

```
1 SELECT usuarios.nombre_completo, prestamos.id_prestamo
2 FROM usuarios
3 LEFT JOIN prestamos
4   ON usuarios.id_usuario = prestamos.id_usuario
```

Solución esperada:

```
1 SELECT usuarios.nombre_completo, prestamos.id_prestamo
2 FROM usuarios
3 LEFT JOIN prestamos
4   ON usuarios.id_usuario = prestamos.id_usuario
```

Ejercicio 1 (Nivel medio - Gimnasio)

- **Enunciado:** Lista todas las clases y el nombre de su entrenador (incluye las clases que no tienen ningún entrenador asignado).
- **Pista:** La tabla principal (izquierda) debe ser `clases`.

Solución esperada:

```
1 SELECT clases.nombre_clase, entrenadores.nombre
2 FROM clases
3 LEFT JOIN entrenadores
4   ON clases.id_entrenador = entrenadores.id_entrenador
```

Ejercicio 2 (Nivel difícil - Aeropuerto)

- **Enunciado:** Nombre del modelo de avión y número de vuelo (incluye los modelos que no tienen ningún vuelo programado actualmente).
- **Pista:** Une `modelos_avion` con `vuelos` usando `LEFT JOIN`.

Solución esperada:

```
1 SELECT nombre_modelo, numero_vuelo
2 FROM modelos_avion
3 LEFT JOIN vuelos
4   ON modelos_avion.id_modelo = vuelos.id_avion
```

CAPÍTULO 5

Implementación

El presente capítulo detalla el proceso de construcción y desarrollo de la plataforma SeeQL. Se exponen las decisiones técnicas adoptadas para materializar el diseño establecido en el capítulo anterior, abordando desde la configuración de la arquitectura base hasta la integración de los componentes clave: el motor de base de datos en el cliente, el analizador sintáctico de consultas y el sistema de renderizado visual.

5.1 Configuración del entorno y arquitectura *frontend*

El desarrollo de la aplicación se inicializó utilizando Vite, un empaquetador moderno de módulos web ultrarrápido que sustituye a herramientas tradicionales como Webpack, optimizando drásticamente los tiempos de compilación y el arranque del servidor local durante la fase de desarrollo. El núcleo de la interfaz se construyó con React en su versión 18 [26], utilizando TypeScript para dotar al proyecto de tipado estático estricto, mitigando errores en tiempo de ejecución al manipular estructuras de datos complejas.

Para la gestión del estado global de la aplicación, fundamental para coordinar la información entre el editor de código, el motor visual y las lecciones, se implementó una arquitectura basada en el *context API* de React, combinada con *hooks* personalizados. Esta separación de responsabilidades permite que los componentes visuales reaccionen instantáneamente a los cambios en el árbol de sintaxis abstracta de las consultas SQL sin forzar renderizados innecesarios en la totalidad de la interfaz.

5.2 Integración del motor SQL en el cliente

Uno de los pilares tecnológicos de SeeQL es la capacidad de ejecutar consultas SQL sin depender de llamadas constantes a un servidor *backend*, reduciendo la latencia a cero y garantizando un entorno de pruebas seguro.

Para lograr esto, se integró la librería SQL.js, una adaptación de SQLite compilada a WebAssembly [27]. Durante la carga de la aplicación o al iniciar un nuevo nivel, el sistema instancia una base de datos relacional directamente en la memoria del navegador. Los datos iniciales se inyectan en esta instancia virtual; al acceder al modo *Sandbox* se carga por defecto el esquema básico de la biblioteca, mientras que en el modo Lecciones la semilla de datos se inyecta dinámicamente en función del ejercicio activo. Cuando el estudiante ejecuta una instrucción desde el editor de código, la orden se procesa localmente en el binario WebAssembly, logrando un rendimiento casi a la par que la ejecución

nativa [28] y devolviendo un conjunto de resultados estructurado que la interfaz procesa e imprime en la consola inmediatamente.

5.3 Persistencia y modelado de datos

Aunque gran parte de la ejecución de sentencias ocurre en el cliente, la plataforma requiere persistencia para guardar el progreso académico. Para ello, se integró Supabase, respaldado por la robustez de PostgreSQL [29], como plataforma de BaaS.

Se diseñó una estructura de esquemas segregados para mantener la integridad de la aplicación:

- **Esquema *public*:** Contiene las tablas de sistema de SeeQL, incluyendo la gestión de perfiles y el registro de *progreso_usuarios*, donde se almacenan las lecciones completadas.
- **Esquemas educativos:** Se definieron esquemas aislados (*facil_biblioteca*, *medio_gym*, *dificil_aeropuerto*) para almacenar la estructura y los datos semilla de los ejercicios. Esta separación evita colisiones y permite escalabilidad horizontal.

Complementariamente, para la inicialización de los entornos, tanto en el modo Lecciones como en el modo *Sandbox*, se han delegado responsabilidades de lectura a funciones *remote procedure call* (RPC) estrictamente limitadas a metadatos:

- `obtener_estructura_esquema`: Función que consulta el catálogo del sistema para devolver dinámicamente al *frontend* la estructura de las tablas, permitiendo inyectar estas semillas de datos en la instancia local de SQL.js al inicio de la sesión.

De este modo, se elimina por completo la necesidad de funciones RPC para la ejecución de código SQL, garantizando que el procesamiento posterior sea 100 % *client-side*.

5.4 Procesamiento y análisis de consultas (SQL parser)

Para que la pizarra interactiva reaccione en tiempo real, el sistema debe interpretar la semántica de la consulta. Se ha integrado la librería *node-sql-parser* para generar un AST [30] que descompone la sentencia en sus componentes fundamentales: *type*, *columns*, *from*, *where*, *groupby* y *having*.

Mediante un *hook* personalizado (`useSqlVisualizer`), el sistema extrae estos nodos y los transforma en un estado reactivo. Este proceso permite ignorar variaciones sintácticas menores, como el uso de mayúsculas o espacios, y centrarse en la intención lógica del estudiante.

Para ilustrar este proceso, si el usuario introduce la consulta `SELECT nombre FROM clientes WHERE id = 1`, la librería genera automáticamente el siguiente objeto JSON:

```

1 {
2   "type": "select",
3   "columns": [{ "expr": { "type": "column_ref", "column": "nombre" } }],
4   "from": [{ "table": "clientes" }],
5   "where": {
6     "type": "binary_expr",
7     "operator": "=",
8     "left": { "type": "column_ref", "column": "id" },
9     "right": { "type": "number", "value": 1 }
10  }
11 }

```

Listing 5.1: Ejemplo simplificado del AST generado a partir de una instrucción SELECT.

5.4.1. Módulo de intercepción y traducción pedagógica de errores

Como se justificó en los objetivos pedagógicos del sistema, la sustitución de los mensajes de error complejos de los SGBD tradicionales por trazas formativas en el idioma nativo del alumno es una pieza clave para mitigar la frustración inicial [7]. Para materializar este comportamiento, se ha implementado el módulo de utilidad `traductorSQL.ts`, cuyo núcleo operativo reside en la función `traducirErrorSQL`.

La arquitectura de este submódulo no se limita a un mero mapeo estático de cadenas de texto, sino que aplica una estrategia combinada de análisis de patrones por subcadenas y extracción dinámica mediante expresiones regulares (*regex*). El flujo de procesamiento técnico se detalla a continuación:

- **Normalización de cadenas:** En primera instancia, el mensaje original devuelto por el hilo de ejecución local de `SQL.js` o los servicios de metadatos se convierte íntegramente a minúsculas. Esto blindo al sistema frente a las diferencias entre mayúsculas y minúsculas (*case sensitivity*) aplicadas por los diferentes compiladores o motores de base de datos.
- **Extracción dinámica de contexto:** Para errores de carácter estructural, como columnas ambiguas, tablas no encontradas o atributos inexistentes, la función aplica grupos de captura sobre expresiones regulares (`mensajeOriginal.match(/.../i)`). Esto permite aislar el nombre exacto del identificador conflictivo, como por ejemplo, el nombre de la columna duplicada o la tabla mal escrita, e inyectarlo dinámicamente en una plantilla de lenguaje natural en español.
- **Prevención de errores sintácticos comunes:** El analizador intercepta de forma específica las desviaciones típicas derivadas de la transición desde entornos de consola profesionales, como la inserción del carácter de punto y coma al final de la instrucción, recordando proactivamente al usuario las restricciones operativas de la interfaz trilateral de SeeQL.

El código fuente íntegro que da soporte a esta lógica de negocio evaluativa se detalla en el Listado 5.2.

```

1 export const traducirErrorSQL = (mensajeOriginal: string): string => {
2   if (!mensajeOriginal) return 'Error desconocido en la consulta.';
3
4   const msg = mensajeOriginal.toLowerCase();
5
6   if (msg.includes('empty query') || msg.includes('syntax error at end of input
    ')) {

```

```

7   return 'La consulta está vacía. Escribe algo de código SQL para poder
    ejecutarlo.';
8   }
9
10  console.log("Supabase error original:", mensajeOriginal);
11
12  if (msg.includes('ambiguous')) {
13    const match = mensajeOriginal.match(/column(?: reference)? ["']?([^"'\s]+)
    ["']?/i);
14    const columna = match ? match[1] : 'ambigua';
15    return 'Columna ambigua: Hay más de una tabla que tiene la columna "${
    columna}". Tienes que ser más específico escribiendo "nombre_tabla.${
    columna}".';
16  }
17
18  if (msg.includes('relation') && msg.includes('does not exist')) {
19    const match = mensajeOriginal.match(/relation ["']?([^"'\s]+) ["']?/i);
20    const tabla = match ? match[1] : 'que has escrito';
21    return 'Tabla no encontrada: Estás intentando usar la tabla "${tabla}",
    pero no existe en este esquema. ¡Revisa si hay algún error tipográfico!
    ';
22  }
23
24  if (msg.includes('column') && (msg.includes('does not exist') || msg.includes
    ('could not find')))) {
25    const match = mensajeOriginal.match(/column ["']?([^"'\s]+) ["']?/i) ||
    mensajeOriginal.match(/find the ["']?([^"'\s]+) ["']? column/i);
26    const columna = match ? match[1] : 'que has escrito';
27    return 'Columna inexistente: No encuentro la columna "${columna}". Revisa
    en la pizarra de estructura que esté bien escrita y que pertenezca a la
    tabla correcta.';
28  }
29
30  if (msg.includes('syntax error')) {
31    const match = mensajeOriginal.match(/syntax error at or near ["']?([^"']+)
    ["']?/i);
32    const cercaDe = match ? match[1] : '';
33    return cercaDe
34      ? 'Error de sintaxis: Hay algo raro cerca de "${cercaDe}". Revisa si te
    falta alguna coma o si has escrito mal una palabra reservada.
    Recuerda que en SeeQL no utilizamos punto y coma al final.'
35      : 'Error de sintaxis: Revisa la estructura de tu consulta, hay algo que
    no encaja.';
36  }
37
38  if (msg.includes('operator does not exist') || msg.includes('invalid input
    syntax')) {
39    return 'Error de tipos: Estás intentando operar o comparar cosas
    incompatibles (por ejemplo, texto con números). Revisa los tipos de
    datos en la estructura.';
40  }
41
42  return 'Ups, error del motor SQL: ${mensajeOriginal}';
43 };

```

Listing 5.2: Módulo de intercepción y formateo pedagógico de excepciones de la base de datos.

5.4.2. Orquestación del estado gráfico: El *hook* useSqlVisualizer

El verdadero núcleo de sincronización reactiva de la interfaz trilateral de SeeQL reside en el hook personalizado `useSqlVisualizer.ts`. Este componente actúa como un mediador de estado (*state mediator pattern*), centralizando la comunicación entre las respuestas


```
35 };
```

Listing 5.3: Hook personalizado para la sincronización reactiva de los estados de iluminación del lienzo en React Flow.

5.5 Motor de visualización gráfica

El núcleo de la interactividad reside en el mapeo del AST sobre los datos cargados en memoria. Para ello, se ha desarrollado un evaluador lógico recursivo en TypeScript denominado `evaluateWhere`.

5.5.1. Evaluación lógica y resaltado dinámico

La función `evaluateWhere` recorre el árbol de condiciones de forma descendente, resolviendo comparadores matemáticos (`=`, `>`, `<`), lógicos (`AND`, `OR`) y de patrones (`LIKE`). El algoritmo implementado en TypeScript traduce estas condiciones a operaciones de ejecución nativa en el entorno del navegador, determinando instantáneamente qué registros cumplen los criterios.

```
1 export const evaluateWhere = (row: any, ast: any): boolean => {
2   if (!ast) return true;
3
4   if (ast.type === 'column_ref') {
5     const val = resolveColumn(row, ast.column, ast.table);
6     return val !== undefined ? val : false;
7   }
8
9   if (ast.type === 'bool' || ast.type === 'boolean') {
10    return ast.value;
11  }
12
13  if (ast.type === 'binary_expr') {
14    const { left, right, operator } = ast;
15
16    if (operator === 'AND') {
17      return evaluateWhere(row, left) && evaluateWhere(row, right);
18    }
19    if (operator === 'OR') {
20      return evaluateWhere(row, left) || evaluateWhere(row, right);
21    }
22
23    const resolveValue = (node: any) => {
24      if (!node) return undefined;
25      if (node.type === 'column_ref') return resolveColumn(row, node.column,
26        node.table);
27      return node.value;
28    };
29
30    const leftVal = resolveValue(left);
31    const rightVal = resolveValue(right);
32
33    if (leftVal === undefined && left.type === 'column_ref') return false;
34
35    const leftNum = Number(leftVal);
36    const rightNum = Number(rightVal);
37    const esNumerico = !isNaN(leftNum) && !isNaN(rightNum);
38
39    switch (operator) {
40      case '=':
```

```

40     return String(leftVal).toLowerCase() === String(rightVal).toLowerCase()
41     ;
42     case '>':
43         return esNumerico ? leftNum > rightNum : String(leftVal) > String(
44             rightVal);
45     case '<':
46         return esNumerico ? leftNum < rightNum : String(leftVal) < String(
47             rightVal);
48     case '>=':
49         return esNumerico ? leftNum >= rightNum : String(leftVal) >= String(
50             rightVal);
51     case '<=':
52         return esNumerico ? leftNum <= rightNum : String(leftVal) <= String(
53             rightVal);
54     case '!=':
55     case '<>':
56         return String(leftVal).toLowerCase() !== String(rightVal).toLowerCase()
57         ;
58     case 'LIKE':
59         const escapadoRegex = String(rightVal).replace(/[\.*+?^${}()|[\]\\"/g, '\\$&');
60         const patronRegex = escapadoRegex.replace(/%/g, '%. ');
61         return new RegExp('^${patronRegex}$', 'i').test(String(leftVal));
62     default:
63         return true;
64 }
65 }
66 return true;
67 };

```

Listing 5.4: Algoritmo de evaluación recursiva del AST.

5.5.2. Representación de operaciones avanzadas

Más allá del filtrado básico, el motor visual implementa lógica para explicar conceptos complejos del álgebra relacional:

- **Agregación dinámica (GROUP BY):** El sistema detecta funciones de agregado (SUM, COUNT, AVG) y genera dinámicamente una *tabla virtual* de color distintivo naranja. Esta tabla agrupa los registros originales y calcula las métricas en tiempo real, permitiendo al alumno visualizar el proceso de *agregación* de datos.
- **Filtrado tras agrupación (HAVING):** A diferencia del WHERE, el motor aplica una segunda capa de evaluación sobre la tabla virtual generada, resaltando en verde los grupos que superan la condición de agregado.
- **Gestión de límites (LIMIT):** La interfaz procesa el nodo *limit* para mantener en su estado base, es decir, sin iluminación, los registros que quedan fuera del rango solicitado. Esto refuerza la comprensión del orden de ejecución de las cláusulas SQL, permitiendo visualizar el subconjunto final destacado frente a la totalidad del conjunto original.

5.5.3. Interactividad espacial y navegación en el lienzo

Notificaciones inteligentes de generación de tablas

Debido a la naturaleza de lienzo infinito proporcionada por la librería React Flow, existía el riesgo de que las nuevas tablas virtuales generadas por composiciones (JOIN) o agrupaciones (GROUP BY) se renderizaran fuera del campo visual actual del usuario. Para solucionarlo, se implementó un sistema de notificaciones flotantes interactivas que alertan de la creación de la nueva entidad relacional y ofrecen un botón de acción rápida «Ver resultado ahora». Al pulsarlo, el sistema ejecuta una animación de auto-centrado mediante la función `fitView`, aplicando una transición suave de 500 ms y un margen de seguridad (*padding*), dirigiendo la cámara exactamente a las coordenadas del nuevo nodo generado.

Tablas de ancho dinámico y ergonomía de navegación

Para evitar el problema común del doble *scroll* (un *scroll* interno en la tabla superpuesto a la navegación de la página), se eliminó la restricción de anchura máxima en los componentes de la pizarra. Al configurar los contenedores con la clase utilitaria `w-max` para que crezcan dinámicamente ajustándose a su contenido (`width: max-content`), se delega la navegación espacial íntegramente al lienzo infinito del usuario. Esto resulta en una interacción táctil y de ratón mucho más natural y ergonómica a la hora de explorar conjuntos de datos masivos.

5.5.4. Casos de uso visuales: Representación de operaciones complejas

Con el objetivo de ilustrar la capacidad didáctica del motor de visualización, a continuación se exponen tres casos prácticos que demuestran cómo el sistema traduce visualmente algunas de las operaciones más abstractas del álgebra relacional: el filtrado de registros, la composición entre tablas y la agregación de datos.

Filtrado de registros (WHERE)

El filtrado mediante la cláusula `WHERE` representa uno de los pilares de la interactividad en SeeQL. En lugar de ocultar la información que no cumple el criterio, el sistema implementa una técnica de *contraste de iluminación*: las filas descartadas permanecen en la pizarra en un estado visual base, mientras que los registros válidos se iluminan con bordes brillantes y tipografía resaltada. Esta decisión de diseño busca evitar el salto visual brusco y permitir al estudiante validar empíricamente por qué ciertos registros han sido seleccionados frente a otros.

ENTORNO DE DATOS

Biblioteca (fact)

ENTRADA SQL

```
1 SELECT *
2 FROM libros
3 WHERE anio_publicacion > 1990
```

libros

id_libro	titulo	id_autor	id_categoria	isbn	anio_publicacion	copias_disponibles
1	Clean Code	1	1	978-0132350884	2008	5
2	Harry Potter	2	2	978-8478884451	1997	3
3	Don Quijote	3	3	978-8424116347	1605	1
4	Harry Potter y el Cáliz	2	2	978-8478886456	2000	2
5	Juego de Tronos	4	2	978-8496208377	1996	0
6	Frankenstein	5	4	978-8420676063	1818	5
7	Danza de Dragones	4	2	978-8496208919	2011	2
8	Cantar de mio Cid	3	3	978-8420676000	1200	2

ejecutar query

Figura 5.1: Efecto de iluminación dinámica sobre los registros que cumplen la condición de filtro.

Composición relacional (INNER JOIN)

Cuando el alumno ejecuta un cruce entre dos tablas, el analizador AST identifica la condición de unión (cláusula ON). El motor gráfico instancia ambas tablas en la pizarra y, mediante una tabla virtual de color morado, visualiza el producto de la combinación. Para facilitar la comprensión, se traza una arista dinámica de color fucsia que conecta la clave primaria de la tabla origen con la clave foránea de la tabla destino. Como se observa en la Figura 5.2, esta codificación cromática atrae la atención del estudiante hacia el vínculo lógico exacto que permite la relación, materializando un concepto tradicionalmente opaco.

ENTORNO DE DATOS

Biblioteca (fact)

ENTRADA SQL

```
1 SELECT libros.titulo, autores.nombre
2 FROM libros
3 JOIN autores
4 ON libros.id_autor =
5 autores.id_autor
```

CONSULTA PROCESADA

Un resultado nuevo

libros

id_libro	titulo	id_autor	id_categoria	isbn	anio_publicacion	copias_disponibles
1	Clean Code	1	1	978-0132350884	2008	5
2	Harry Potter	2	2	978-8478884451	1997	3
3	Don Quijote	3	3	978-8424116347	1605	1
4	Harry Potter y el Cáliz	2	2	978-8478886456	2000	2
5	Juego de Tronos	4	2	978-8496208377	1996	0
6	Frankenstein	5	4	978-8420676063	1818	5
7	Danza de Dragones	4	2	978-8496208919	2011	2
8	Cantar de mio Cid	3	3	978-8420676000	1200	2

autores

id_autor	nombre	nacionalidad	fecha_nacimiento
1	Robert C. Martin	USA	1962-12-09
2	J.K. Rowling	UK	1965-07-31
3	Miguel de Cervantes	española	1547-09-29
4	George R.R. Martin	USA	1948-09-20
5	Mary Shelley	UK	1797-08-30

RESULTADO PÉDIDO INNER JOIN

id_libro	titulo	id_autor	id_categoria	isbn	anio_publicacion	copias_disponibles	id_autor	nombre	nacionalidad	fecha_nacimiento
1	Clean Code	1	1	978-0132350884	2008	5	1	Robert C. Martin	USA	1962-12-09
2	Harry Potter	2	2	978-8478884451	1997	3	2	J.K. Rowling	UK	1965-07-31
3	Don Quijote	3	3	978-8424116347	1605	1	3	Miguel de Cervantes	española	1547-09-29
4	Harry Potter y el Cáliz	2	2	978-8478886456	2000	2	2	J.K. Rowling	UK	1965-07-31
5	Juego de Tronos	4	2	978-8496208377	1996	0	4	George R.R. Martin	USA	1948-09-20
6	Frankenstein	5	4	978-8420676063	1818	5	5	Mary Shelley	UK	1797-08-30
7	Danza de Dragones	4	2	978-8496208919	2011	2	4	George R.R. Martin	USA	1948-09-20

ejecutar query

Figura 5.2: Representación visual de un INNER JOIN destacando la relación entre claves.

Agregación de datos (GROUP BY)

La cláusula GROUP BY supone un salto cognitivo importante, ya que transforma un conjunto de filas individuales en una entidad resumida. Al detectar esta instrucción, SeeQL no altera las tablas originales, sino que genera dinámicamente un nodo de tabla virtual de color naranja. En la Figura 5.3 se aprecia cómo esta nueva entidad se posiciona jerárquicamente en la parte inferior del flujo, mostrando el resultado de la función

de agregación y permitiendo al usuario comparar el dato transformado con los registros originales en tiempo real.

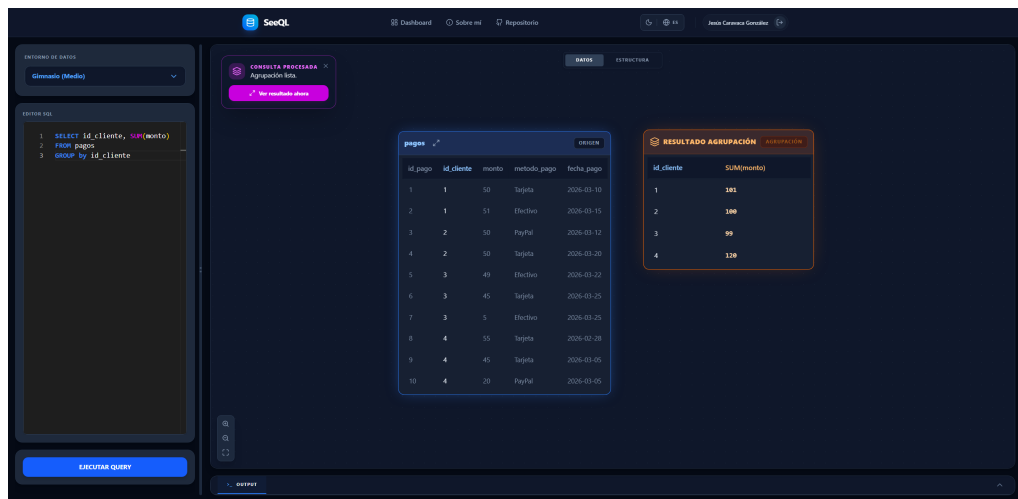


Figura 5.3: Generación dinámica de una tabla virtual tras aplicar una función de agregación.

5.6 Lógica de gamificación y motor de lecciones

El itinerario de aprendizaje se gestiona mediante un motor que orquesta la progresión del estudiante. Se ha definido la interfaz de TypeScript `Leccion`, la cual impone un contrato riguroso sobre los metadatos de cada ejercicio (dificultad, esquema asociado, teoría, `array` de pistas escalonadas y `querySolucion`).

Cuando un alumno ejecuta una consulta, el motor intercepta el conjunto de datos resultante y lo compara algorítmicamente con el resultado de la solución almacenada. Si la validación es exitosa, se actualiza el progreso en la interfaz, se desbloquea el siguiente reto y se realiza una llamada asíncrona a Supabase para persistir el avance.

Esta progresión lineal e interdependiente se materializa de forma directa en la interfaz gráfica del usuario, tal como se ilustra en la Figura 5.4. En dicho escenario, el sistema refleja el estado dinámico del itinerario tras la resolución del primer ejercicio: la primera unidad didáctica se muestra con el indicador visual de completado, lo que desencadena de manera reactiva la apertura del segundo reto, quedando disponible para su ejecución. De forma simultánea, los bloques del nivel comprendidos entre las lecciones 3 y 10 permanecen en estado de bloqueo pasivo con opacidad reducida, impidiendo el acceso desordenado y garantizando la asimilación gradual de las estructuras sintácticas del nivel.

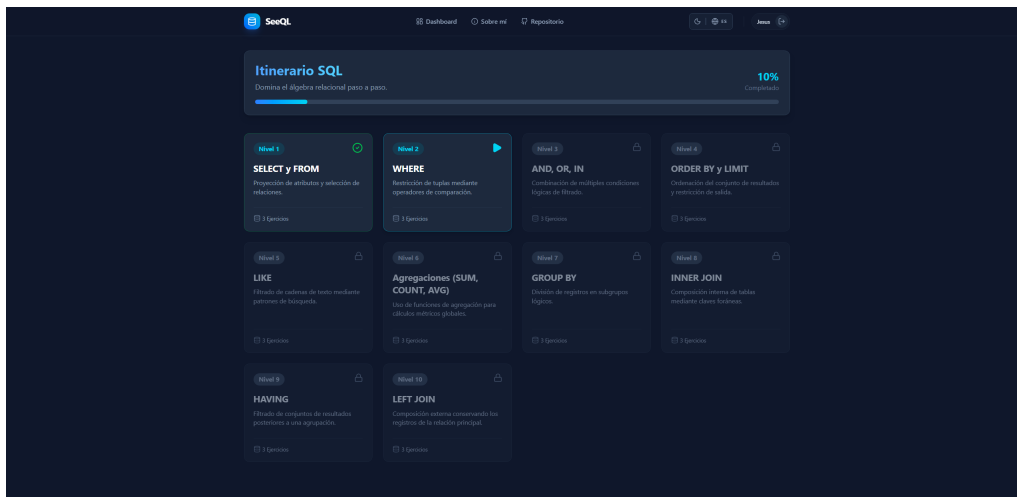


Figura 5.4: Estado del módulo de lecciones con progresión lineal y desbloqueo secuencial de retos.

5.7 Desarrollo de la interfaz de usuario

El diseño de la interfaz se implementó utilizando React y el *framework* de utilidades Tailwind CSS, garantizando una estética limpia y responsiva. El desarrollo de los componentes visuales se ha guiado por los principios de usabilidad y heurísticas de diseño de interfaces definidos por el Nielsen Norman Group [31].

Específicamente, se ha priorizado la visibilidad del estado del sistema, permitiendo que el usuario vea el resultado de su consulta de forma inmediata, y la consistencia estética, asegurando que los elementos interactivos sigan patrones predecibles. La puerta de entrada a esta zona de trabajo es el *dashboard* principal (Figura 5.5), el cual actúa como centro de mando para que el usuario navegue entre los diferentes módulos y esquemas de la aplicación, minimizando la carga de memoria del usuario mediante una navegación intuitiva y centralizada.



Figura 5.5: *Dashboard* principal de navegación y selección de esquemas de SeeQL.

5.7.1. El espacio de trabajo trilateral

Una vez dentro de un módulo, la pantalla principal se diseñó bajo un enfoque *desktop-first* adaptado a entornos de desarrollo. El espacio de trabajo se dividió estratégicamente en tres áreas inamovibles para evitar el salto de contenido y maximizar la concentración:

- **Panel lateral:** Alberga la instancia de *Monaco Editor*. En el modo aprendizaje, contiene además la información pedagógica del reto.
- **Panel visual (pizarra):** Área central interactiva dotada de un fondo de lienzo técnico que alterna entre el renderizado del esquema y la previsualización interactiva de los datos.
- **Consola de *output*:** Situada en la zona inferior, emula una terminal tradicional. Muestra mensajes de estado y renderiza los conjuntos de resultados.

Esta estructura base se adapta dinámicamente según el contexto de ejecución. En el **modo Lecciones** (Figura 5.6), el panel lateral integra el enunciado del reto. Por el contrario, en el modo *Sandbox* (Figura 5.7), al no requerir narrativa, el sistema oculta dicha sección para otorgar mayor verticalidad al editor de código y emular un entorno de experimentación libre.

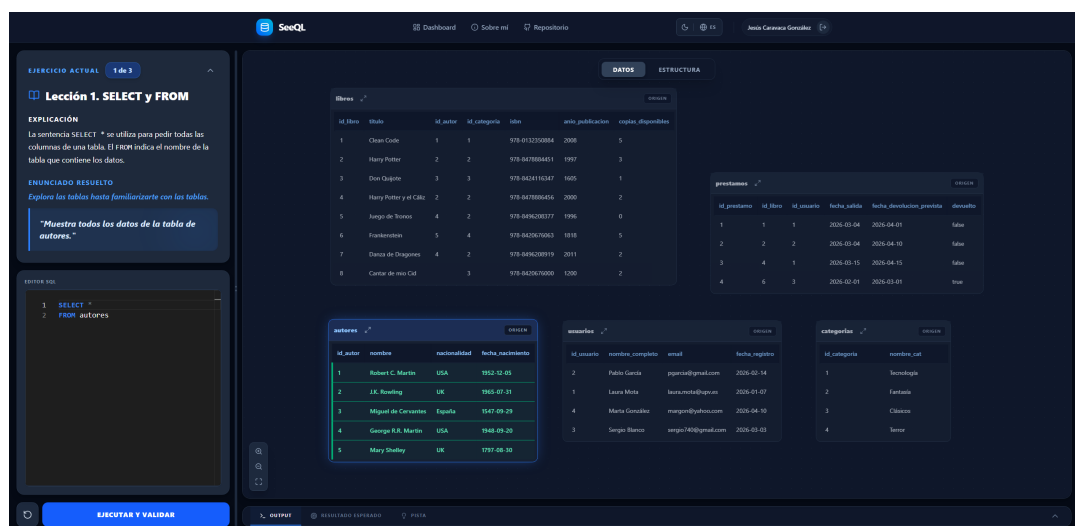


Figura 5.6: Interfaz del modo Lecciones con panel de narrativa integrado.

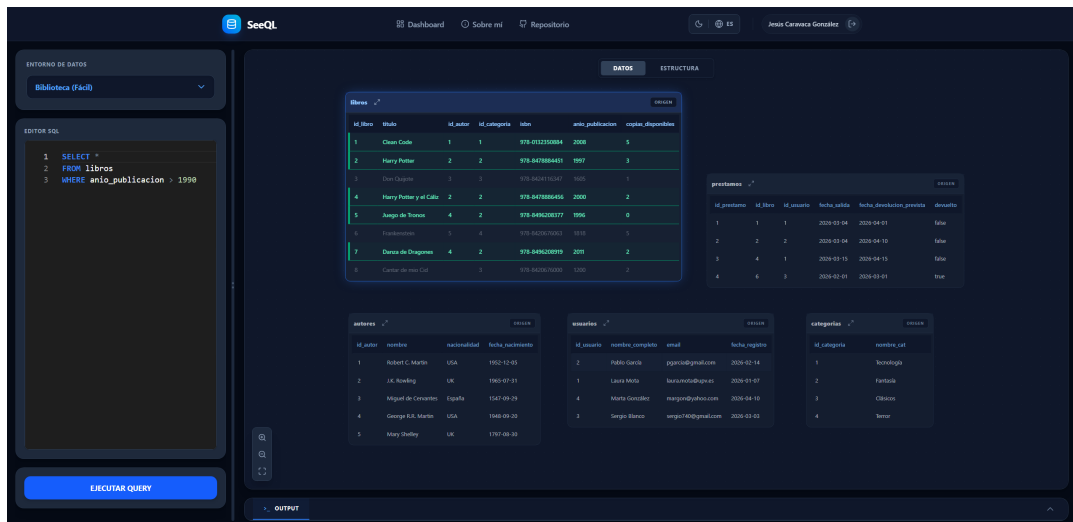


Figura 5.7: Interfaz del modo *Sandbox*, optimizada para experimentación.

5.7.2. Psicología del color y semántica visual

En el diseño de interfaces orientadas a la programación, la carga visual es un factor crítico. Para el entorno SeeQL, se adoptó una paleta de colores basada en principios de ergonomía cognitiva y psicología del color:

- **Modo oscuro nativo:** El lienzo principal y los editores operan sobre fondos oscuros (tonalidades *slate* #0f172a). Esta decisión no responde únicamente a una tendencia estética, sino a la necesidad de reducir la fatiga visual del alumno durante sesiones prolongadas de escritura de código.
- **Codificación semántica de datos:** Para facilitar la distinción de la procedencia de los datos en tiempo real, se implementó un sistema de colores estricto:
 - **Azul (blue-300):** Utilizado para tablas origen y datos inmutables. Transmite estabilidad y constituye el estado base de la información.
 - **Fucsia (fuchsia-400):** Reservado para las claves primarias y foráneas durante los cruces de tablas (*JOIN*). Este color vibrante atrae la atención del usuario hacia el *nexo relacional* que acopla ambas entidades.
 - **Naranja (orange-400):** Aplicado a las tablas virtuales de agrupación matemática (*GROUP BY*). El naranja sugiere transformación y procesamiento activo.
- **Representación del descarte mediante contraste de iluminación:** Para asimilar conceptualmente el truncamiento de resultados (*LIMIT*) o el filtrado de registros (*WHERE*), en lugar de hacer desaparecer inmediatamente los datos de la pizarra interactiva, el sistema mantiene las filas descartadas en su estado visual base sin iluminación especial. Esto permite al estudiante comparar directamente el subconjunto válido, resaltado con bordes y tipografía brillante, con el conjunto total de datos. La visualización estricta de las filas resultantes se reserva para la consola de *output* inferior, separando pedagógicamente la asimilación del proceso lógico de la validación del resultado técnico.

5.7.3. Layout relacional inteligente

Uno de los desafíos técnicos en la representación de esquemas dinámicos mediante la librería React Flow es evitar el solapamiento de las tablas cuando la base de datos tiene múltiples entidades o cuando se generan tablas virtuales en tiempo real.

Para solucionar este problema en bases de datos genéricas, el sistema implementa un algoritmo de *layout* relacional inteligente por defecto. Este motor calcula las coordenadas cartesianas (X, Y) de cada nodo basándose en su índice de aparición, distribuyendo las tablas origen en una cuadrícula fluida de dos columnas mediante operaciones modulares. Asimismo, cuando el alumno ejecuta una composición (JOIN) o una agrupación (GROUP BY), el sistema detecta la creación de la nueva entidad virtual y la posiciona jerárquicamente en la parte inferior, imitando el flujo natural de procesamiento de datos *top-down*.

Posicionamiento heurístico estático para esquemas educativos

Sin embargo, durante las fases de diseño de la interfaz se identificó que el enfoque matemático de cuadrícula podía generar cruces caóticos de aristas en esquemas con una alta densidad de claves foráneas.

Para solventarlo y ofrecer una experiencia pedagógica inmejorable, se desarrolló un sistema de **posicionamiento heurístico estático** aplicado específicamente a los tres esquemas educativos predefinidos de la plataforma (Biblioteca, Gimnasio y Aeropuerto). En lugar de delegar la distribución al algoritmo dinámico, se programaron coordenadas exactas y precalculadas para cada tabla de estos modelos.

Esta decisión de diseño manual cumple un objetivo didáctico fundamental: simular visualmente la distribución limpia, estructurada y jerárquica de un diagrama entidad-relación profesional. Al evitar por completo el solapamiento visual y los cruces innecesarios de relaciones, el estudiante puede comprender intuitivamente la arquitectura de la base de datos antes de comenzar a formular sus consultas.

5.7.4. Mejoras de experiencia de usuario e interacciones avanzadas

Con el objetivo de reducir la carga cognitiva y fomentar un aprendizaje profundo, se han integrado diversas funcionalidades orientadas a optimizar el flujo de trabajo del estudiante dentro de la plataforma.

Optimización del espacio de Trabajo: Panel de enunciado plegable

Dada la densidad de información requerida en ejercicios avanzados, la gestión del espacio vertical en el editor de código es crítica. Para ello, se ha implementado un panel lateral de enunciado colapsable mediante el estado reactivo `isEnunciadoOpen`.

Esta funcionalidad permite que el usuario, tras asimilar la teoría y los requisitos del reto, minimice el bloque informativo. Al hacerlo, el sistema redistribuye dinámicamente el *layout*, otorgando el 100 % de la altura disponible al `SqlEditor` y a la consola de resultados. Esta mejora técnica es fundamental para facilitar el análisis de consultas SQL extensas y la depuración de errores sin necesidad de realizar desplazamientos constantes en la interfaz.

Aprendizaje por descubrimiento

Las plataformas de *e-learning* tradicionales suelen forzar la progresión inmediata al siguiente nivel una vez detectada la respuesta correcta. En SeeQL, se ha introducido la opción de «Quedarme explorando» dentro del flujo de éxito del ejercicio.

Mediante el control del estado `isExploring`, el sistema permite que el alumno mantenga activa su solución válida en la pizarra interactiva. Esta decisión de diseño busca incentivar el aprendizaje autónomo, permitiendo que el estudiante modifique parámetros de su consulta ya validada para observar cómo reacciona el motor visual en tiempo real, sin la presión de avanzar prematuramente en el itinerario pedagógico.

Retroalimentación no intrusiva: Sistema de notificaciones en el navbar

Para informar sobre cambios en la configuración global, como la alternancia de temas visuales o la selección de idioma, se ha integrado un sistema de notificaciones contextuales (*toasts*) en la barra de navegación superior. A diferencia de las ventanas modales, estas notificaciones aparecen de forma efímera y no interrumpen el foco del estudiante en el editor de código. Esta técnica de micro-interacción garantiza que el usuario sea consciente del estado de la aplicación manteniendo una interfaz fluida y profesional.

Diseño reactivo de la consola de *feedback*

Como se detalló en el modelado conceptual del sistema (Sección 4.5.2), la consola inferior actúa como el principal canal de comunicación entre el motor de evaluación y el estudiante. Desde el punto de vista de la implementación de la interfaz, este componente, desarrollado como un contenedor dinámico en React, muta su estado visual y tipográfico basándose en el código de respuesta del validador:

- **Estado de error sintáctico:** Cuando el analizador AST o el motor SQL.js abortan la ejecución debido a una instrucción mal formada, la consola adopta tonalidades rojas de alerta crítica y renderiza el mensaje traducido al español por el módulo de interceptación, indicando al alumno el fallo estructural exacto.
- **Estado de error lógico:** Si la ejecución es sintácticamente válida pero los datos extraídos no coinciden con la solución esperada, la consola muta hacia una tonalidad naranja de advertencia. El sistema informa de la discrepancia cualitativa o cuantitativa, invitando al estudiante a auditar visualmente los registros proyectados en la pizarra interactiva para reformular su lógica.
- **Estado de éxito:** Al lograr la coincidencia exacta de los conjuntos de datos, el componente aplica una iluminación esmeralda de refuerzo positivo y despliega de forma dinámica el *call to action* pertinente para sincronizar de manera asíncrona el progreso en Supabase y avanzar al siguiente reto del itinerario.

A continuación, se ilustra la materialización visual de estos tres estados reactivos en la interfaz final de la plataforma: el error sintáctico (Figura 5.8), el error lógico o semántico (Figura 5.9) y, finalmente, la validación exitosa del ejercicio (Figura 5.10).

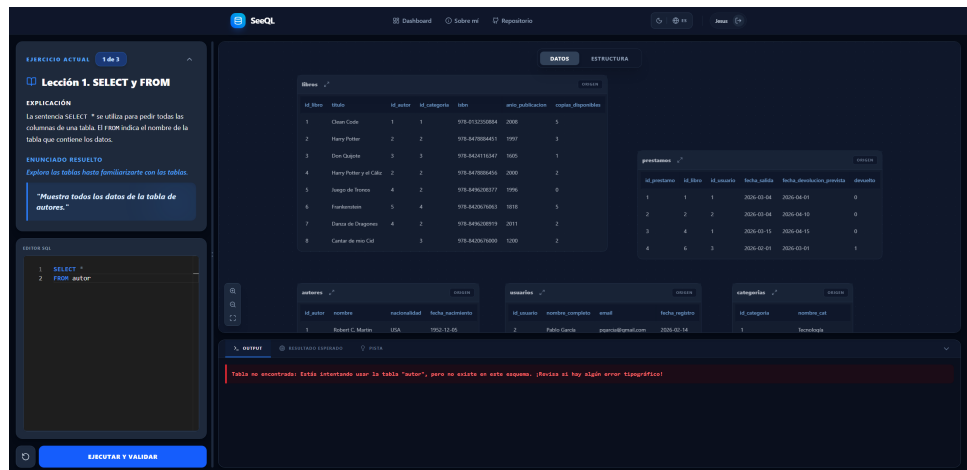


Figura 5.8: Interfaz de la consola de resultados ante un error de sintaxis.

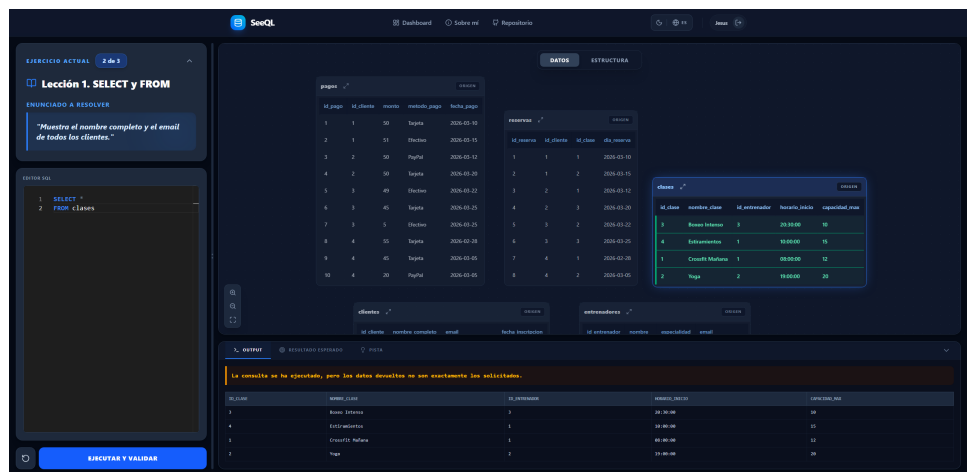


Figura 5.9: Feedback pedagógico ante un error lógico (datos no coincidentes).

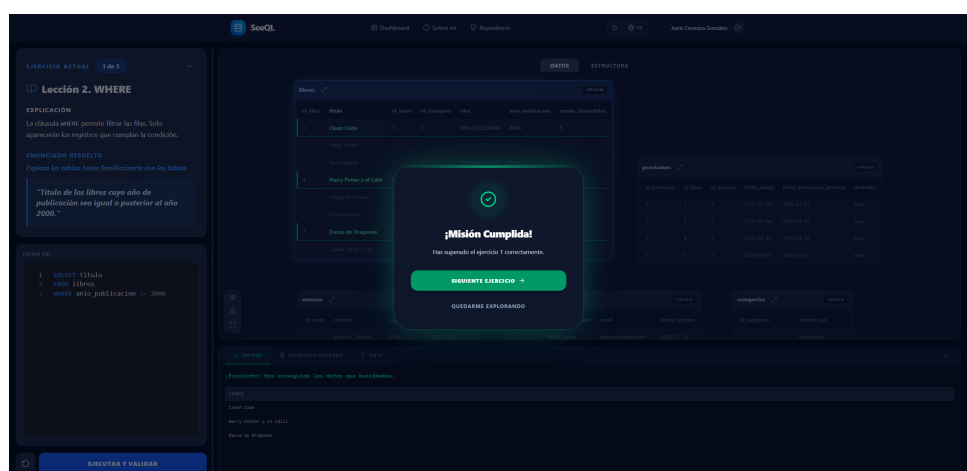


Figura 5.10: Representación visual de la consola tras una validación exitosa del reto.

5.8 Sistema de autenticación y gestión de sesión

Para garantizar un control de acceso seguro y permitir la persistencia del progreso académico, se ha desarrollado un sistema de autenticación completo operado a través de un contexto global de React (AuthContext). Este sistema va más allá de un registro básico y aborda desafíos reales de aplicaciones en producción.

5.8.1. Autenticación multicanal

La gestión de identidades se ha delegado en los servicios de autenticación de Supabase. El sistema soporta dos vías de acceso:

- **Autenticación tradicional:** Mediante correo electrónico y contraseña, implementando validación cruzada (confirmación de contraseña idéntica) en el cliente antes de emitir la petición de red, ahorrando así llamadas innecesarias al servidor.
- **Proveedores OAuth:** Integración nativa con Google Auth, reduciendo la fricción en el proceso de registro y delegando la seguridad criptográfica y gestión de contraseñas al proveedor de identidad externo.

5.8.2. Modo invitado

Uno de los retos técnicos en arquitecturas SPA es la persistencia automática de los *tokens* de sesión en el almacenamiento local del navegador.

Si un usuario finaliza su sesión cerrando abruptamente la pestaña, su *token* permanece activo. Si posteriormente otro usuario (o el mismo) intenta acceder pulsando en «Continuar como invitado» para usar la plataforma libremente sin necesidad de registro ni inicio de sesión, el sistema recuperaría automáticamente la sesión anterior, produciendo una filtración de contexto (*data leakage*). Para prevenir esto, se implementó un mecanismo de «Amnesia de sesión»: el botón de acceso como invitado ejecuta forzosamente una llamada asíncrona a `signOut()` antes de redirigir al *dashboard*, purgando cualquier credencial residual en el navegador y garantizando un estado verdaderamente anónimo.

5.8.3. Feedback en tiempo real y prevención de errores

A nivel de interfaz, el flujo de autenticación incorpora elementos de control en tiempo real para blindar la experiencia del usuario:

- **Modales de confirmación:** Tras un registro exitoso, la interfaz despliega un *popup* modal superpuesto que confirma la correcta creación de la cuenta y presenta un mensaje de bienvenida al usuario. Este elemento actúa como un paso de transición amigable antes de efectuar una redirección automática y controlada a la vista de *login*, asegurando que el flujo de autenticación concluya de manera limpia y predecible.
- **Bloqueo de estado:** Durante las transacciones de red (*login* o registro), los botones de acción mutan a un estado de carga (bloqueo interactivo `disabled` y animación *spinner*). Esta técnica previene el envío duplicado de formularios, un fallo crítico recurrente en aplicaciones web sin validación de estado.

5.9 Evolución arquitectónica y optimización del rendimiento

El desarrollo de SeeQL se ha regido por una metodología de carácter iterativo. Durante la primera fase de implementación, la arquitectura de datos se diseñó bajo un modelo tradicional cliente-servidor (*server-side computing*). En esta etapa, la extracción de esquemas y la validación de consultas dependían de llamadas RPC directas al *backend* (Supabase), empleando una estrategia de *eager loading* para garantizar la disponibilidad inmediata de toda la información en la interfaz.

Una vez estabilizada la lógica de negocio y el analizador AST, se procedió a la auditoría de métricas de rendimiento mediante Core Web Vitals. Los resultados iniciales revelaron un cuello de botella crítico en el tiempo de renderizado, con un *largest contentful paint* (LCP) de **3.65 segundos**. Este retraso evidenció que la dependencia de red penalizaba severamente la fluidez de una aplicación orientada a la interacción didáctica constante.

Para solventar este impedimento, se tomó la decisión técnica de refactorizar el núcleo del sistema hacia un paradigma de ejecución local en el cliente. Se migró la ejecución relacional a un motor WebAssembly instanciado localmente en la memoria del navegador, y se sustituyó la carga ansiosa por una estrategia de *lazy loading* gestionada mediante un servicio centralizado (DbManager).

Esta evolución arquitectónica permitió descargar el hilo principal de ejecución y eliminar la latencia de red durante el arranque del sistema. La reevaluación empírica tras la refactorización demostró un éxito rotundo: el LCP disminuyó a **0.21 segundos**, lo que representa una **mejora superior al 94 %**. Este resultado garantiza una percepción de carga instantánea y blindo el entorno de ejecución del alumno frente a fluctuaciones en la conectividad.

5.10 Despliegue y entorno de producción

Para garantizar la accesibilidad y disponibilidad de SeeQL a los usuarios finales, se definió una arquitectura de despliegue basada en servicios en la nube altamente escalables, adoptando prácticas de integración y despliegue continuo (CI/CD). El entorno de producción se encuentra físicamente desacoplado, siguiendo la arquitectura de componentes diseñada:

- **Capa de *frontend* y motor local (Vercel):** La aplicación SPA construida con React y Vite ha sido desplegada en **Vercel**, una plataforma de infraestructura en la nube optimizada para entornos *frontend*. Vercel se integró directamente con el repositorio de control de versiones (GitHub). Gracias a este flujo de CI/CD, cada nueva subida de código (*push*) a la rama principal desencadena automáticamente un proceso de compilación estática (`vite build`). Este proceso empaqueta el código fuente, optimiza los recursos estáticos y distribuye el binario WebAssembly de SQL.js a través de una red de entrega de contenidos global, garantizando tiempos de carga mínimos para los estudiantes.
- **Capa de *backend* y autenticación (Supabase):** La base de datos PostgreSQL, junto con los servicios de autenticación, operan en la infraestructura gestionada de Supabase. Al actuar puramente como un BaaS, Supabase se encarga automáticamente de la seguridad de los datos, las copias de seguridad periódicas y de la gestión de la concurrencia de sesiones.

Para garantizar la seguridad de la infraestructura y aislar los entornos de desarrollo y producción, las credenciales de conexión (`VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY`) no se incluyeron en el código fuente. En su lugar, se inyectaron de forma segura como variables de entorno directamente en el panel de administración de Vercel durante el proceso de despliegue.

Esta estrategia de implantación *serverless* da como resultado una plataforma robusta, que no requiere mantenimiento activo de servidores tradicionales, ofreciendo alta disponibilidad y una escalabilidad horizontal automática ante picos de demanda en entornos educativos.

CAPÍTULO 6

Evaluación y pruebas

En este capítulo se describe el proceso de validación de la plataforma SeeQL, abordando tanto la verificación técnica del software como la evaluación de su impacto pedagógico y la usabilidad con usuarios finales.

6.1 Validación técnica del software

Más allá de la validación con usuarios, la robustez de SeeQL se fundamenta en un conjunto de pruebas automatizadas y manuales que aseguran la integridad de la lógica de negocio, la comunicación entre módulos y el flujo pedagógico.

6.1.1. *Testing* unitario

Dado que el sistema delega la ejecución final en el motor SQL.js, el enfoque del *testing* unitario se ha centrado en los algoritmos críticos de la plataforma: el procesamiento del AST y la traducción de los errores al alumno.

Para la implementación de estas pruebas, se ha utilizado el entorno **Vitest** [32], elegido por su integración nativa con el ecosistema de Vite y su alta velocidad de ejecución. Tras diseñar una batería de 36 casos de prueba estructurados, el sistema ha alcanzado un **100 % de cobertura en líneas de ejecución** para los módulos evaluados, garantizando un comportamiento determinista. Se han cubierto tres áreas críticas:

1. **Traductor de errores pedagógicos** (`traductorSQL.ts`): Se ha verificado que el sistema es capaz de interceptar mensajes de error en bruto procedentes del motor de base de datos y transformarlos en retroalimentación formativa en español.
 - **Caso de prueba:** Detección de una columna ambigua (`column reference is ambiguous`) o tablas inexistentes (`relation does not exist`).
 - **Resultado esperado:** El traductor identifica el patrón mediante expresiones regulares, extrae el nombre del elemento conflictivo y devuelve un mensaje amigable indicando al alumno dónde está el fallo lógico.
2. **Motor de evaluación lógica** (`evaluateWhere`): El corazón visual de SeeQL depende de saber qué filas cumplen las condiciones escritas por el alumno. Se han programado *tests* que simulan filas de datos reales y las someten a nodos AST.
 - **Caso de prueba:** Evaluación de operadores matemáticos (`>`, `=`), operadores de comparación de patrones (`LIKE`) y lógicas combinadas (`AND`, `OR`, `IN`).

- **Resultado esperado:** La función pura devuelve estrictamente `true` o `false` evaluando correctamente la precedencia de los operadores lógicos sobre los datos simulados.
3. **Extractor de estructura SQL (useSqlVisualizer):** Se ha validado el *hook* de React encargado de desglosar las consultas. Utilizando herramientas de renderizado simulado (*testing library*), se comprueba que el estado interno se actualiza correctamente al inyectar código SQL.
- **Caso de prueba:** Envío de una consulta `SELECT` con especificación de múltiples columnas y alias.
 - **Resultado esperado:** El visualizador reinicia su estado previo y extrae un *array* exacto con las tablas activas y las columnas seleccionadas, datos indispensables para configurar los diagramas interactivos posteriores.

6.1.2. Pruebas de integración

Mientras que las pruebas unitarias aseguran el correcto funcionamiento de los algoritmos aislados, las pruebas de integración tienen como objetivo verificar la correcta comunicación y el flujo de datos entre los distintos módulos arquitectónicos del sistema. En SeeQL, la complejidad reside en la orquestación entre la interfaz de usuario (React), el AST y el motor de ejecución en memoria (SQL.js).

Para validar este ecosistema, se diseñaron casos de prueba de integración utilizando **React Testing Library** en combinación con Vitest. Esto permitió simular interacciones reales en el modelo de objetos del documento (DOM) sin necesidad de levantar un navegador completo.

- **Flujo de ejecución:** Se programó un *test* en el que se simula la escritura de una consulta `SELECT` con un `INNER JOIN` en el componente del editor.
- **Validación del flujo:** El *test* verifica de forma secuencial que (1) el componente editor envía correctamente la cadena de texto al *hook* encargado de su interpretación, (2) el motor SQL.js procesa la consulta devolviendo el conjunto de resultados esperado, y (3) el estado global de la aplicación se actualiza, forzando el renderizado de las aristas de conexión en la pizarra interactiva.
- **Resultado:** Se confirmó empíricamente que la comunicación entre el motor de renderizado de React y la instancia en memoria de WebAssembly se realiza de forma directa, síncrona y sin pérdida de contexto de datos, validando así la robustez del núcleo interactivo de la plataforma.

6.1.3. Pruebas de aceptación y *smoke testing*

Antes de someter la plataforma a la evaluación empírica con usuarios finales y docentes, se llevó a cabo una fase de pruebas de aceptación usuario (*user acceptance testing* o UAT) internas. El propósito principal de esta fase es establecer una trazabilidad directa para validar que el software construido cumple con los requisitos fundamentales especificados en la Sección 3.1, garantizando que no existen bloqueos críticos.

Se ejecutó una técnica de *smoke testing* simulando el «camino feliz» (*happy path*) de un estudiante recién registrado. Cada hito de esta prueba certifica la viabilidad de los requisitos de mayor prioridad:

1. **Autenticación y persistencia:** Registro de una cuenta de prueba mediante los servicios de Supabase. [*Valida RF-02 (Registro de usuarios) y RNF-04 (Persistencia de datos)*]
2. **Interacción de lección y feedback:** Acceso a la primera lección del Nivel Básico (esquema Biblioteca), introducción de una respuesta sintácticamente incorrecta para forzar la activación del traductor de errores sintácticos, posterior introducción de una consulta lógicamente incorrecta para evaluar la respuesta del sistema ante una devolución de datos errónea, y finalmente la inserción de la respuesta correcta. [*Valida RF-04 (Validación de consulta) y RF-05 (Traductor de errores)*]
3. **Progresión de estado y visualización:** Verificación de que el sistema valida el reto, dibuja las tablas resultantes en el lienzo, actualiza la base de datos remota con el progreso del alumno y desbloquea el acceso al siguiente nivel de dificultad. [*Valida RF-03 (Progreso lineal), RF-08 (Visualización gráfica) y RF-13 (Indicadores de progreso)*]

La superación de estos escenarios de aceptación, ejecutados dentro de los márgenes de tiempo de respuesta exigidos [*validando RNF-05 (Rendimiento)*], garantizó que la plataforma era lo suficientemente estable e íntegra como para ser distribuida a la muestra de usuarios reales sin riesgo de corromper la experiencia de evaluación.

6.2 Metodología de evaluación con usuarios

Para validar el grado de cumplimiento de los objetivos propuestos, se ha diseñado un plan de evaluación empírica basado en la recopilación de métricas cualitativas y cuantitativas. La recolección de datos se articula mediante cuestionarios estructurados adaptados a los distintos perfiles, inspirándose en modelos de evaluación estandarizados: la *system usability scale* (SUS) [33] y el *technology acceptance model* (TAM) [34] para el análisis exhaustivo del alumnado, y la métrica ágil *usability metric for user experience* (UMUX) [35] para la validación docente.

El instrumento evalúa tres dimensiones clave del sistema:

- **Efectividad:** La capacidad de la herramienta para cumplir con los requisitos docentes y facilitar la comprensión visual de SQL.
- **Satisfacción:** Medición de la fricción de uso y reducción de la frustración frente a interfaces tradicionales (evaluada mediante preguntas con escalas invertidas).
- **Eficiencia:** Percepción sobre la optimización del tiempo empleado en la corrección y diagnóstico de errores durante las sesiones.

Dado que SeeQL es una herramienta educativa, la validación se ha segmentado en dos perfiles de usuario diferenciados:

6.2.1. Diseño del instrumento: Perfil estudiante

El cuestionario dirigido a estudiantes mide la usabilidad del sistema, la reducción de la carga cognitiva y la efectividad del modelo visual trilateral. El instrumento se estructura en cinco dimensiones clave evaluadas mediante campos de texto abierto y escalas Likert de cinco puntos (donde 1 indica *totalmente en desacuerdo* y 5 *totalmente de acuerdo*):

1. **Dimensión 1: Demografía y conocimiento previo.** Contextualización técnica del usuario.
2. **Dimensión 2: Usabilidad y experiencia de usuario.** Evaluación de la curva de aprendizaje y la interacción física.
3. **Dimensión 3: Efectividad de la representación visual.** Validación del objetivo pedagógico de la distinción entre estructura y datos.
4. **Dimensión 4: Valor pedagógico y herramientas de apoyo.** Análisis del *feedback* y la progresión del modo Lecciones.
5. **Dimensión 5: Satisfacción global y fidelización.** Impresiones finales y oportunidades de mejora.

6.2.2. Diseño del instrumento: Perfil docente (validación experta)

Resulta imprescindible someter la plataforma a una evaluación por expertos para medir la viabilidad pedagógica y su capacidad de integración en un aula real. Dada la limitada disponibilidad de tiempo de este perfil, se optó por diseñar un instrumento híbrido y ágil. La sección cuantitativa se fundamenta en el estándar UMUX, complementado con preguntas de categorización y respuestas abiertas. El cuestionario se divide en tres bloques principales:

1. **Perfil y experiencia docente:** Nivel de *seniority* del evaluador medido en años impartiendo docencia en la asignatura de Bases de Datos.
2. **Evaluación de usabilidad y utilidad (UMUX):** Escala estandarizada de cuatro ítems evaluados mediante escala Likert del 1 al 5 (donde 1 indica *totalmente en desacuerdo* y 5 *totalmente de acuerdo*) que mide las siguientes dimensiones:
 - **Cumplimiento de requisitos:** Mide si las capacidades de SeeQL permiten a los alumnos poner en práctica los objetivos docentes.
 - **Experiencia fluida (escala invertida):** Evaluación de la ausencia de frustración durante el uso de la herramienta.
 - **Intuitividad:** Medición directa sobre la facilidad de uso y comprensión de la interfaz.
 - **Carga operativa (escala invertida):** Evaluación de la eficiencia del sistema, asegurando que el docente/alumno no deba invertir tiempo excesivo peleando con la interfaz en lugar de centrarse en el código SQL.
3. **Feedback cualitativo y oportunidades de integración:** Bloque de preguntas abiertas destinado a identificar los aspectos de mayor valor pedagógico y recopilar sugerencias de mejora técnica antes de su despliegue oficial con alumnos.

6.3 Análisis de resultados: Evaluación de estudiantes

Una vez concluida la fase de despliegue de SeeQL, se procedió a la recopilación y análisis de los datos obtenidos mediante cuestionarios de evaluación. La muestra final consta de 11 estudiantes que interactuaron con la herramienta bajo un protocolo de pruebas guiado, que consistió en la exploración libre del modo *Sandbox* y la resolución secuencial de tres lecciones pedagógicas. Tras finalizar estas tareas, los participantes evaluaron diferentes dimensiones críticas del sistema.

El análisis de estos resultados no solo permite validar las hipótesis de diseño e implementación planteadas en las fases iniciales de este proyecto, sino que también arroja conclusiones valiosas sobre el comportamiento de los usuarios frente a entornos gamificados de aprendizaje de bases de datos.

6.3.1. Perfil demográfico y conocimientos previos

Para garantizar que la evaluación de SeeQL fuera representativa, era fundamental contar con una muestra heterogénea en cuanto a experiencia previa con bases de datos y lenguaje SQL. Como se observa en la Figura 6.1, la distribución de los participantes abarca los tres perfiles clave de la aplicación.

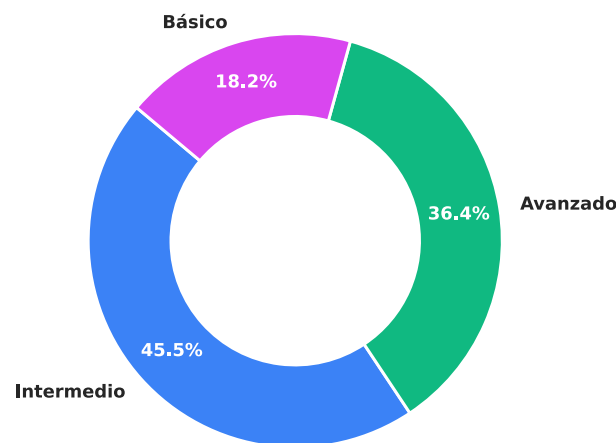


Figura 6.1: Distribución de los participantes según su nivel previo de conocimientos en SQL.

El bloque mayoritario (45.5 %) corresponde a usuarios de nivel intermedio, seguidos por un 36.4 % de perfiles avanzados y un 18.2 % de perfiles básicos. Esta diversidad es altamente positiva para el estudio: los usuarios básicos permiten evaluar la curva de aprendizaje inicial y la barrera de entrada; los intermedios ponen a prueba la utilidad didáctica real del sistema; y los avanzados actúan como validadores de la precisión técnica y el rendimiento de las consultas soportadas por el motor de SeeQL.

6.3.2. Evaluación de dimensiones de usabilidad y diseño

Uno de los pilares de SeeQL es ofrecer una interfaz moderna, intuitiva y que reduzca la fricción habitual de las consolas de bases de datos tradicionales. Se solicitó a los usuarios que valoraran del 1 al 5 (donde 1 indica *totalmente en desacuerdo* y 5 *totalmente de acuerdo*) diversas dimensiones de la aplicación, obteniendo los resultados reflejados en la Figura 6.2.

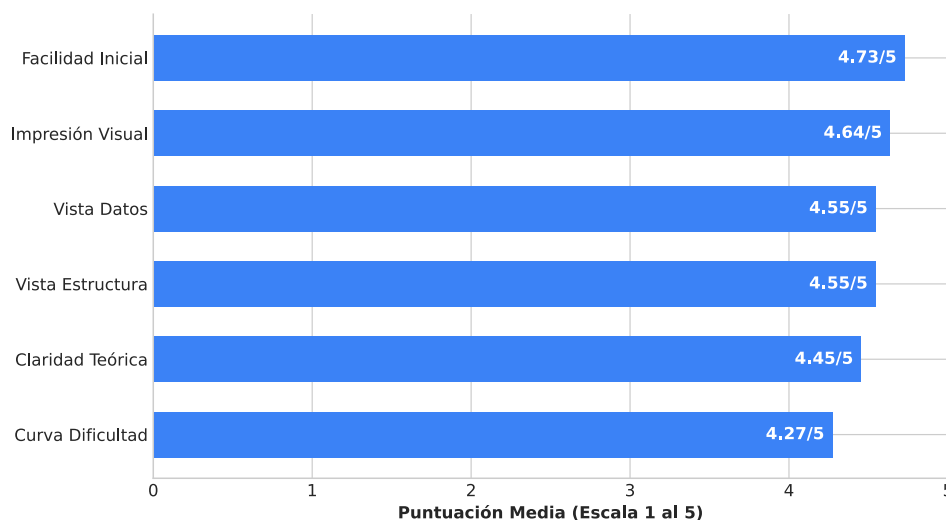


Figura 6.2: Evaluación media de las dimensiones de usabilidad y diseño del sistema (Escala 1-5).

Las métricas revelan una aceptación sobresaliente en el apartado visual y de experiencia de usuario. Destaca especialmente la *facilidad inicial* (4.73/5), lo que demuestra que el *onboarding* y la disposición de la interfaz comunican eficazmente el propósito de la herramienta desde el primer segundo. Del mismo modo, tanto la *vista estructura* como la *vista datos* obtienen una nota de 4.55/5, confirmando que la división del espacio de trabajo ayuda a los alumnos a abstraer el modelo relacional.

Es particularmente interesante el resultado de la *curva de dificultad* (4.27/5). Siendo la puntuación más baja del conjunto, sigue representando un notable alto. En el contexto de una aplicación de aprendizaje gamificada, que esta métrica no alcance la nota máxima es, de hecho, un indicador positivo: demuestra que los retos finales suponen un verdadero desafío cognitivo y no resultan triviales, manteniendo el interés del estudiante.

6.3.3. Interacción pedagógica: El uso de ayudas

Para analizar cómo los usuarios utilizan las ayudas pedagógicas de la plataforma, se cruzó el nivel de conocimientos previos de cada alumno con su interacción con el sistema de pistas/ayudas (resultado esperado y pista). Los resultados (Figura 6.3) revelan patrones de comportamiento muy definidos y descubrimientos inesperados en el plano de la experiencia de usuario.

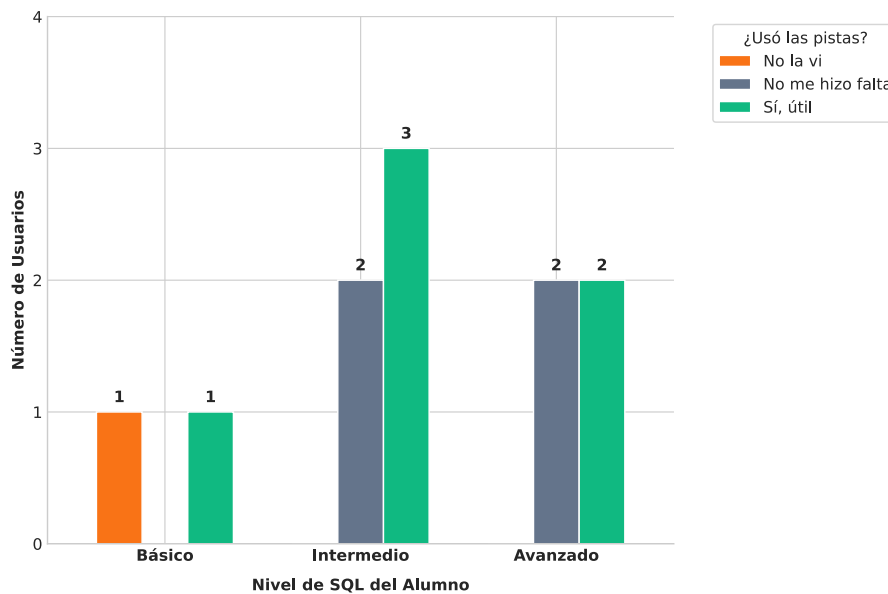


Figura 6.3: Relación entre el nivel previo de SQL y el uso de las pistas integradas en la interfaz.

En el caso de los usuarios avanzados, los resultados muestran un comportamiento dividido: dos participantes resolvieron los ejercicios sin necesidad de consultar las ayudas, mientras que los otros dos utilizaron las pistas y las valoraron positivamente. Por otra parte, el grupo intermedio fue el que más recurrió a esta funcionalidad: la gran mayoría utilizó las pistas y las calificó como útiles para superar los bloqueos.

Sin embargo, el hallazgo más revelador del estudio radica en el comportamiento de los usuarios con un nivel básico. De este grupo, un porcentaje significativo reportó no haberse percatado de la presencia de las pistas. Lejos de ser un fallo funcional, esto ilustra un fenómeno clásico de carga cognitiva: cuando un usuario novato se enfrenta a un entorno técnico completamente nuevo y escribe código SQL por primera vez, puede desarrollar un efecto de visión de túnel. Su atención se centra de forma tan exclusiva en el problema principal (el editor de código y el enunciado) que el cerebro ignora los elementos periféricos de la interfaz, por muy accesibles que estos sean. Esta conclusión abre una interesante línea de trabajo futuro para SeeQL: la implementación de un sistema de pistas dinámico o proactivo que, tras un tiempo determinado de inactividad o fallos reiterados, ofrezca ayuda visualmente intrusiva a los usuarios más inexpertos.

6.3.4. Fidelización y viabilidad futura

Como métrica final para medir el éxito global del proyecto y su viabilidad como producto educativo a largo plazo, se evaluó la intención de retorno de los usuarios.

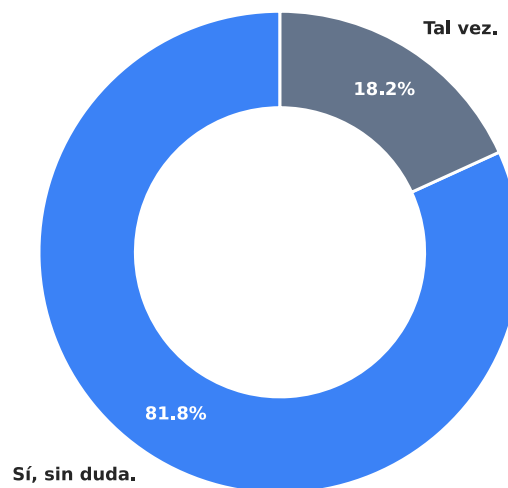


Figura 6.4: Intención de los usuarios de seguir utilizando la plataforma para aprender SQL.

Ante la pregunta de si seguirían utilizando SeeQL en caso de existir más niveles, un contundente 81.8 % respondió *Sí, sin duda*, mientras que el 18.2 % restante indicó *Tal vez* (Figura 6.4). Resulta especialmente destacable que ningún usuario haya descartado la herramienta. Esta alta tasa de retención teórica valida el enfoque arquitectónico, visual y pedagógico del TFG, confirmando que la gamificación y la retroalimentación visual en tiempo real son mecanismos altamente efectivos para la enseñanza de lenguajes de consulta de bases de datos.

6.4 Análisis de resultados: Perfil docente

Para validar la utilidad de SeeQL desde una perspectiva pedagógica y profesional, se ha contado con la participación de un panel de expertos compuesto por tres docentes con experiencia en la enseñanza de Bases de Datos y SQL. Dada la naturaleza técnica del proyecto, esta validación cualitativa resulta fundamental para asegurar que la herramienta cumple con los estándares requeridos en un entorno educativo universitario.

6.4.1. Valoración cuantitativa de expertos

Los resultados obtenidos a través del cuestionario docente reflejan una aceptación excepcional de la herramienta. Como se ilustra en la Figura 6.5, las valoraciones se sitúan de forma unánime en el rango máximo de la escala de Likert de cinco puntos (donde 1 indica *totalmente en desacuerdo* y 5 *totalmente de acuerdo*).

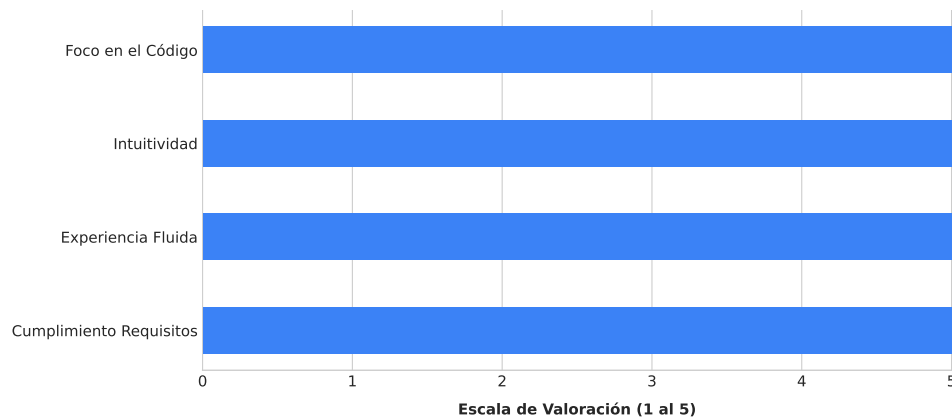


Figura 6.5: Evaluación de dimensiones pedagógicas y de usabilidad por parte de docentes.

Esta consistencia absoluta en las respuestas indica que SeeQL no solo es percibida como una herramienta intuitiva y carente de frustración para el usuario, sino que se alinea perfectamente con los requisitos docentes. Los expertos coinciden en que la aplicación permite al alumno observar de manera directa el impacto real de sus consultas, eliminando la barrera de abstracción que suele dificultar el aprendizaje del modelo relacional.

6.4.2. Análisis cualitativo del *feedback*

El *feedback* abierto proporcionado por los docentes permite identificar los puntos fuertes que hacen de SeeQL un recurso didáctico de alto valor:

- **Control del flujo de aprendizaje:** Se valora positivamente que el sistema de itinerario impida el acceso a contenidos avanzados sin haber consolidado previamente los conceptos básicos, asegurando una progresión lógica del estudiante.
- **Fidelidad del entorno:** La interfaz es percibida como un entorno de programación real gracias a funciones como el autocompletado inteligente y la inspección de esquemas.
- **Retroalimentación visual inmediata:** El mayor valor destacado es la posibilidad de inspeccionar qué parte específica de la base de datos está siendo afectada por una consulta. Según los expertos, esta característica fomenta el autoaprendizaje y la detección autónoma de errores lógicos.

En conclusión, la validación docente confirma que SeeQL es una herramienta viable y recomendada para su implementación como apoyo en el aula, logrando reducir la carga cognitiva del alumno y facilitando la labor del tutor mediante una interfaz limpia y centrada en el código.

CAPÍTULO 7

Conclusiones y trabajo futuro

En este capítulo final se recapitulan los resultados obtenidos durante el desarrollo de la plataforma SeeQL, analizando el grado de cumplimiento de los objetivos planteados inicialmente. Asimismo, se exponen las líneas de trabajo futuro identificadas tras el proceso de evaluación empírica y las reflexiones personales sobre el impacto del proyecto.

7.1 Cumplimiento de objetivos

Tras la finalización del ciclo de desarrollo e implementación, se puede afirmar que el proyecto ha cumplido satisfactoriamente con el objetivo general de diseñar y construir una plataforma interactiva, segura y escalable para el aprendizaje de SQL.

A continuación, se detalla la consecución de los hitos técnicos y funcionales específicos:

- **Arquitectura *client-side computing* segura:** Se ha logrado implementar con éxito un modelo de ejecución íntegramente en el lado del cliente mediante WebAssembly. Esta decisión técnica elimina la latencia de red en el procesamiento de consultas y garantiza la integridad de la infraestructura, permitiendo un entorno de experimentación libre con un coste de servidor nulo y seguridad absoluta por aislamiento.
- **Motor de visualización reactivo:** La plataforma traduce de forma efectiva la lógica de las consultas en una representación gráfica dinámica. El diseño trilateral inamovible y el uso de semántica cromática reducen drásticamente la carga cognitiva del estudiante.
- **Itinerario pedagógico gamificado:** Se ha consolidado un catálogo de 10 niveles que guían al usuario desde conceptos básicos hasta composiciones relacionales complejas, apoyados por un motor de validación algorítmica flexible y tolerante a variaciones sintácticas.
- **Gestión de acceso y persistencia:** La integración con Supabase permite un seguimiento fiable del progreso académico de los estudiantes registrados, manteniendo de forma simultánea un acceso fluido para usuarios invitados.

7.2 Trabajo futuro

A pesar de haber alcanzado un MVP robusto, la naturaleza modular y escalable de SeeQL permite trazar una ambiciosa hoja de ruta. Basado en el análisis técnico y en el

feedback obtenido durante la evaluación con usuarios, se proponen las siguientes vías de expansión:

1. **Expansión a sentencias de manipulación y definición:** Aunque la versión actual se centra en la consulta `SELECT`, la arquitectura basada en `SQL.js` permite una transición inmediata hacia módulos dedicados a `INSERT`, `UPDATE`, `DELETE` y `CREATE`. Esto permitiría al alumno visualizar cómo se altera físicamente la estructura de las tablas y las restricciones de integridad en tiempo real.
2. **Importación de esquemas personalizados en el modo *Sandbox*:** Ampliar el laboratorio de experimentación permitiendo a los usuarios subir sus propios archivos `.sql`. Esto requeriría desarrollar un módulo de validación previo a la ejecución para instanciar bases de datos personalizadas de forma segura.
3. **Evolución del grafo relacional (aportación docente):** Atendiendo al *feedback* recibido por los expertos académicos, se propone mejorar el motor de renderizado de `React Flow` para incluir marcadores visuales explícitos en las claves foráneas y restricciones de nulidad, emulando la notación estándar del modelado de datos.
4. **Herramientas de exportación e historial:** Implementar un historial de sentencias ejecutadas en el editor y añadir funcionalidades para descargar los conjuntos de resultados obtenidos en formatos estándar como `CSV` o `JSON`, facilitando la integración de la herramienta con herramientas de análisis de datos externas.
5. **Panel de administración académica (modo docente):** Desarrollar un *dashboard* para profesores que permita visualizar métricas de rendimiento de sus grupos de alumnos, identificar qué conceptos SQL generan mayor tasa de error estadístico y permitir la creación de lecciones propias mediante un editor visual.
6. **Internacionalización y accesibilidad:** Aprovechar la arquitectura de estados globales ya implementada para facilitar la inyección dinámica de diccionarios de traducción. Asimismo, se propone la integración de esquemas de alto contraste y temas claros para cumplir con las normativas de accesibilidad web universales (WCAG) [36], garantizando una experiencia inclusiva para usuarios con diversas necesidades visuales.

7.3 Reflexión personal

El desarrollo de SeeQL ha supuesto un reto técnico e intelectual muy significativo, permitiéndome consolidar y unificar gran parte de las competencias transversales adquiridas a lo largo del Grado en Ingeniería Informática.

A nivel técnico, la concepción de una aplicación SPA de esta envergadura me ha exigido profundizar en arquitecturas modernas de *frontend*, en la sincronización de estados reactivos complejos y en la integración de tecnologías de vanguardia como `WebAssembly`. A nivel de producto, este proyecto me ha enseñado la importancia crítica de la experiencia de usuario y cómo el diseño de interfaces optimizadas para reducir la fatiga cognitiva [25] impacta directamente en la asimilación de conceptos técnicos.

Considero que SeeQL aporta una solución tecnológica tangible a una carencia real detectada en las aulas: la necesidad de «ver» el SQL. Como apasionado del desarrollo *frontend*, siempre he creído que la interfaz no es solo una capa estética, sino una herramienta pedagógica en sí misma. Lograr que un estudiante observe cómo dos tablas se

cruzan e interactúan visualmente en cuestión de milisegundos convierte una tarea tradicionalmente abstracta en un proceso de descubrimiento activo. Para mí, este proyecto ha sido la oportunidad perfecta de unir mi vocación por la enseñanza con mi interés por crear productos visualmente atractivos, demostrando que la ingeniería de software es, en esencia, una herramienta para facilitar el aprendizaje y mejorar la experiencia de las personas.

Bibliografía

- [1] T. J. van Weert and F. Mulder, "Modern curriculum development for informatics (computing science)," in *Informatics and the Digital Society: Social, Ethical and Cognitive Issues*, pp. 285–296, Springer, 2003.
- [2] P. Aithal, "Student centric curriculum design and implementation—challenges & opportunities in business management & it education," *IRA International Journal of Education and Multidisciplinary Studies*, vol. 4, no. 3, pp. 423–437, 2016.
- [3] T. Taipalus and V. Seppänen, "Sql education: A systematic mapping study and future research agenda," *ACM Transactions on Computing Education (TOCE)*, vol. 20, no. 3, pp. 1–33, 2020.
- [4] UPV, "Guía docente de la asignatura: Bases de datos (11547)," 2025.
- [5] M. Celma Giménez, J. C. Casamayor Ródenas, and L. Mota Herranz, *Bases de datos relacionales*. Madrid: Pearson Prentice Hall, 2003.
- [6] T. Taipalus, M. Siponen, and T. Vartiainen, "Errors and complications in sql query formulation," *ACM Transactions on Computing Education (TOCE)*, vol. 18, no. 3, pp. 1–29, 2018.
- [7] F. M. Van der Kleij, R. C. Feskens, and T. J. Eggen, "Effects of feedback in a computer-based learning environment on students' learning outcomes: A meta-analysis," *Review of educational research*, vol. 85, no. 4, pp. 475–511, 2015.
- [8] I. Meta Platforms, "React - a javascript library for building user interfaces." <https://react.dev/>, 2026. Consultado el 8 de mayo de 2026.
- [9] Microsoft, "Typescript: Javascript with syntax for types." <https://www.typescriptlang.org/>, 2026. Consultado el 8 de mayo de 2026.
- [10] Supabase, "Supabase: The open source firebase alternative." <https://supabase.com/>, 2026. Consultado el 8 de mayo de 2026.
- [11] M. Staron, *Action research in software engineering*. Springer, 2020.
- [12] Git, "Git: Fast, distributed version control system." <https://git-scm.com/>, 2026. Consultado el 8 de mayo de 2026.
- [13] I. GitHub, "Github: Where the world builds software." <https://github.com/>, 2026. Consultado el 8 de mayo de 2026.
- [14] B. Marín, "Lessons learned about gamification in software engineering education," in *Research anthology on developments in gamification and game-based learning*, pp. 1473–1496, IGI Global Scientific Publishing, 2022.

- [15] B. Marín, “Gamification to ignite learning in modern times (keynote),” in *Proceedings of the 2nd International Workshop on Gamification in Software Development, Verification, and Validation*, pp. 1–1, 2023.
- [16] B. Marín, J. Frez, J. Cruz-Lemus, and M. Genero, “An empirical investigation on the benefits of gamification in programming courses,” *ACM Transactions on Computing Education (TOCE)*, vol. 19, no. 1, pp. 1–22, 2018.
- [17] M. Soflano, T. Connolly, and T. Hainey, “An application of adaptive games-based learning based on learning style to teach sql,” in *Proceedings of the 7th European Conference on Games Based Learning*, pp. 531–538, Academic Conferences and Publishing International Limited, 2013.
- [18] E. Silva, R. Fidalgo, M. Ferro, and N. Franco, “Visual query languages to design complex queries: a systematic literature review,” *Software and Systems Modeling*, vol. 22, no. 4, pp. 1217–1249, 2023.
- [19] M. Jung and U. Oh, “Sqlgo: Design of a node-based interface for learning sql,” *International journal of advanced smart convergence*, pp. 270–278, 2025.
- [20] T. Balla, S. Kiraly, and R. Kiraly, “Enhancing sql programming assessments through educational games: a gamified approach,” *Discover Education*, vol. 4, no. 1, p. 365, 2025.
- [21] C. Ayora, J. L. de la Vara, B. Marín, and G. Giachetti, “Rethinking class exercises for teaching database management in the genai era,” in *Companion Proceedings of the 44th International Conference on Conceptual Modeling (ER 2025)*, (Poitiers, France), October 2025.
- [22] A. Álvarez and P. Del Río, “Educación y desarrollo: la teoría de Vigotsky y la zona de desarrollo próximo,” *Desarrollo psicológico y educación*, vol. 2, no. 0, pp. 93–120, 1990.
- [23] T. Kravchenko, T. Bogdanova, and T. Shevgunov, “Ranking requirements using moscow methodology in practice,” in *Computer science on-line conference*, pp. 188–199, Springer, 2022.
- [24] Object Management Group (OMG), *Unified Modeling Language (UML) Specification, Version 2.5.1*, 2017. Consultado el 8 de mayo de 2026.
- [25] C. Panakaduwa, P. Coates, M. Munir, and O. De Silva, “Optimising visual user interfaces to reduce cognitive fatigue and enhance mental well-being,” in *Proceedings of the 35th Annual Conference of the International Information Management Association (IIMA)*, Manchester, UK, pp. 2–4, 2024.
- [26] A. Banks and E. Porcello, *Learning React: modern patterns for developing React apps*. O’Reilly Media, 2020.
- [27] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J.-F. Bastien, “Bringing the web up to speed with webassembly,” in *Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation*, pp. 185–200, 2017.
- [28] A. Jangda, B. Powers, E. D. Berger, and A. Guha, “Not so fast: Analyzing the performance of {WebAssembly} vs. native code,” in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pp. 107–120, 2019.

- [29] E. Redmond, L. Perkins, and J. Wilson, *Seven databases in seven weeks: a guide to modern databases and the NoSQL movement*. The Pragmatic Bookshelf, 2018.
- [30] J. Jones, “Abstract syntax tree implementation idioms,” tech. rep., Department of Computer Science, University of Alabama, 2003.
- [31] J. Nielsen, “10 usability heuristics for user interface design.” <https://www.nngroup.com/articles/ten-usability-heuristics/>, 1994. Consultado el 8 de mayo de 2026.
- [32] Vitest, “Vitest: A native next-generation testing framework based on vite.” <https://vitest.dev/>, 2026. Consultado el 8 de mayo de 2026.
- [33] J. Brooke *et al.*, “Sus-a quick and dirty usability scale,” *Usability evaluation in industry*, vol. 189, no. 194, pp. 4–7, 1996.
- [34] F. D. Davis, “Perceived usefulness, perceived ease of use, and user acceptance of information technology,” *MIS quarterly*, vol. 13, no. 3, pp. 319–340, 1989.
- [35] K. Finstad, “The usability metric for user experience,” *Interacting with computers*, vol. 22, no. 5, pp. 323–327, 2010.
- [36] W3C, “Web content accessibility guidelines (wcag) 2.2.” <https://www.w3.org/TR/WCAG22/>, 2023. Consultado el 8 de mayo de 2026.

APÉNDICE A

Esquema de datos y scripts SQL

El presente anexo detalla la estructura física de datos y las sentencias de inserción inicial correspondientes a los tres entornos relacionales diseñados para los itinerarios didácticos de la plataforma **SeeQL**.

Con el propósito de mantener una coherencia absoluta con la arquitectura técnica del sistema, las especificaciones expuestas en este documento se presentan bajo el estándar de **SQLite**. Dado que el núcleo evaluativo y de visualización de la aplicación procesa las instrucciones de forma 100 % local en el cliente mediante el binario WebAssembly de **SQL.js**, este es el dialecto real que se instancia en la memoria del navegador durante la interacción del estudiante. Por tanto, se ha omitido la sintaxis propia de PostgreSQL, como la organización en *schemas* o la gestión manual de las secuencias. Esto permite ofrecer bases de datos más limpias y fáciles de replicar en local, conservando perfectamente sus claves y relaciones estructurales.

Asimismo, con el objetivo de garantizar la transparencia metodológica y maximizar la replicabilidad del proyecto por parte del tribunal evaluador, la distribución de datos se ha estructurado de forma dual dentro del repositorio de código fuente adjunto, organizándose en el directorio raíz bajo la ruta `/database/`:

- **Subdirectorio** `/postgres_backend/`: Alberga los *scripts* en dialecto PostgreSQL optimizados para su ejecución directa en el editor SQL de Supabase. Este bloque permite levantar e inicializar por completo el servidor de persistencia en la nube con las restricciones de integridad y estructuras multi-esquema nativas de la infraestructura de producción.
- **Subdirectorio** `/sqlite_frontend/`: Contiene los archivos ejecutables con extensión `.sql` que se corresponden unívocamente con las estructuras de este anexo. Estos componentes están diseñados para una réplica ágil e inmediata en entornos locales *offline*, facilitando su importación en cualquier gestor de bases de datos ligero compatible con SQLite sin necesidad de configurar servicios de red ni credenciales en la nube.

A continuación, se exponen de forma secuencial las sentencias de creación y población que dan soporte a los tres niveles de complejidad educativa de la plataforma.

A.1 Esquema educativo 1: Biblioteca (dificultad básica)

El esquema inicial constituye el entorno de trabajo para los niveles básicos de la plataforma. Su arquitectura relacional está diseñada para introducir al estudiante en el diseño

de relaciones estándar de uno a muchos (1:N), simulando la gestión de inventario, la categorización de obras y el registro temporal de préstamos mediante estructuras sencillas que minimizan la carga cognitiva inicial.

```

1 CREATE TABLE autores (
2   id_autor INTEGER PRIMARY KEY AUTOINCREMENT,
3   nombre VARCHAR NOT NULL,
4   nacionalidad VARCHAR,
5   fecha_nacimiento DATE
6 );
7
8 CREATE TABLE categorias (
9   id_categoria INTEGER PRIMARY KEY AUTOINCREMENT,
10  nombre_cat VARCHAR NOT NULL UNIQUE
11 );
12
13 CREATE TABLE libros (
14   id_libro INTEGER PRIMARY KEY AUTOINCREMENT,
15   titulo VARCHAR NOT NULL,
16   id_autor INTEGER REFERENCES autores(id_autor),
17   id_categoria INTEGER REFERENCES categorias(id_categoria),
18   isbn VARCHAR UNIQUE,
19   anio_publicacion INTEGER,
20   copias_disponibles INTEGER DEFAULT 0
21 );
22
23 CREATE TABLE usuarios (
24   id_usuario INTEGER PRIMARY KEY AUTOINCREMENT,
25   nombre_completo VARCHAR NOT NULL,
26   email VARCHAR NOT NULL UNIQUE,
27   fecha_registro DATE DEFAULT (DATE('now'))
28 );
29
30 CREATE TABLE prestamos (
31   id_prestamo INTEGER PRIMARY KEY AUTOINCREMENT,
32   id_libro INTEGER REFERENCES libros(id_libro),
33   id_usuario INTEGER REFERENCES usuarios(id_usuario),
34   fecha_salida DATE NOT NULL DEFAULT (DATE('now')),
35   fecha_devolucion_prevista DATE NOT NULL,
36   devuelto BOOLEAN DEFAULT false
37 );
38
39 INSERT INTO autores (id_autor, nombre, nacionalidad, fecha_nacimiento) VALUES
40 (1, 'Robert C. Martin', 'USA', '1952-12-05'),
41 (2, 'J.K. Rowling', 'UK', '1965-07-31'),
42 (3, 'Miguel de Cervantes', 'España', '1547-09-29'),
43 (4, 'George R.R. Martin', 'USA', '1948-09-20'),
44 (5, 'Mary Shelley', 'UK', '1797-08-30');
45
46 INSERT INTO categorias (id_categoria, nombre_cat) VALUES
47 (1, 'Tecnología'),
48 (2, 'Fantasía'),
49 (3, 'Clásicos'),
50 (4, 'Terror');
51
52 INSERT INTO libros (id_libro, titulo, id_autor, id_categoria, isbn,
53   anio_publicacion, copias_disponibles) VALUES
54 (1, 'Clean Code', 1, 1, '978-0132350884', 2008, 5),
55 (2, 'Harry Potter', 2, 2, '978-8478884451', 1997, 3),
56 (3, 'Don Quijote', 3, 3, '978-8424116347', 1605, 1),
57 (4, 'Harry Potter y el Cáliz', 2, 2, '978-8478886456', 2000, 2),
58 (5, 'Juego de Tronos', 4, 2, '978-8496208377', 1996, 0),
59 (6, 'Frankenstein', 5, 4, '978-8420676063', 1818, 5),
60 (7, 'Danza de Dragones', 4, 2, '978-8496208919', 2011, 2),

```

```

60 (8, 'Cantar de mio Cid', NULL, 3, '978-8420676000', 1200, 2);
61
62 INSERT INTO usuarios (id_usuario, nombre_completo, email, fecha_registro)
63 VALUES
64 (1, 'Laura Mota', 'laura.mota@upv.es', '2026-01-07'),
65 (2, 'Pablo García', 'pgarcia@gmail.com', '2026-02-14'),
66 (3, 'Sergio Blanco', 'sergio740@gmail.com', '2026-03-03'),
67 (4, 'Marta González', 'margon@yahoo.com', '2026-04-10');
68
69 INSERT INTO prestamos (id_prestamo, id_libro, id_usuario, fecha_salida,
70 fecha_devolucion_prevista, devuelto) VALUES
71 (1, 1, 1, '2026-03-04', '2026-04-01', false),
72 (2, 2, 2, '2026-03-04', '2026-04-10', false),
73 (3, 4, 1, '2026-03-15', '2026-04-15', false),
74 (4, 6, 3, '2026-02-01', '2026-03-01', true);

```

Listing A.1: Script DDL en sintaxis SQLite para la creación del esquema Biblioteca.

A.2 Esquema educativo 2: Gimnasio (dificultad media)

El esquema de nivel medio eleva la complejidad introduciendo tipos de datos temporales (TIME, DATE), numéricos exactos (NUMERIC) y una mayor densidad de relaciones. Está orientado a la práctica intermedia de sentencias de agrupación y cruces relacionales.

```

1 CREATE TABLE entrenadores (
2   id_entrenador INTEGER PRIMARY KEY AUTOINCREMENT,
3   nombre VARCHAR NOT NULL,
4   especialidad VARCHAR,
5   email VARCHAR UNIQUE
6 );
7
8 CREATE TABLE clases (
9   id_clase INTEGER PRIMARY KEY AUTOINCREMENT,
10  nombre_clase VARCHAR NOT NULL,
11  id_entrenador INTEGER REFERENCES entrenadores(id_entrenador),
12  horario_inicio TIME,
13  capacidad_max INTEGER DEFAULT 20
14 );
15
16 CREATE TABLE clientes (
17   id_cliente INTEGER PRIMARY KEY AUTOINCREMENT,
18   nombre_completo VARCHAR NOT NULL,
19   email VARCHAR NOT NULL UNIQUE,
20   fecha_inscripcion DATE DEFAULT (DATE('now'))
21 );
22
23 CREATE TABLE pagos (
24   id_pago INTEGER PRIMARY KEY AUTOINCREMENT,
25   id_cliente INTEGER REFERENCES clientes(id_cliente),
26   monto NUMERIC NOT NULL,
27   metodo_pago VARCHAR,
28   fecha_pago DATE DEFAULT (DATE('now'))
29 );
30
31 CREATE TABLE reservas (
32   id_reserva INTEGER PRIMARY KEY AUTOINCREMENT,
33   id_cliente INTEGER REFERENCES clientes(id_cliente),
34   id_clase INTEGER REFERENCES clases(id_clase),
35   dia_reserva DATE
36 );
37

```

```

38 INSERT INTO entrenadores (id_entrenador, nombre, especialidad, email) VALUES
39 (1, 'Alex Fit', 'Crossfit', 'alex@gym.com'),
40 (2, 'Sara Yoga', 'Pilates', 'sara@gym.com'),
41 (3, 'Dani Box', 'Boxeo', 'dani@gym.com'),
42 (4, 'Rodrigo', 'Kárate', 'rodrigo@gym.com');
43
44 INSERT INTO clientes (id_cliente, nombre_completo, email, fecha_inscripcion)
45     VALUES
46 (1, 'Marc Torres', 'mtorres@gmail.com', '2026-02-02'),
47 (2, 'Paula Ruiz', 'pruiz@yahoo.es', '2026-03-05'),
48 (3, 'Jordi Valls', 'jvalls@gmail.com', '2026-03-20'),
49 (4, 'Carla Sanz', 'csanz@outlook.com', '2026-02-25');
50
51 INSERT INTO clases (id_clase, nombre_clase, id_entrenador, horario_inicio,
52     capacidad_max) VALUES
53 (1, 'Crossfit Mañana', 1, '08:00:00', 20),
54 (2, 'Yoga Flow', 2, '19:00:00', 25),
55 (3, 'Boxeo Intenso', 3, '20:30:00', 10),
56 (4, 'Estiramientos en grupo', NULL, '10:00:00', 20);
57
58 INSERT INTO pagos (id_pago, id_cliente, monto, metodo_pago, fecha_pago) VALUES
59 (1, 1, 50, 'Tarjeta', '2026-03-10'),
60 (2, 1, 51, 'Efectivo', '2026-03-15'),
61 (3, 2, 50, 'PayPal', '2026-03-12'),
62 (4, 2, 50, 'Tarjeta', '2026-03-20'),
63 (5, 3, 49, 'Efectivo', '2026-03-22'),
64 (6, 3, 45, 'Tarjeta', '2026-03-25'),
65 (7, 3, 5, 'Efectivo', '2026-03-25'),
66 (8, 4, 55, 'Tarjeta', '2026-02-28'),
67 (9, 4, 45, 'Tarjeta', '2026-03-05'),
68 (10, 4, 20, 'PayPal', '2026-03-05');
69
70 INSERT INTO reservas (id_reserva, id_cliente, id_clase, dia_reserva) VALUES
71 (1, 1, 1, '2026-03-10'),
72 (2, 1, 2, '2026-03-15'),
73 (3, 2, 1, '2026-03-12'),
74 (4, 2, 3, '2026-03-20'),
75 (5, 3, 2, '2026-03-22'),
76 (6, 3, 3, '2026-03-25'),
77 (7, 4, 1, '2026-02-28'),
78 (8, 4, 2, '2026-03-05');

```

Listing A.2: Script DDL en sintaxis SQLite para la creación del esquema Gimnasio.

A.3 Esquema educativo 3: Aeropuerto (dificultad avanzada)

El esquema avanzado es el modelo más complejo, diseñado para validar las destrezas adquiridas. Presenta entidades fuertemente acopladas, como la tabla vuelos, la cual concentra múltiples dependencias obligando al estudiante a formular composiciones (*JOIN*) de alto nivel.

```

1 CREATE TABLE aeropuertos (
2     id_aeropuerto INTEGER PRIMARY KEY AUTOINCREMENT,
3     nombre VARCHAR NOT NULL,
4     ciudad VARCHAR,
5     codigo_iata VARCHAR NOT NULL UNIQUE
6 );
7
8 CREATE TABLE modelos_avion (
9     id_modelo INTEGER PRIMARY KEY AUTOINCREMENT,
10    fabricante VARCHAR,

```

```

11 nombre_modelo VARCHAR,
12 capacidad_pasajeros INTEGER
13 );
14
15 CREATE TABLE aviones (
16     id_avion INTEGER PRIMARY KEY AUTOINCREMENT,
17     matricula VARCHAR NOT NULL UNIQUE,
18     id_modelo INTEGER REFERENCES modelos_avion(id_modelo),
19     estado VARCHAR DEFAULT 'activo'
20 );
21
22 CREATE TABLE pasajeros (
23     id_pasajero INTEGER PRIMARY KEY AUTOINCREMENT,
24     nombre_completo VARCHAR NOT NULL,
25     pasaporte VARCHAR NOT NULL UNIQUE,
26     nacionalidad VARCHAR
27 );
28
29 CREATE TABLE pilotos (
30     id_piloto INTEGER PRIMARY KEY AUTOINCREMENT,
31     nombre_completo VARCHAR NOT NULL,
32     licencia VARCHAR UNIQUE,
33     horas_vuelo INTEGER DEFAULT 0
34 );
35
36 CREATE TABLE vuelos (
37     id_vuelo INTEGER PRIMARY KEY AUTOINCREMENT,
38     numero_vuelo VARCHAR NOT NULL UNIQUE,
39     id_avion INTEGER REFERENCES aviones(id_avion),
40     id_aeropuerto_origen INTEGER REFERENCES aeropuertos(id_aeropuerto),
41     id_aeropuerto_destino INTEGER REFERENCES aeropuertos(id_aeropuerto),
42     id_piloto INTEGER REFERENCES pilotos(id_piloto)
43 );
44
45 CREATE TABLE reservas (
46     id_reserva INTEGER PRIMARY KEY AUTOINCREMENT,
47     id_vuelo INTEGER REFERENCES vuelos(id_vuelo),
48     id_pasajero INTEGER REFERENCES pasajeros(id_pasajero),
49     asiento VARCHAR,
50     clase VARCHAR DEFAULT 'turista',
51     precio_billete NUMERIC
52 );
53
54 INSERT INTO aeropuertos (id_aeropuerto, nombre, ciudad, codigo_iata) VALUES
55 (1, 'Adolfo Suárez Madrid-Barajas', 'Madrid', 'MAD'),
56 (2, 'Josep Tarradellas Barcelona-El Prat', 'Barcelona', 'BCN'),
57 (3, 'John F. Kennedy International', 'New York', 'JFK'),
58 (4, 'London Heathrow', 'London', 'LHR');
59
60 INSERT INTO modelos_avion (id_modelo, fabricante, nombre_modelo,
61     capacidad_pasajeros) VALUES
62 (1, 'Airbus', 'A320neo', 186),
63 (2, 'Boeing', '787-9 Dreamliner', 290),
64 (3, 'Boeing', '787 Dreamliner', 250),
65 (4, 'Boeing', '737 MAX', 170),
66 (5, 'Airbus', 'A380', 505);
67
68 INSERT INTO pasajeros (id_pasajero, nombre_completo, pasaporte, nacionalidad)
69     VALUES
70 (1, 'Marta Vidal', 'PA9876543', 'Española'),
71 (2, 'James Wilson', 'US1234567', 'Estadounidense'),
72 (3, 'Laura Miller', 'UK7654321', 'Británica'),
73 (4, 'John Smith', 'US9988776', 'Estadounidense'),
74 (5, 'Emma Watson', 'UK1122334', 'Británica');

```

```
73
74 INSERT INTO pilotos (id_piloto, nombre_completo, licencia, horas_vuelo) VALUES
75 (1, 'Javier Mendoza', 'EASA-102938', 4500),
76 (2, 'Robert Harrison', 'FAA-992011', 8200),
77 (3, 'Luisa Fernández', 'EASA-33', 5200);
78
79 INSERT INTO aviones (id_avion, matricula, id_modelo, estado) VALUES
80 (1, 'EC-NPR', 1, 'activo'),
81 (2, 'N834AA', 2, 'activo'),
82 (3, 'N888XX', 3, 'activo'),
83 (4, 'EC-LXY', 1, 'activo'),
84 (5, 'EC-MZS', 1, 'mantenimiento'),
85 (6, 'N900AA', 2, 'activo'),
86 (7, 'N899XX', 3, 'inactivo'),
87 (8, 'N737ZZ', 4, 'activo');
88
89 INSERT INTO vuelos (id_vuelo, numero_vuelo, id_avion, id_aeropuerto_origen,
90 id_aeropuerto_destino, id_piloto) VALUES
91 (1, 'IB0123', 1, 1, 2, 1),
92 (2, 'BA0116', 2, 3, 4, 2),
93 (3, 'IB0124', 1, 1, 3, 1),
94 (4, 'BA0117', 3, 3, 1, 3),
95 (5, 'BA0118', 3, 3, 2, 3);
96
97 INSERT INTO reservas (id_reserva, id_vuelo, id_pasajero, asiento, clase,
98 precio_billete) VALUES
99 (1, 1, 1, '14F', 'turista', 75.5),
100 (2, 2, 2, '02A', 'business', 1450),
101 (3, 2, 3, '22C', 'turista', 420),
(4, 3, 4, '12A', 'business', 1450),
(5, 4, 5, '30C', 'turista', 85.5);
```

Listing A.3: Script DDL en sintaxis SQLite para la creación del esquema Aeropuerto.

APÉNDICE B

Instrumentos de evaluación

En este apéndice se detallan los cuestionarios exactos suministrados a los participantes durante la fase de validación empírica de SeeQL, descrita en el Capítulo 6.

Para las preguntas de valoración cuantitativa, ambos cuestionarios utilizan una escala Likert de 5 puntos (donde 1 indica *Totalmente en desacuerdo* y 5 indica *Totalmente de acuerdo*), salvo que se indique un formato de respuesta abierta o de selección múltiple. Las preguntas marcadas con un asterisco (*) corresponden a ítems con escala invertida, diseñados para medir la frustración o la carga operativa y garantizar la fiabilidad de las respuestas.

B.1 Cuestionario para estudiantes

Dimensión 1: Demografía y conocimiento previo

- ¿Cuál es tu nivel previo de conocimientos sobre bases de datos y SQL?
 - Básico (Sé qué es pero nunca he profundizado).
 - Intermedio (He escrito consultas sencillas).
 - Avanzado (Trabajo con SQL o tengo un nivel profundo).
- ¿Con qué frecuencia aprendes nuevas herramientas o lenguajes?
 - Raramente o nunca.
 - De vez en cuando.
 - Habitualmente.

Dimensión 2: Usabilidad y experiencia de usuario

- Valora del 1 al 5 la primera impresión visual de la aplicación.
- ¿Te resultó fácil entender para qué sirve la app nada más entrar? (*Escala 1-5*)
- ¿Tuviste algún problema al moverte, redimensionar paneles o usar la consola? Cuéntame tu experiencia. (*Respuesta abierta*)

Dimensión 3: Efectividad de la representación visual

- ¿Te pareció útil la vista “Estructura” para comprender cómo se relacionan las tablas? (*Escala 1-5*)

- ¿Te pareció útil la vista “Datos” para entender qué información real contiene cada tabla? (*Escala 1-5*)

Dimensión 4: Valor pedagógico y herramientas de apoyo

- ¿Qué te pareció la claridad de las explicaciones teóricas y de los enunciados? (*Escala 1-5*)
- ¿Te resultó útil la pestaña “Resultado Esperado” para entender qué datos tenías que conseguir?
 - Sí, la he utilizado y me ha parecido útil.
 - La he utilizado pero no me ha resultado útil.
 - No me hizo falta utilizarla.
- ¿Te pareció útil la pestaña “Pista” para resolver los ejercicios?
 - Sí, la he utilizado y me ha parecido útil.
 - La he utilizado pero no me ha resultado útil.
 - No me hizo falta utilizarla.
- ¿Cómo valorarías la curva de dificultad y la progresión al avanzar? (*Escala 1-5*)

Dimensión 5: Satisfacción global y fidelización

- ¿Qué es lo que MÁS te ha gustado de SeeQL? (*Respuesta abierta*)
- ¿Qué es lo que MENOS te ha gustado o qué cambiarías para mejorarlo? (*Respuesta abierta*)
- Si existieran más niveles, ¿seguirías utilizándolo para aprender?
 - Sí, sin duda.
 - Tal vez.
 - No, no creo.

B.2 Cuestionario para docentes (validación experta)

El siguiente instrumento se basa en la métrica ágil UMUX aplicada a entornos de evaluación pedagógica.

Perfil docente

- ¿Cuántos años de experiencia tienes impartiendo docencia en Bases de Datos o SQL?
 - Menos de 1 año.
 - Entre 1 año y 5 años.
 - Entre 5 y 10 años.
 - Más de 10 años.

Métricas UMUX y utilidad pedagógica

- **[Cumplimiento de requisitos]** Las capacidades de SeeQL cumplen con mis requisitos docentes; es decir, permite a los alumnos poner en práctica el SQL y entender visualmente el impacto de sus consultas. *(Escala 1-5)*
- **[Experiencia fluida]** (*) Usar SeeQL resulta en una experiencia frustrante. *(Escala 1-5)*
- **[Intuitividad]** En general, creo que SeeQL es una herramienta fácil e intuitiva de usar. *(Escala 1-5)*
- **[Carga operativa]** (*) Siento que tengo que dedicar demasiado tiempo a corregir cosas o pelear con la interfaz de SeeQL en lugar de centrarme en el código SQL. *(Escala 1-5)*

Feedback cualitativo y oportunidades de integración

- ¿Qué es lo que más te ha gustado de SeeQL para hacer docencia en bases de datos? *(Respuesta abierta)*
- ¿Qué cambiarías, añadirías o mejorarías antes de usarlo con tus alumnos? *(Respuesta abierta)*

APÉNDICE C

Manual de instalación

Este anexo detalla los pasos necesarios para configurar el entorno de desarrollo, instalar las dependencias y desplegar la plataforma **SeeQL** en un entorno local o de producción.

C.1 Requisitos previos

Antes de iniciar la instalación, es necesario contar con las siguientes herramientas instaladas en el sistema:

- **Git:** Para el control de versiones y clonación del repositorio.
- **Node.js** (Versión 18.0 o superior): Entorno de ejecución para JavaScript.
- **npm** o **pnpm:** Gestores de paquetes para la instalación de dependencias.
- **Cuenta en Supabase:** Necesaria para la gestión de la base de datos en la nube y el sistema de autenticación.

C.2 Clonación del repositorio

El primer paso consiste en obtener una copia local del código fuente mediante el siguiente comando en la terminal:

```
git clone https://github.com/magicnube/seeql.git
cd seeql
```

C.3 Instalación de dependencias

Una vez dentro del directorio del proyecto, se deben instalar las librerías necesarias (incluyendo React, Lucide-React, React Flow, SQL.js y Tailwind CSS) ejecutando:

```
npm install
```

C.4 Configuración de variables de entorno

Para que la aplicación pueda comunicarse con los servicios de autenticación y la base de datos remota del modo *Sandbox*, se debe crear un archivo llamado `.env` en la raíz del proyecto. Este archivo debe contener las credenciales del proyecto de Supabase:

```
VITE_SUPABASE_URL=url_de_supabase  
VITE_SUPABASE_ANON_KEY=clave_anonima_de_supabase
```

Nota: Estas claves se encuentran en el panel de configuración (Project Settings > API) de la consola de Supabase.

C.5 Ejecución en entorno de desarrollo

Para levantar el servidor de desarrollo local y previsualizar la aplicación, se utiliza el siguiente comando:

```
npm run dev
```

Por defecto, la aplicación estará disponible en la dirección `http://localhost:5173`.

C.6 Compilación para producción

Si se desea generar una versión optimizada para su despliegue en un servidor web (como Vercel, Netlify o un servidor Apache), se debe ejecutar el proceso de compilación:

```
npm run build
```

Este comando generará una carpeta llamada `dist/` que contiene todos los archivos estáticos (HTML, CSS y JS) listos para ser servidos.

C.7 Consideraciones sobre WebAssembly

Dado que SeeQL utiliza SQL.js para el motor local de las lecciones, es fundamental asegurar que los archivos `.wasm` se carguen correctamente. En el entorno de desarrollo basado en Vite, la configuración ya incluye los *plugins* necesarios para servir estos archivos binarios con los tipos MIME adecuados.

APÉNDICE D

Alineación con los Objetivos de Desarrollo Sostenible

En cumplimiento con la normativa de la Escuela Técnica Superior de Ingeniería Informática (ETSINF), este anexo detalla la contribución del presente TFG a los Objetivos de Desarrollo Sostenible (ODS).

La plataforma **SeeQL** impacta de forma directa en la democratización del conocimiento técnico y la eficiencia tecnológica. A continuación, se presenta la matriz de evaluación de impacto para cada uno de los 17 objetivos prioritarios:

Objetivos de Desarrollo Sostenible	Alto	Medio	Bajo	N/A
1. Fin de la pobreza.				X
2. Hambre cero.				X
3. Salud y bienestar.				X
4. Educación de calidad.	X			
5. Igualdad de género.				X
6. Agua limpia y saneamiento.				X
7. Energía asequible y no contaminante.				X
8. Trabajo decente y crecimiento económico.			X	
9. Industria, innovación e infraestructuras.	X			
10. Reducción de las desigualdades.		X		
11. Ciudades y comunidades sostenibles.				X
12. Producción y consumo responsables.				X
13. Acción por el clima.				X
14. Vida submarina.				X
15. Vida de ecosistemas terrestres.				X
16. Paz, justicia e instituciones sólidas.				X
17. Alianzas para lograr objetivos.				X

Tabla D.1: Matriz de impacto de SeeQL en los Objetivos de Desarrollo Sostenible (Agenda 2030).

Justificación de los impactos principales

Tal y como se desprende de la tabla anterior, el proyecto SeeQL focaliza su contribución en tres ejes fundamentales:

- **ODS 4 (Educación de calidad) - Impacto alto:** La herramienta se ha diseñado como un recurso educativo abierto que elimina las barreras técnicas de acceso al aprendizaje de bases de datos, permitiendo a cualquier usuario con un navegador web desarrollar competencias tecnológicas de alta demanda.
- **ODS 9 (Industria, innovación e infraestructuras) - Impacto alto:** El uso de tecnologías de vanguardia como WebAssembly y arquitecturas basadas en *client-side computing* demuestra una innovación en la infraestructura de software educativo, optimizando el rendimiento y la escalabilidad del sistema.
- **ODS 10 (Reducción de las desigualdades) - Impacto medio:** Al descentralizar la ejecución relacional mediante ejecución en el cliente, la plataforma minimiza la dependencia de una conexión constante a la red. Esto permite que el sistema sea plenamente funcional en entornos con conectividad limitada o infraestructuras de red básicas, mitigando de forma efectiva la brecha digital en el acceso a herramientas de aprendizaje avanzadas.

APÉNDICE E

Licencia de software

El código fuente desarrollado para la plataforma **SeeQL** se ha concebido con un propósito fundamentalmente educativo y académico. Para fomentar la filosofía del software libre (*open source*) y permitir que futuros estudiantes, docentes o desarrolladores puedan analizar, utilizar y mejorar esta herramienta, el proyecto se distribuye bajo los términos de la **Licencia MIT**.

Se ha seleccionado esta licencia por ser una de las más permisivas y estandarizadas en el ecosistema de desarrollo web moderno. Su adopción facilita la integración de SeeQL en futuros proyectos académicos o derivados sin imponer restricciones, garantizando al mismo tiempo la autoría y eximiendo de responsabilidad legal al creador original.

A continuación, se reproduce el texto legal íntegro que acompaña al repositorio del proyecto:

MIT License

Copyright (c) 2026 Jesús Caravaca

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.